# EBA Framework

1.0

Generated on Tue Apr 15 2025 12:44:58 for EBA Framework by Doxygen 1.9.8

Tue Apr 15 2025 12:44:58

**1 Class Index** 1
1.1 Class List . . . 1

**2 File Index** 3
2.1 File List . . . 3

**3 Class Documentation** 5
3.1 eba_alloc_data Struct Reference . . . 5
3.1.1 Detailed Description . . . 5
3.1.2 Member Data Documentation . . . 5
3.1.2.1 buff_id . . . 5
3.1.2.2 life_time . . . 5
3.1.2.3 size . . . 6
3.2 eba_buffer Struct Reference . . . 6
3.2.1 Detailed Description . . . 6
3.2.2 Member Data Documentation . . . 6
3.2.2.1 address . . . 6
3.2.2.2 expires . . . 6
3.2.2.3 lock . . . 7
3.2.2.4 node . . . 7
3.2.2.5 size . . . 7
3.2.2.6 type . . . 7
3.3 eba_remote_alloc Struct Reference . . . 7
3.3.1 Detailed Description . . . 8
3.3.2 Member Data Documentation . . . 8
3.3.2.1 buffer_id . . . 8
3.3.2.2 life_time . . . 8
3.3.2.3 mac . . . 8
3.3.2.4 size . . . 8
3.4 eba_remote_read Struct Reference . . . 8
3.4.1 Detailed Description . . . 9
3.4.2 Member Data Documentation . . . 9
3.4.2.1 dst_buffer_id . . . 9
3.4.2.2 dst_offset . . . 9
3.4.2.3 mac . . . 9
3.4.2.4 size . . . 10
3.4.2.5 src_buffer_id . . . 10
3.4.2.6 src_offset . . . 10
3.5 eba_remote_write Struct Reference . . . 10
3.5.1 Detailed Description . . . 10
3.5.2 Member Data Documentation . . . 11
3.5.2.1 buff_id . . . 11
3.5.2.2 mac . . . 11

3.5.2.3 offset . . . 11
3.5.2.4 payload . . . 11
3.5.2.5 size . . . 11
3.6 eba_rw_data Struct Reference . . . 11
3.6.1 Detailed Description . . . 12
3.6.2 Member Data Documentation . . . 12
3.6.2.1 buff_id . . . 12
3.6.2.2 off . . . 12
3.6.2.3 size . . . 12
3.6.2.4 user_addr . . . 12
3.7 ebp_discover_ack Struct Reference . . . 13
3.7.1 Detailed Description . . . 13
3.7.2 Member Data Documentation . . . 13
3.7.2.1 buffer_id . . . 13
3.7.2.2 header . . . 13
3.8 ebp_discover_req Struct Reference . . . 14
3.8.1 Detailed Description . . . 14
3.8.2 Member Data Documentation . . . 14
3.8.2.1 header . . . 14
3.8.2.2 mtu . . . 14
3.9 ebp_header Struct Reference . . . 15
3.9.1 Detailed Description . . . 15
3.9.2 Member Data Documentation . . . 15
3.9.2.1 msgType . . . 15
3.10 ebp_invoke_ack Struct Reference . . . 15
3.10.1 Detailed Description . . . 16
3.10.2 Member Data Documentation . . . 16
3.10.2.1 header . . . 16
3.10.2.2 status . . . 16
3.11 ebp_invoke_req Struct Reference . . . 16
3.11.1 Detailed Description . . . 17
3.11.2 Member Data Documentation . . . 17
3.11.2.1 args . . . 17
3.11.2.2 args_len . . . 17
3.11.2.3 header . . . 17
3.11.2.4 iid . . . 17
3.11.2.5 opid . . . 18
3.12 ebp_op_alloc_args Struct Reference . . . 18
3.12.1 Detailed Description . . . 18
3.12.2 Member Data Documentation . . . 18
3.12.2.1 buffer_id . . . 18
3.12.2.2 life_time . . . 18

3.12.2.3 size . . . 19
3.13 ebp_op_read_args Struct Reference . . . 19
3.13.1 Detailed Description . . . 19
3.13.2 Member Data Documentation . . . 19
3.13.2.1 dst_buffer_id . . . 19
3.13.2.2 dst_offset . . . 19
3.13.2.3 size . . . 20
3.13.2.4 src_buffer_id . . . 20
3.13.2.5 src_offset . . . 20
3.14 ebp_op_write_args Struct Reference . . . 20
3.14.1 Detailed Description . . . 20
3.14.2 Member Data Documentation . . . 20
3.14.2.1 buff_id . . . 20
3.14.2.2 offset . . . 21
3.14.2.3 size . . . 21
3.15 <Ethernet> Protocol Reference . . . 21
3.15.1 Detailed Description . . . 21
3.16 invoke_tracker Struct Reference . . . 21
3.16.1 Detailed Description . . . 22
3.16.2 Member Data Documentation . . . 22
3.16.2.1 done . . . 22
3.16.2.2 iid . . . 22
3.16.2.3 pid . . . 22
3.16.2.4 status . . . 22
3.17 node_info Struct Reference . . . 22
3.17.1 Detailed Description . . . 23
3.17.2 Member Data Documentation . . . 23
3.17.2.1 id . . . 23
3.17.2.2 mac . . . 23
3.17.2.3 mtu . . . 23
3.17.2.4 node_specs . . . 23
3.18 op_entry Struct Reference . . . 24
3.18.1 Detailed Description . . . 24
3.18.2 Member Data Documentation . . . 24
3.18.2.1 op_id . . . 24
3.18.2.2 op_ptr . . . 24

**4 File Documentation** **25**
4.1 include/eba.h File Reference . . . 25
4.1.1 Detailed Description . . . 26
4.1.2 Macro Definition Documentation . . . 26
4.1.2.1 EBA_CLEAN_BUFFER_CALLBACK_TIMER . . . 26

4.1.2.2 EBA_ERR . . . 26
4.1.2.3 EBA_INFO . . . 26
4.1.2.4 EBA_IOC_MAGIC . . . 27
4.1.2.5 EBA_IOC_MAXNR . . . 27
4.1.2.6 EBA_IOCTL_ALLOC . . . 27
4.1.2.7 EBA_IOCTL_READ . . . 27
4.1.2.8 EBA_IOCTL_REMOTE_ALLOC . . . 27
4.1.2.9 EBA_IOCTL_REMOTE_READ . . . 28
4.1.2.10 EBA_IOCTL_REMOTE_WRITE . . . 28
4.1.2.11 EBA_IOCTL_WRITE . . . 28
4.1.2.12 EBA_WARN . . . 28
4.2 eba.h . . . 29
4.3 include/eba_internals.h File Reference . . . 30
4.3.1 Detailed Description . . . 31
4.3.2 Macro Definition Documentation . . . 31
4.3.2.1 MEMORY_POOL_SIZE . . . 31
4.3.3 Function Documentation . . . 31
4.3.3.1 eba_check_expired_buffers() . . . 31
4.3.3.2 eba_internals_free() . . . 32
4.3.3.3 eba_internals_malloc() . . . 32
4.3.3.4 eba_internals_mem_stress_test() . . . 34
4.3.3.5 eba_internals_mempool_free() . . . 35
4.3.3.6 eba_internals_mempool_init() . . . 35
4.3.3.7 eba_internals_read() . . . 36
4.3.3.8 eba_internals_rw_stress_test() . . . 37
4.3.3.9 eba_internals_write() . . . 38
4.4 eba_internals.h . . . 39
4.5 include/eba_net.h File Reference . . . 40
4.5.1 Detailed Description . . . 41
4.5.2 Function Documentation . . . 41
4.5.2.1 build_discover_ack_packet() . . . 41
4.5.2.2 build_discover_req_packet() . . . 42
4.5.2.3 build_invoke_ack_packet() . . . 42
4.5.2.4 build_invoke_req_packet() . . . 43
4.5.2.5 eba_net_get_max_mtu() . . . 43
4.5.2.6 eba_net_set_mtu() . . . 44
4.5.2.7 send_discover_ack_packet() . . . 44
4.5.2.8 send_discover_req_packet() . . . 45
4.5.2.9 send_invoke_ack_packet() . . . 45
4.5.2.10 send_invoke_req_packet() . . . 46
4.5.2.11 send_raw_ethernet_packet() . . . 47
4.6 eba_net.h . . . 48

4.7 include/eba_user.h File Reference . . . 48
4.7.1 Detailed Description . . . 49
4.7.2 Function Documentation . . . 49
4.7.2.1 eba_alloc() . . . 49
4.7.2.2 eba_read() . . . 50
4.7.2.3 eba_remote_alloc() . . . 50
4.7.2.4 eba_remote_read() . . . 51
4.7.2.5 eba_remote_write() . . . 52
4.7.2.6 eba_write() . . . 52
4.8 eba_user.h . . . 53
4.9 include/eba_utils.h File Reference . . . 53
4.9.1 Detailed Description . . . 54
4.9.2 Function Documentation . . . 55
4.9.2.1 eba_export_node_specs() . . . 55
4.9.2.2 eba_utils_buf_to_file() . . . 56
4.9.2.3 eba_utils_file_to_buf() . . . 57
4.9.2.4 test_eba_export_node_specs() . . . 58
4.9.2.5 test_eba_utils_buf_to_file() . . . 58
4.9.2.6 test_eba_utils_file_to_buf() . . . 59
4.10 eba_utils.h . . . 60
4.11 include/ebp.h File Reference . . . 60
4.11.1 Detailed Description . . . 62
4.11.2 Macro Definition Documentation . . . 63
4.11.2.1 EBP_ETHERTYPE . . . 63
4.11.2.2 EBP_NODE_SPECS_MAX_LIFE_TIME . . . 63
4.11.2.3 EBP_NODE_SPECS_MAX_SIZE . . . 63
4.11.2.4 MAX_INVOKE_COUNT . . . 63
4.11.2.5 MAX_NODE_COUNT . . . 63
4.11.2.6 MAX_OP_COUNT . . . 63
4.11.3 Typedef Documentation . . . 64
4.11.3.1 ebp_op_t . . . 64
4.11.4 Enumeration Type Documentation . . . 64
4.11.4.1 EBP_MSG . . . 64
4.11.4.2 EBP_OP_IDS . . . 64
4.11.4.3 INVOKE_STATUS . . . 65
4.11.5 Function Documentation . . . 65
4.11.5.1 __attribute__() . . . 65
4.11.5.2 ebp_exit() . . . 65
4.11.5.3 ebp_handle_packets() . . . 66
4.11.5.4 ebp_init() . . . 68
4.11.5.5 ebp_invoke_op() . . . 68
4.11.5.6 ebp_op_alloc() . . . 69

4.11.5.7 ebp_op_read() . . . 69
4.11.5.8 ebp_op_write() . . . 70
4.11.5.9 ebp_ops_init() . . . 71
4.11.5.10 ebp_register_node() . . . 71
4.11.5.11 ebp_register_op() . . . 72
4.11.5.12 ebp_remote_alloc() . . . 72
4.11.5.13 ebp_remote_read() . . . 73
4.11.5.14 ebp_remote_write() . . . 74
4.11.5.15 invoke_tracker_array_init() . . . 74
4.11.5.16 node_info_array_init() . . . 75
4.11.5.17 op_entry_array_init() . . . 75
4.11.5.18 print_node_infos() . . . 76
4.11.5.19 print_op_entries() . . . 76
4.11.6 Variable Documentation . . . 76
4.11.6.1 args . . . 76
4.11.6.2 args_len . . . 76
4.11.6.3 buff_id . . . 77
4.11.6.4 buffer_id . . . 77
4.11.6.5 dst_buffer_id . . . 77
4.11.6.6 dst_offset . . . 77
4.11.6.7 header . . . 77
4.11.6.8 iid . . . 77
4.11.6.9 life_time . . . 77
4.11.6.10 msgType . . . 77
4.11.6.11 mtu . . . 78
4.11.6.12 offset . . . 78
4.11.6.13 opid . . . 78
4.11.6.14 size . . . 78
4.11.6.15 src_buffer_id . . . 78
4.11.6.16 src_offset . . . 78
4.11.6.17 status . . . 78
4.12 ebp.h . . . 79
4.13 src/eba.c File Reference . . . 81
4.13.1 Macro Definition Documentation . . . 82
4.13.1.1 DEBUG . . . 82
4.13.2 Function Documentation . . . 82
4.13.2.1 eba_ioctl() . . . 82
4.13.2.2 eba_module_exit() . . . 83
4.13.2.3 eba_module_init() . . . 83
4.13.2.4 eba_open() . . . 84
4.13.2.5 eba_release() . . . 84
4.13.2.6 eba_timer_callback() . . . 84

4.13.2.7 MODULE_AUTHOR() . . . 84
4.13.2.8 MODULE_DESCRIPTION() . . . 85
4.13.2.9 module_exit() . . . 85
4.13.2.10 module_init() . . . 85
4.13.2.11 MODULE_LICENSE() . . . 85
4.13.2.12 MODULE_VERSION() . . . 85
4.13.3 Variable Documentation . . . 85
4.13.3.1 eba_cdev . . . 85
4.13.3.2 eba_class . . . 85
4.13.3.3 eba_devno . . . 85
4.13.3.4 eba_fops . . . 86
4.13.3.5 eba_timer . . . 86
4.14 eba.c . . . 86
4.15 src/eba_internals.c File Reference . . . 89
4.15.1 Function Documentation . . . 89
4.15.1.1 eba_check_expired_buffers() . . . 89
4.15.1.2 eba_internals_free() . . . 90
4.15.1.3 eba_internals_malloc() . . . 91
4.15.1.4 eba_internals_mem_stress_test() . . . 92
4.15.1.5 eba_internals_mempool_free() . . . 93
4.15.1.6 eba_internals_mempool_init() . . . 93
4.15.1.7 eba_internals_read() . . . 94
4.15.1.8 eba_internals_rw_stress_test() . . . 95
4.15.1.9 eba_internals_write() . . . 96
4.15.1.10 LIST_HEAD() . . . 97
4.15.2 Variable Documentation . . . 97
4.15.2.1 eba_buffer_list_lock . . . 97
4.15.2.2 eba_pool . . . 97
4.15.2.3 eba_pool_mem . . . 97
4.16 eba_internals.c . . . 98
4.17 src/eba_net.c File Reference . . . 103
4.17.1 Function Documentation . . . 104
4.17.1.1 build_discover_ack_packet() . . . 104
4.17.1.2 build_discover_req_packet() . . . 104
4.17.1.3 build_invoke_ack_packet() . . . 105
4.17.1.4 build_invoke_req_packet() . . . 105
4.17.1.5 eba_net_get_max_mtu() . . . 106
4.17.1.6 eba_net_set_mtu() . . . 106
4.17.1.7 send_discover_ack_packet() . . . 107
4.17.1.8 send_discover_req_packet() . . . 107
4.17.1.9 send_invoke_ack_packet() . . . 108
4.17.1.10 send_invoke_req_packet() . . . 108

4.17.1.11 send_raw_ethernet_packet() . . . 109
4.18 eba_net.c . . . 110
4.19 src/eba_user.c File Reference . . . 113
4.19.1 Function Documentation . . . 114
4.19.1.1 eba_alloc() . . . 114
4.19.1.2 eba_read() . . . 114
4.19.1.3 eba_remote_alloc() . . . 115
4.19.1.4 eba_remote_read() . . . 116
4.19.1.5 eba_remote_write() . . . 116
4.19.1.6 eba_write() . . . 117
4.19.1.7 open_eba_device() . . . 118
4.20 eba_user.c . . . 118
4.21 src/eba_utils.c File Reference . . . 120
4.21.1 Function Documentation . . . 121
4.21.1.1 eba_export_node_specs() . . . 121
4.21.1.2 eba_utils_buf_to_file() . . . 122
4.21.1.3 eba_utils_file_to_buf() . . . 122
4.21.1.4 test_eba_export_node_specs() . . . 123
4.21.1.5 test_eba_utils_buf_to_file() . . . 124
4.21.1.6 test_eba_utils_file_to_buf() . . . 125
4.21.2 Variable Documentation . . . 126
4.21.2.1 eba_buffer_list . . . 126
4.21.2.2 eba_buffer_list_lock . . . 126
4.21.2.3 node_infos . . . 126
4.21.2.4 nodes_count . . . 126
4.22 eba_utils.c . . . 126
4.23 src/ebp.c File Reference . . . 130
4.23.1 Macro Definition Documentation . . . 131
4.23.1.1 INVALID_NODE_ID . . . 131
4.23.2 Function Documentation . . . 131
4.23.2.1 ebp_exit() . . . 131
4.23.2.2 ebp_handle_packets() . . . 132
4.23.2.3 ebp_init() . . . 134
4.23.2.4 ebp_invoke_op() . . . 134
4.23.2.5 ebp_op_alloc() . . . 135
4.23.2.6 ebp_op_read() . . . 135
4.23.2.7 ebp_op_write() . . . 136
4.23.2.8 ebp_ops_init() . . . 137
4.23.2.9 ebp_register_node() . . . 137
4.23.2.10 ebp_register_op() . . . 138
4.23.2.11 ebp_remote_alloc() . . . 138
4.23.2.12 ebp_remote_read() . . . 139

4.23.2.13 ebp_remote_write() . . . 140
4.23.2.14 invoke_tracker_array_init() . . . 140
4.23.2.15 node_info_array_init() . . . 141
4.23.2.16 op_entry_array_init() . . . 141
4.23.2.17 print_node_infos() . . . 142
4.23.2.18 print_op_entries() . . . 142
4.23.3 Variable Documentation . . . 142
4.23.3.1 ebp_packet_type . . . 142
4.23.3.2 invoke_trackers . . . 143
4.23.3.3 local_specs . . . 143
4.23.3.4 node_infos . . . 143
4.23.3.5 nodes_count . . . 143
4.23.3.6 op_entries . . . 143
4.24 ebp.c . . . 143
4.25 user_space/user.c File Reference . . . 149
4.25.1 Function Documentation . . . 149
4.25.1.1 main() . . . 149
4.26 user.c . . . 150

**Index** **151**

# Chapter 1

# Class Index

## 1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:


eba_alloc_data . . . . . 5
eba_buffer . . . . . 6
eba_remote_alloc . . . . . 7
eba_remote_read . . . . . 8
eba_remote_write . . . . . 10
eba_rw_data . . . . . 11
ebp_discover_ack . . . . . 13
ebp_discover_req . . . . . 14
ebp_header . . . . . 15
ebp_invoke_ack . . . . . 15
ebp_invoke_req . . . . . 16
ebp_op_alloc_args . . . . . 18
ebp_op_read_args . . . . . 19
ebp_op_write_args . . . . . 20
<Ethernet> . . . . . 21
invoke_tracker . . . . . 21
node_info . . . . . 22
op_entry . . . . . 24

# Chapter 2

# File Index

## 2.1 File List

Here is a list of all files with brief descriptions:


include/eba.h
EBA IOCTL interface header . . . . . 25
include/eba_internals.h
EBA Internal Memory Management Header . . . . . 30
include/eba_net.h
EBA Networking Interface Header . . . . . 40
include/eba_user.h
EBA User API Header . . . . . 48
include/eba_utils.h
EBA Utilities for File and Buffer Operations . . . . . 53
include/ebp.h
EBA Protocol Header File . . . . . 60
src/eba.c . . . . . 81
src/eba_internals.c . . . . . 89
src/eba_net.c . . . . . 103
src/eba_user.c . . . . . 113
src/eba_utils.c . . . . . 120
src/ebp.c . . . . . 130
user_space/user.c . . . . . 149

# Chapter 3

# Class Documentation

## 3.1 eba_alloc_data Struct Reference

```
#include <eba.h>
```

**Public Attributes**

- __u64 size
- __u64 life_time
- __u64 buff_id

### 3.1.1 Detailed Description

struct eba_alloc_data - Structure for buffer allocation via IOCTL. @size: Number of bytes to allocate. @life_time: Lifetime for the buffer allocation in seconds. @buff_id: Returned buffer identifier (virtual address of the allocated buffer).

This structure is used by user-space applications to request a buffer allocation. The user specifies the desired size and lifetime; upon success, the driver returns a buffer identifier that represents a virtual memory address.

Definition at line 93 of file eba.h.

### 3.1.2 Member Data Documentation

#### 3.1.2.1 buff_id

```
__u64 eba_alloc_data::buff_id
```

Definition at line 96 of file eba.h.

#### 3.1.2.2 life_time

```
__u64 eba_alloc_data::life_time
```

Definition at line 95 of file eba.h.

#### 3.1.2.3 size

```
__u64 eba_alloc_data::size
```

Definition at line 94 of file eba.h.

The documentation for this struct was generated from the following file:

- include/eba.h

## 3.2 eba_buffer Struct Reference

```
#include <eba_internals.h>
```

**Public Attributes**

- spinlock_t lock
- uint8_t type
- uint64_t address
- uint64_t size
- ktime_t expires
- struct list_head node

### 3.2.1 Detailed Description

struct eba_buffer - Tracking structure for each allocated memory chunk. @lock: Spinlock to protect this buffer's data. @type: Buffer type. @address: Address of the allocated memory block. @size: Requested allocation size in bytes. @expires: Time when the allocation expires. @node: List node to chain all allocations.

This structure tracks each memory allocation made from the EBA memory pool.

Definition at line 37 of file eba_internals.h.

### 3.2.2 Member Data Documentation

#### 3.2.2.1 address

```
uint64_t eba_buffer::address
```

Definition at line 40 of file eba_internals.h.

#### 3.2.2.2 expires

```
ktime_t eba_buffer::expires
```

Definition at line 42 of file eba_internals.h.

#### 3.2.2.3 lock

```
spinlock_t eba_buffer::lock
```

Definition at line 38 of file eba_internals.h.

#### 3.2.2.4 node

```
struct list_head eba_buffer::node
```

Definition at line 43 of file eba_internals.h.

#### 3.2.2.5 size

```
uint64_t eba_buffer::size
```

Definition at line 41 of file eba_internals.h.

#### 3.2.2.6 type

```
uint8_t eba_buffer::type
```

Definition at line 39 of file eba_internals.h.

The documentation for this struct was generated from the following file:

- include/eba_internals.h

## 3.3 eba_remote_alloc Struct Reference

```
#include <eba.h>
```

### Public Attributes

- __u64 size
- __u64 life_time
- __u64 buffer_id
- const char mac [6]

### 3.3.1 Detailed Description

struct eba_remote_alloc - Structure for remote buffer allocation via IOCTL. @size: Number of bytes to allocate on the remote node. @life_time: Lifetime for the remote buffer allocation in seconds. @buffer_id: Buffer id that will store the remotely allocated buffer. @mac: MAC address of the remote node.

This structure is used to request a buffer allocation on a remote node. The MAC address field identifies the target node.

**Note**

TODO: Modify this field to use a node ID instead of a MAC address.

Definition at line 129 of file eba.h.

### 3.3.2 Member Data Documentation

#### 3.3.2.1 buffer_id

```
__u64 eba_remote_alloc::buffer_id
```

Definition at line 132 of file eba.h.

#### 3.3.2.2 life_time

```
__u64 eba_remote_alloc::life_time
```

Definition at line 131 of file eba.h.

#### 3.3.2.3 mac

```
const char eba_remote_alloc::mac[6]
```

Definition at line 133 of file eba.h.

#### 3.3.2.4 size

```
__u64 eba_remote_alloc::size
```

Definition at line 130 of file eba.h.

The documentation for this struct was generated from the following file:

- include/eba.h

## 3.4 eba_remote_read Struct Reference

```
#include <eba.h>
```

**Public Attributes**

- __u64 dst_buffer_id
- __u64 src_buffer_id
- __u64 dst_offset
- __u64 src_offset
- __u64 size
- const char mac [6]

### 3.4.1 Detailed Description

struct eba_remote_read - Structure for remote buffer read operations via IOCTL. @dst_buffer_id: Identifier for the destination (local) buffer where data will be stored. @src_buffer_id: Identifier for the source (remote) buffer from which data is to be read. @dst_offset: Offset within the destination buffer where the data should be written. @src↩_offset: Offset within the source buffer where the reading should begin. @size: Number of bytes to read. @mac: MAC address of the remote node.

This structure is used to request a read operation from a remote node. It specifies both the source (remote) buffer and the destination (local) buffer along with the respective offsets and the size of data to be transferred.

**Note**

TODO: Modify the MAC address field to use a node ID instead.

Definition at line 172 of file eba.h.

### 3.4.2 Member Data Documentation

#### 3.4.2.1 dst_buffer_id

```
__u64 eba_remote_read::dst_buffer_id
```

Definition at line 173 of file eba.h.

#### 3.4.2.2 dst_offset

```
__u64 eba_remote_read::dst_offset
```

Definition at line 175 of file eba.h.

#### 3.4.2.3 mac

```
const char eba_remote_read::mac[6]
```

Definition at line 178 of file eba.h.

#### 3.4.2.4 size

```
__u64 eba_remote_read::size
```

Definition at line 177 of file eba.h.

#### 3.4.2.5 src_buffer_id

```
__u64 eba_remote_read::src_buffer_id
```

Definition at line 174 of file eba.h.

#### 3.4.2.6 src_offset

```
__u64 eba_remote_read::src_offset
```

Definition at line 176 of file eba.h.

The documentation for this struct was generated from the following file:

- include/eba.h

## 3.5 eba_remote_write Struct Reference

```
#include <eba.h>
```

**Public Attributes**

- __u64 buff_id
- __u64 offset
- __u64 size
- char * payload
- const char mac [6]

### 3.5.1 Detailed Description

struct eba_remote_write - Structure for remote buffer write operations via IOCTL. @buff_id: Remote buffer identifier. @offset: Offset within the remote buffer where the write operation should begin. @size: Number of bytes to write. @payload: Pointer to the data payload to be written. @mac: MAC address of the remote node.

This structure is used to send a write request to a remote node by specifying the target buffer, the offset within that buffer, and the data to be written.

**Note**

TODO: Modify the MAC address field to use a node ID instead.

Definition at line 149 of file eba.h.

### 3.5.2 Member Data Documentation

#### 3.5.2.1 buff_id

```
__u64 eba_remote_write::buff_id
```

Definition at line 150 of file eba.h.

#### 3.5.2.2 mac

```
const char eba_remote_write::mac[6]
```

Definition at line 154 of file eba.h.

#### 3.5.2.3 offset

```
__u64 eba_remote_write::offset
```

Definition at line 151 of file eba.h.

#### 3.5.2.4 payload

```
char* eba_remote_write::payload
```

Definition at line 153 of file eba.h.

#### 3.5.2.5 size

```
__u64 eba_remote_write::size
```

Definition at line 152 of file eba.h.

The documentation for this struct was generated from the following file:

- include/eba.h

## 3.6 eba_rw_data Struct Reference

```
#include <eba.h>
```

#### Public Attributes

- __u64 buff_id
- __u64 off
- __u64 size
- __u64 user_addr

### 3.6.1 Detailed Description

struct eba_rw_data - Structure for read and write operations via IOCTL. @buff_id: Buffer identifier (virtual address returned by an allocation call). @off: Offset within the buffer where the operation begins. @size: Number of bytes to read or write. @user_addr: User-space pointer (represented as a 64-bit value) for data transfer.

This structure is used to perform read and write operations on a previously allocated buffer. It contains the necessary parameters to identify the buffer location and the data size, as well as the user-space address for transferring data.

Definition at line 110 of file eba.h.

### 3.6.2 Member Data Documentation

#### 3.6.2.1 buff_id

```
__u64 eba_rw_data::buff_id
```

Definition at line 111 of file eba.h.

#### 3.6.2.2 off

```
__u64 eba_rw_data::off
```

Definition at line 112 of file eba.h.

#### 3.6.2.3 size

```
__u64 eba_rw_data::size
```

Definition at line 113 of file eba.h.

#### 3.6.2.4 user_addr

```
__u64 eba_rw_data::user_addr
```

Definition at line 114 of file eba.h.

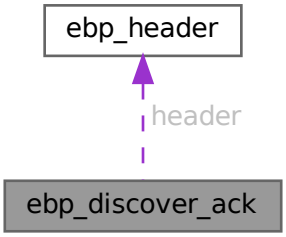
The documentation for this struct was generated from the following file:

- include/eba.h

## 3.7 ebp_discover_ack Struct Reference

```
#include <ebp.h>
```

Collaboration diagram for ebp_discover_ack:




**Public Attributes**

- struct ebp_header header
- uint64_t buffer_id

### 3.7.1 Detailed Description

struct ebp_discover_ack - Structure for an EBA Discover Acknowledgment message. @header: Header containing the message type (should be set to EBP_MSG_DISCOVER_ACK). @buffer_id: Identifier of the buffer containing node information.

The receiving node sends this message in response to a discovery request to provide the sender with the necessary node information.

Definition at line 254 of file ebp.h.

### 3.7.2 Member Data Documentation

#### 3.7.2.1 buffer_id

```
uint64_t ebp_discover_ack::buffer_id
```

Definition at line 256 of file ebp.h.

#### 3.7.2.2 header

```
struct ebp_header ebp_discover_ack::header
```

Definition at line 255 of file ebp.h.

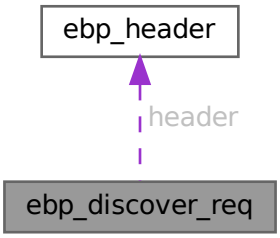
The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.8 ebp_discover_req Struct Reference

```
#include <ebp.h>
```

Collaboration diagram for ebp_discover_req:




**Public Attributes**

- struct ebp_header header
- uint16_t mtu

### 3.8.1 Detailed Description

struct ebp_discover_req - Structure for an EBA Discover Request message. @header: Header containing the message type (should be set to EBP_MSG_DISCOVER). @mtu: Maximum Transmission Unit of the sender.

This message is broadcast by a node during initialization to discover neighboring nodes. It can be exposed as an API also.

Definition at line 241 of file ebp.h.

### 3.8.2 Member Data Documentation

#### 3.8.2.1 header

```
struct ebp_header ebp_discover_req::header
```

Definition at line 242 of file ebp.h.

#### 3.8.2.2 mtu

```
uint16_t ebp_discover_req::mtu
```

Definition at line 243 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.9 ebp_header Struct Reference

```
#include <ebp.h>
```

**Public Attributes**

- uint8_t msgType

### 3.9.1 Detailed Description

struct ebp_header - Minimal header for EBA protocol messages. @msgType: Indicates the type of EBA message (see enum EBP_MSG).

This header is placed immediately after the Ethernet header in an EBA frame. It allows nodes to determine the processing logic based on the message type.

Definition at line 229 of file ebp.h.

### 3.9.2 Member Data Documentation

#### 3.9.2.1 msgType

```
uint8_t ebp_header::msgType
```

Definition at line 230 of file ebp.h.

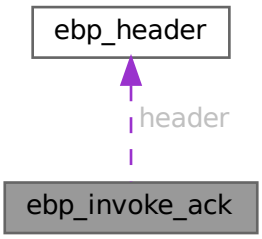
The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.10 ebp_invoke_ack Struct Reference

```
#include <ebp.h>
```

Collaboration diagram for ebp_invoke_ack:



**Public Attributes**

- struct ebp_header header
- uint8_t status

### 3.10.1 Detailed Description

struct ebp_invoke_ack - Structure for an EBA Invoke Acknowledgment message. @header: Header containing the message type (should be set to EBP_MSG_INVOKE_ACK). @status: Status code indicating the result of the invocation.

This message is used to acknowledge that an Invoke Request has been processed, reflecting whether the operation was queued, completed, or failed.

Definition at line 286 of file ebp.h.

### 3.10.2 Member Data Documentation

#### 3.10.2.1 header

```
struct ebp_header ebp_invoke_ack::header
```

Definition at line 287 of file ebp.h.

#### 3.10.2.2 status

```
uint8_t ebp_invoke_ack::status
```

Definition at line 288 of file ebp.h.

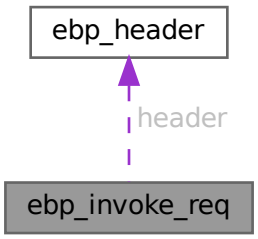
The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.11 ebp_invoke_req Struct Reference

```
#include <ebp.h>
```

Collaboration diagram for ebp_invoke_req:



**Public Attributes**

- struct ebp_header header
- uint32_t iid
- uint32_t opid
- uint64_t args_len
- uint8_t args [ ]

## 3.11.1 Detailed Description

struct ebp_invoke_req - Structure for an EBA Invoke Request message. @header: Header containing the message type. @iid: 32-bit Invocation ID. @opid: 32-bit Operation ID indicating the requested operation. @args_len: 64-bit length of the arguments blob. @args: Flexible array member containing the raw arguments.

This message is used to remotely invoke an operation on another node. The variable-length arguments are parsed within the handler function.

Definition at line 270 of file ebp.h.

## 3.11.2 Member Data Documentation

### 3.11.2.1 args

```
uint8_t ebp_invoke_req::args[]
```

Definition at line 275 of file ebp.h.

### 3.11.2.2 args_len

```
uint64_t ebp_invoke_req::args_len
```

Definition at line 274 of file ebp.h.

### 3.11.2.3 header

```
struct ebp_header ebp_invoke_req::header
```

Definition at line 271 of file ebp.h.

### 3.11.2.4 iid

```
uint32_t ebp_invoke_req::iid
```

Definition at line 272 of file ebp.h.

#### 3.11.2.5 opid

```
uint32_t ebp_invoke_req::opid
```

Definition at line 273 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.12 ebp_op_alloc_args Struct Reference

```
#include <ebp.h>
```

**Public Attributes**

- uint64_t size
- uint64_t life_time
- uint64_t buffer_id

### 3.12.1 Detailed Description

struct ebp_op_alloc_args - Arguments for the exposed EBA "alloc" operation. @size: Number of bytes to allocate. @life_time: Lifetime for the allocation (in a defined time unit). @buffer_id: Field that the target node will fill with the allocated buffer's unique identifier.

These parameters are provided by a remote node to request a buffer allocation.

Definition at line 196 of file ebp.h.

### 3.12.2 Member Data Documentation

#### 3.12.2.1 buffer_id

```
uint64_t ebp_op_alloc_args::buffer_id
```

Definition at line 199 of file ebp.h.

#### 3.12.2.2 life_time

```
uint64_t ebp_op_alloc_args::life_time
```

Definition at line 198 of file ebp.h.

#### 3.12.2.3 size

```
uint64_t ebp_op_alloc_args::size
```

Definition at line 197 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.13 ebp_op_read_args Struct Reference

```
#include <ebp.h>
```

**Public Attributes**

- uint64_t dst_buffer_id
- uint64_t src_buffer_id
- uint64_t dst_offset
- uint64_t src_offset
- uint64_t size

### 3.13.1 Detailed Description

struct ebp_op_read_args - Arguments for the exposed EBA "read" operation. @dst_buffer_id: Identifier (virtual address) of the destination buffer where data will be copied. @src_buffer_id: Identifier (virtual address) of the source buffer to read from. @dst_offset: Byte offset within the destination buffer. @src_offset: Byte offset within the source buffer to start reading. @size: Number of bytes to read.

This structure specifies the parameters for a remote read operation.

Definition at line 213 of file ebp.h.

### 3.13.2 Member Data Documentation

#### 3.13.2.1 dst_buffer_id

```
uint64_t ebp_op_read_args::dst_buffer_id
```

Definition at line 214 of file ebp.h.

#### 3.13.2.2 dst_offset

```
uint64_t ebp_op_read_args::dst_offset
```

Definition at line 216 of file ebp.h.

#### 3.13.2.3 size

```
uint64_t ebp_op_read_args::size
```

Definition at line 218 of file ebp.h.

#### 3.13.2.4 src_buffer_id

```
uint64_t ebp_op_read_args::src_buffer_id
```

Definition at line 215 of file ebp.h.

#### 3.13.2.5 src_offset

```
uint64_t ebp_op_read_args::src_offset
```

Definition at line 217 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.14 ebp_op_write_args Struct Reference

```
#include <ebp.h>
```

**Public Attributes**

- uint64_t buff_id
- uint64_t offset
- uint64_t size

### 3.14.1 Detailed Description

struct ebp_op_write_args - Arguments for the exposed EBA "write" operation. @buff_id: Identifier (virtual address) of the target buffer. @offset: Byte offset within the target buffer to begin writing. @size: Number of bytes to write.

This structure contains the parameters required for a write operation.

Definition at line 181 of file ebp.h.

### 3.14.2 Member Data Documentation

#### 3.14.2.1 buff_id

```
uint64_t ebp_op_write_args::buff_id
```

Definition at line 182 of file ebp.h.

#### 3.14.2.2 offset

```
uint64_t ebp_op_write_args::offset
```

Definition at line 183 of file ebp.h.

#### 3.14.2.3 size

```
uint64_t ebp_op_write_args::size
```

Definition at line 184 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.15 <Ethernet> Protocol Reference

### 3.15.1 Detailed Description

send_raw_ethernet_packet - Send a raw Ethernet packet. @payload: Pointer to the payload data to include in the packet. @payload_len: Length of the payload in bytes. @dst_mac: Pointer to the destination MAC address (typically a 6-byte array). (e.g., ETH_P_IP, ETH_P_ARP) in host byte order. @ifname: Network interface name (e.g., "eth0") through which the packet is transmitted.

This function allocates an sk_buff and constructs the Ethernet header by setting the destination and source MAC addresses as well as the specified protocol. It then appends the provided payload and transmits the packet using the appropriate network device.

Return: 0 on success or a negative error code on failure.

The documentation for this protocol was generated from the following file:

- include/eba_net.h

## 3.16 invoke_tracker Struct Reference

```
#include <ebp.h>
```

#### Public Attributes

- uint32_t iid
- pid_t pid
- bool done
- enum INVOKE_STATUS status

### 3.16.1 Detailed Description

struct invoke_tracker - Tracks the status of an EBA protocol invocation. @iid: Unique identifier for the invocation. @pid: Process identifier associated with the invocation. @done: Boolean flag indicating whether the invocation is complete. @status: Current status of the invocation (see INVOKE_STATUS).

Definition at line 70 of file ebp.h.

### 3.16.2 Member Data Documentation

#### 3.16.2.1 done

```
bool invoke_tracker::done
```

Definition at line 73 of file ebp.h.

#### 3.16.2.2 iid

```
uint32_t invoke_tracker::iid
```

Definition at line 71 of file ebp.h.

#### 3.16.2.3 pid

```
pid_t invoke_tracker::pid
```

Definition at line 72 of file ebp.h.

#### 3.16.2.4 status

```
enum INVOKE_STATUS invoke_tracker::status
```

Definition at line 74 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.17 node_info Struct Reference

```
#include <ebp.h>
```

**Public Attributes**

- uint16_t id
- uint16_t mtu
- char mac [6]
- uint64_t node_specs

## 3.17.1 Detailed Description

struct node_info - Contains metadata for an EBA node. @id: Unique identifier for the node. @mtu: Maximum transmission unit supported by the node. @mac: 6-byte MAC address of the node. @node_specs: Identifier for the buffer containing node specification details.

Definition at line 56 of file ebp.h.

## 3.17.2 Member Data Documentation

### 3.17.2.1 id

```
uint16_t node_info::id
```

Definition at line 57 of file ebp.h.

### 3.17.2.2 mac

```
char node_info::mac[6]
```

Definition at line 59 of file ebp.h.

### 3.17.2.3 mtu

```
uint16_t node_info::mtu
```

Definition at line 58 of file ebp.h.

### 3.17.2.4 node_specs

```
uint64_t node_info::node_specs
```

Definition at line 60 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

## 3.18 op_entry Struct Reference

```
#include <ebp.h>
```

**Public Attributes**

- uint32_t op_id
- ebp_op_t op_ptr

### 3.18.1 Detailed Description

struct op_entry - Represents an operation entry in the EBA protocol. @op_id: Unique operation identifier. @op_↩ ptr: Function pointer to the operation implementation.

This structure maps operation IDs to the corresponding function implementations that process remote EBA operations.

Definition at line 100 of file ebp.h.

### 3.18.2 Member Data Documentation

#### 3.18.2.1 op_id

```
uint32_t op_entry::op_id
```

Definition at line 101 of file ebp.h.

#### 3.18.2.2 op_ptr

```
ebp_op_t op_entry::op_ptr
```

Definition at line 102 of file ebp.h.

The documentation for this struct was generated from the following file:

- include/ebp.h

# Chapter 4

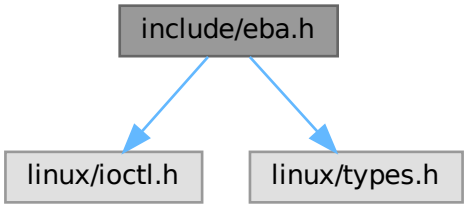
# File Documentation

## 4.1 include/eba.h File Reference

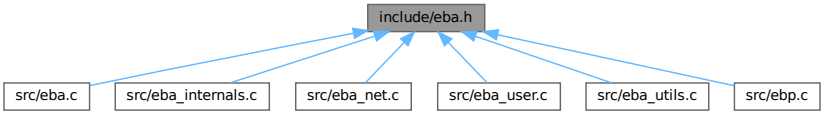
EBA IOCTL interface header.

```
#include <linux/ioctl.h>
#include <linux/types.h>
```

Include dependency graph for eba.h:




This graph shows which files directly or indirectly include this file:




### Classes

- struct eba_alloc_data
- struct eba_rw_data
- struct eba_remote_alloc
- struct eba_remote_write
- struct eba_remote_read

**Macros**

- #define EBA_IOC_MAGIC 'E'
- #define EBA_IOCTL_ALLOC _IOWR(EBA_IOC_MAGIC, 1, struct eba_alloc_data)
- #define EBA_IOCTL_WRITE _IOW(EBA_IOC_MAGIC, 2, struct eba_rw_data)
- #define EBA_IOCTL_READ _IOR(EBA_IOC_MAGIC, 3, struct eba_rw_data)
- #define EBA_IOCTL_REMOTE_ALLOC _IOWR(EBA_IOC_MAGIC, 4, struct eba_remote_alloc)
- #define EBA_IOCTL_REMOTE_WRITE _IOWR(EBA_IOC_MAGIC, 5, struct eba_remote_write)
- #define EBA_IOCTL_REMOTE_READ _IOWR(EBA_IOC_MAGIC, 6, struct eba_remote_read)
- #define EBA_IOC_MAXNR 6
- #define EBA_INFO(fmt, ...) pr_info("EBA: " fmt, ##__VA_ARGS__)
- #define EBA_ERR(fmt, ...) pr_err("EBA: " fmt, ##__VA_ARGS__)
- #define EBA_WARN(fmt, ...) pr_warn("EBA: " fmt, ##__VA_ARGS__)
- #define EBA_CLEAN_BUFFER_CALLBACK_TIMER 60000

## 4.1.1 Detailed Description

EBA IOCTL interface header.

This header defines the data structures, macros, and IOCTL command definitions for interacting with the EBA driver. It provides the interface for user-space applications to perform buffer allocation, reading, writing, and remote operations via IOCTL calls.

Definition in file eba.h.

## 4.1.2 Macro Definition Documentation

### 4.1.2.1 EBA_CLEAN_BUFFER_CALLBACK_TIMER

```
#define EBA_CLEAN_BUFFER_CALLBACK_TIMER 60000
```

Timer period for the buffer clean-up callback in milliseconds (60000 ms = 1 minute).

Definition at line 81 of file eba.h.

### 4.1.2.2 EBA_ERR

```
#define EBA_ERR(
            fmt,
            ...  ) pr_err("EBA: " fmt, ##__VA_ARGS__)
```

Log an error message for the EBA driver.

Definition at line 75 of file eba.h.

### 4.1.2.3 EBA_INFO

```
#define EBA_INFO(
            fmt,
            ...  ) pr_info("EBA: " fmt, ##__VA_ARGS__)
```

Log an informational message for the EBA driver.

Definition at line 73 of file eba.h.

#### 4.1.2.4 EBA_IOC_MAGIC

```
#define EBA_IOC_MAGIC 'E'
```

EBA IOCTL magic number used to create unique IOCTL command codes.

Definition at line 18 of file eba.h.

#### 4.1.2.5 EBA_IOC_MAXNR

```
#define EBA_IOC_MAXNR 6
```

Maximum number of EBA IOCTL commands supported.

Definition at line 68 of file eba.h.

#### 4.1.2.6 EBA_IOCTL_ALLOC

```
#define EBA_IOCTL_ALLOC _IOWR(EBA_IOC_MAGIC, 1, struct eba_alloc_data)
```

EBA_IOCTL_ALLOC - IOCTL command for buffer allocation.

This command uses the struct eba_alloc_data structure for both input and output.

Definition at line 28 of file eba.h.

#### 4.1.2.7 EBA_IOCTL_READ

```
#define EBA_IOCTL_READ _IOR(EBA_IOC_MAGIC, 3, struct eba_rw_data)
```

EBA_IOCTL_READ - IOCTL command for reading data from an allocated buffer.

This command uses the struct eba_rw_data structure to specify read operations.

Definition at line 42 of file eba.h.

#### 4.1.2.8 EBA_IOCTL_REMOTE_ALLOC

```
#define EBA_IOCTL_REMOTE_ALLOC _IOWR(EBA_IOC_MAGIC, 4, struct eba_remote_alloc)
```

EBA_IOCTL_REMOTE_ALLOC - IOCTL command for remote buffer allocation.

This command uses the struct eba_remote_alloc structure to request and obtain a buffer allocation on a remote node.

Definition at line 51 of file eba.h.

#### 4.1.2.9 EBA_IOCTL_REMOTE_READ

```
#define EBA_IOCTL_REMOTE_READ _IOWR(EBA_IOC_MAGIC, 6, struct eba_remote_read)
```

EBA_IOCTL_REMOTE_READ - IOCTL command for reading from a remote buffer.

This command uses the struct eba_remote_read structure to perform a remote read operation.

Definition at line 65 of file eba.h.

#### 4.1.2.10 EBA_IOCTL_REMOTE_WRITE

```
#define EBA_IOCTL_REMOTE_WRITE _IOWR(EBA_IOC_MAGIC, 5, struct eba_remote_write)
```

EBA_IOCTL_REMOTE_WRITE - IOCTL command for writing to a remote buffer.

This command uses the struct eba_remote_write structure to perform a remote write operation.

Definition at line 58 of file eba.h.

#### 4.1.2.11 EBA_IOCTL_WRITE

```
#define EBA_IOCTL_WRITE _IOW(EBA_IOC_MAGIC, 2, struct eba_rw_data)
```

EBA_IOCTL_WRITE - IOCTL command for writing data to an allocated buffer.

This command uses the struct eba_rw_data structure to specify write operations.

Definition at line 35 of file eba.h.

#### 4.1.2.12 EBA_WARN

```
#define EBA_WARN(
              fmt,
              ...  ) pr_warn("EBA: " fmt, ##__VA_ARGS__)
```

Log a warning message for the EBA driver.

Definition at line 77 of file eba.h.

## 4.2 eba.h

Go to the documentation of this file.

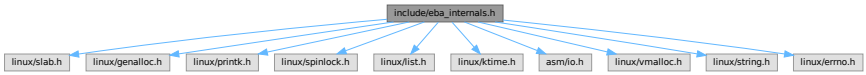
```
#ifndef _EBA_H
#define _EBA_H
#include <linux/ioctl.h>
#include <linux/types.h>

/* IOCTL Magic Number */

#define EBA_IOC_MAGIC 'E'


/* IOCTL Command Definitions */

#define EBA_IOCTL_ALLOC _IOWR(EBA_IOC_MAGIC, 1, struct eba_alloc_data)

#define EBA_IOCTL_WRITE _IOW(EBA_IOC_MAGIC, 2, struct eba_rw_data)

#define EBA_IOCTL_READ  _IOR(EBA_IOC_MAGIC, 3, struct eba_rw_data)


#define EBA_IOCTL_REMOTE_ALLOC _IOWR(EBA_IOC_MAGIC, 4, struct eba_remote_alloc)

#define EBA_IOCTL_REMOTE_WRITE _IOWR(EBA_IOC_MAGIC, 5, struct eba_remote_write)

#define EBA_IOCTL_REMOTE_READ _IOWR(EBA_IOC_MAGIC, 6, struct eba_remote_read)

#define EBA_IOC_MAXNR 6

/* Logging Macros */

#define EBA_INFO(fmt, ...)  pr_info("EBA: " fmt, ##__VA_ARGS__)
#define EBA_ERR(fmt, ...)   pr_err("EBA: " fmt, ##__VA_ARGS__)
#define EBA_WARN(fmt, ...)  pr_warn("EBA: " fmt, ##__VA_ARGS__)

/* Timer Settings */
#define EBA_CLEAN_BUFFER_CALLBACK_TIMER 60000// 1 min

struct eba_alloc_data {
    __u64 size;
    __u64 life_time;
    __u64 buff_id;
};

struct eba_rw_data {
    __u64 buff_id;
    __u64 off;
    __u64 size;
    __u64 user_addr;
};

struct eba_remote_alloc{
    __u64 size;
    __u64 life_time;
    __u64 buffer_id;
    const char mac[6];
};

struct eba_remote_write{
    __u64 buff_id;
    __u64 offset;
    __u64 size;
    char* payload;
    const char mac[6];
};

struct eba_remote_read {
    __u64 dst_buffer_id;
    __u64 src_buffer_id;
    __u64 dst_offset;
    __u64 src_offset;
    __u64 size;
    const char mac[6];
};


#endif /* _EBA_H */
```

## 4.3 include/eba_internals.h File Reference

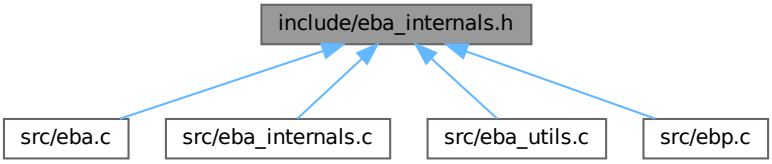
EBA Internal Memory Management Header.

```
#include <linux/slab.h>
#include <linux/genalloc.h>
#include <linux/printk.h>
#include <linux/spinlock.h>
#include <linux/list.h>
#include <linux/ktime.h>
#include <asm/io.h>
#include <linux/vmalloc.h>
#include <linux/string.h>
#include <linux/errno.h>
```

Include dependency graph for eba_internals.h:




This graph shows which files directly or indirectly include this file:




### Classes

- struct eba_buffer

### Macros

- #define MEMORY_POOL_SIZE (1024 ∗ 1024)∗1000

### Functions

- int eba_internals_mempool_init (void)

  *Initializes the memory pool for the EBA module.*
- void ∗ eba_internals_malloc (uint64_t size, uint64_t life_time)

  *Allocates a block of memory from the EBA memory pool.*
- int eba_internals_free (void ∗ptr)

  *Frees a previously allocated memory block from the EBA memory pool.*

- void eba_internals_mempool_free (void)

  *Frees the entire memory pool for the EBA module.*
- int eba_internals_mem_stress_test (void)

  *Performs a stress test on the EBA memory pool.*
- int eba_internals_write (const void ∗data, uint64_t buff_id, uint64_t off, uint64_t size)
- int eba_internals_read (void ∗data_out, uint64_t buff_id, uint64_t off, uint64_t size)
- int eba_internals_rw_stress_test (void)
- void eba_check_expired_buffers (void)

### 4.3.1 Detailed Description

EBA Internal Memory Management Header.

This header provides the internal mechanisms for managing memory within the EBA module. It includes functions to initialize and manage a generic memory pool, allocate and free buffers from that pool, perform stress tests on memory operations, and handle timed expiration of allocated buffers. This file is intended for internal use and is critical for the dynamic memory handling of the EBA framework.

Definition in file eba_internals.h.

### 4.3.2 Macro Definition Documentation

#### 4.3.2.1 MEMORY_POOL_SIZE

```
#define MEMORY_POOL_SIZE (1024 * 1024)*1000
```

Size of the memory pool in bytes.

Definition at line 24 of file eba_internals.h.

### 4.3.3 Function Documentation

#### 4.3.3.1 eba_check_expired_buffers()

```
void eba_check_expired_buffers (
            void  )
```

eba_check_expired_buffers - Free buffers that have passed their expiration time.

This function iterates over the global tracking list of allocated buffers. If a buffer's expiration time is nonzero (i.e., its lifetime is not infinite) and the current time is equal to or greater than the expiration time, the buffer is removed from the list, freed from the generic memory pool, and its tracking structure is deallocated.

**Note**

: This function should be called periodically from a timer or workqueue to reclaim expired buffers.

Definition at line 484 of file eba_internals.c.

```c
{
    struct eba_buffer *buf, *tmp;
    ktime_t now = ktime_get_real_seconds();
    // Acquire the spinlock before iterating the list.
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry_safe(buf, tmp, &eba_buffer_list, node) {
        // If expires is 0, the lifetime is infinite, so skip.
        if (buf->expires != 0 && now >= buf->expires) {
            EBA_INFO("EBA: Buffer at %llu expired (current time %lld >= expires %lld)\n",buf->address, now,buf->expires);
            // Remove the entry from the tracking list
            list_del(&buf->node);
            // Free the allocated memory from the gen_pool
            gen_pool_free(eba_pool, buf->address, buf->size);
            // Free the tracking structure itself
            kfree(buf);
        }
    }
    spin_unlock(&eba_buffer_list_lock);
}
```

#### 4.3.3.2 eba_internals_free()

```
int eba_internals_free (
            void * ptr )
```

Frees a previously allocated memory block from the EBA memory pool.

eba_internals_free - Free a previously allocated block of memory from the EBA memory pool. @ptr: Pointer to the memory block to free.

This function locates the allocation corresponding to the given pointer in the global tracking list, removes it, frees the memory block from the generic memory pool, and then frees the associated tracking structure.

Return: 0 if the memory was successfully freed, or -1 if the pointer is invalid or not found.

This function looks up the tracking structure associated with the provided pointer, removes it from the global tracking list, frees the memory block from the gen_pool, and then frees the tracking structure.

**Parameters**

| | |
|---|---|
| *ptr* | The pointer to the memory block to free. |

**Returns**

0 if the memory was successfully freed, or -1 if the pointer is invalid or not found.

Definition at line 137 of file eba_internals.c.

```
{
    struct eba_buffer *buf;
    int found = 0;

    /* Check if the memory pool is initialized and if the pointer is valid */
    if (!eba_pool || !ptr)
        return -1;

    /* Lock the global list before searching for the tracking structure */
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
        if (buf->address == (unsigned long)ptr) {
            list_del(&buf->node);
            found = 1;
            break;
        }
    }
    spin_unlock(&eba_buffer_list_lock);

    /* If no tracking structure is found, log an error and return -1 */
    if (!found) {
        EBA_ERR("EBA: Attempt to free unknown memory at %llu\n", (uint64_t)ptr);
        return -1;
    }

    /* Free the memory from the gen_pool using the stored size */
    gen_pool_free(eba_pool, (unsigned long)ptr, buf->size);

    /* Free the tracking structure */
    kfree(buf);

    EBA_INFO("EBA: Freed memory at %llu, size: %llu bytes\n", (uint64_t)ptr, buf->size);
    return 0;
}
```

#### 4.3.3.3 eba_internals_malloc()

```
void * eba_internals_malloc (
            uint64_t size,
            uint64_t life_time )
```

Allocates a block of memory from the EBA memory pool.

eba_internals_malloc - Allocate a block of memory from the EBA memory pool. @size: Number of bytes to allocate. @life_time: Lifetime of the allocation in seconds; 0 indicates infinite.

This function allocates a memory block from the generic memory pool, creates a tracking structure to record allocation details (pointer, size, lifetime), and adds this structure to a global list for tracking.

Return: Pointer to the allocated memory on success, or NULL if the allocation fails.

This function allocates a block of memory from the generic memory pool, creates a tracking structure to record the allocation information (including the allocated pointer, size, and lifetime), and adds the tracking structure to a global list.

**Parameters**

| | |
|---|---|
| *size* | The number of bytes to allocate. |
| *life_time* | The lifetime of the allocation in seconds. |

**Returns**

A pointer to the allocated memory on success, or NULL if the allocation fails.

Definition at line 73 of file eba_internals.c.

```
{
    unsigned long addr;
    void *ptr;
    struct eba_buffer *buf;

    /* Check if the memory pool is initialized */
    if (!eba_pool)
        return NULL;

    /* Allocate a block of memory from the gen_pool */
    addr = gen_pool_alloc(eba_pool, size);
    if (!addr) {
        EBA_ERR("EBA: Allocation failed for size %llu\n", size);
        return NULL;
    }
    ptr = (void *)addr;
     /* Zero out the allocated memory */
     memset(ptr, 0, size);
    /* Allocate and initialize the tracking structure */
    buf = kmalloc(sizeof(*buf), GFP_KERNEL);
    if (!buf) {
        EBA_ERR("EBA: Tracking structure allocation failed\n");
        gen_pool_free(eba_pool, addr, size);
        return NULL;
    }

    spin_lock_init(&buf->lock);
    buf->type = 0;    /* Set default type (customize if needed) */

    /* Store the allocated pointer */
    buf->address = (unsigned long)ptr;
    buf->size = size;

    /* Set the expiration time using the provided lifetime (in seconds) */
     if (life_time != 0)
         buf->expires = ktime_get_real_seconds() + life_time;
     else
         buf->expires = 0;
    /* Initialize the list head for tracking */
    INIT_LIST_HEAD(&buf->node);

    /* Add the tracking structure to the global list in a thread-safe manner */
    spin_lock(&eba_buffer_list_lock);
    list_add(&buf->node, &eba_buffer_list);
    spin_unlock(&eba_buffer_list_lock);

    EBA_INFO("EBA: Allocated memory at %llu, size: %llu bytes, lifetime: %llu sec\n",
            (uint64_t)ptr, size, life_time);

    return ptr;
}
```

#### 4.3.3.4 eba_internals_mem_stress_test()

```
int eba_internals_mem_stress_test (
            void  )
```

Performs a stress test on the EBA memory pool.

eba_internals_mem_stress_test - Perform a stress test on the EBA memory pool.

This function repeatedly allocates and frees memory blocks using eba_internals_malloc() and eba_internals_free(). It verifies that both allocation and de-allocation phases complete successfully.

Return: 0 on success, or a negative error code on failure.

This function repeatedly allocates and frees memory blocks using eba_internals_malloc() and eba_internals_free() to verify that the memory pool is robust and that allocations/deallocations work correctly.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 222 of file eba_internals.c.

```
{
    const int num_allocs = 100;  /* Number of allocations for the stress test */
    void **allocations;          /* Array to store pointers to allocated memory */
    int *sizes;                  /* Array to store the sizes for each allocation */
    int i, ret;

    /* Dynamically allocate arrays for tracking allocations and their sizes */
    allocations = kmalloc_array(num_allocs, sizeof(void *), GFP_KERNEL);
    if (!allocations)
        return -ENOMEM;

    sizes = kmalloc_array(num_allocs, sizeof(int), GFP_KERNEL);
    if (!sizes) {
        kfree(allocations);
        return -ENOMEM;
    }

    /* Initialize sizes and set each allocation pointer to NULL */
    for (i = 0; i < num_allocs; i++) {
        sizes[i] = 16 + ((i * 31) % 1024);  /* Example size: between 16 and ~1040 bytes */
        allocations[i] = NULL;
    }

    /* Allocation Phase: Allocate memory chunks and verify each allocation */
    for (i = 0; i < num_allocs; i++) {
        allocations[i] = eba_internals_malloc(sizes[i], 10);  /* Lifetime set to 10 sec */
        if (!allocations[i]) {
            EBA_ERR("EBA-STRESS: Allocation %d of size %d failed\n", i, sizes[i]);
            ret = -ENOMEM;
            goto cleanup;
        }
    }
    EBA_INFO("EBA-STRESS: Allocation phase completed successfully\n");

    /* De-allocation Phase: Free all allocated memory chunks */
    for (i = 0; i < num_allocs; i++) {
        ret = eba_internals_free(allocations[i]);
        if (ret != 0) {
            EBA_ERR("EBA-STRESS: Freeing allocation %d failed\n", i);
            goto cleanup;
        }
    }
    EBA_INFO("EBA-STRESS: De-allocation phase completed successfully\n");

    ret = 0;  /* If all allocations and frees succeed, return success */

cleanup:
    /* Free the tracking arrays */
    kfree(allocations);
    kfree(sizes);
    return ret;
}
```

#### 4.3.3.5 eba_internals_mempool_free()

```
void eba_internals_mempool_free (
            void  )
```

Frees the entire memory pool for the EBA module.

eba_internals_mempool_free - Free the entire memory pool for the EBA module.

This function cleans up all memory allocations by freeing all tracking structures in the global list, destroying the generic memory pool, and releasing the backing memory allocated using the DMA API.

This function cleans up by removing and freeing all tracking structures in the global list, destroying the generic memory pool, and releasing the backing memory that was allocated via vmalloc.

Definition at line 179 of file eba_internals.c.

```
{
    struct eba_buffer *buf, *tmp;
    int outstanding = 0;

    // Count outstanding allocations
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
        outstanding++;
    }
    EBA_INFO("EBA: %d outstanding buffers before cleanup\n", outstanding);
    if (outstanding > 0) {
        list_for_each_entry_safe(buf, tmp, &eba_buffer_list, node) {
            EBA_INFO("EBA: Forcing free of buffer at %llu, size: %llu bytes\n",
                    buf->address, buf->size);
            list_del(&buf->node);
            gen_pool_free(eba_pool, buf->address, buf->size);
            kfree(buf);
        }
    }

    // Now destroy the gen_pool and free the backing memory
    if (eba_pool) {
        gen_pool_destroy(eba_pool);
        eba_pool = NULL;
    }
    if (eba_pool_mem) {
        vfree(eba_pool_mem);
        eba_pool_mem = NULL;
    }
    spin_unlock(&eba_buffer_list_lock);
    EBA_INFO("EBA: Memory pool freed successfully\n");
}
```

#### 4.3.3.6 eba_internals_mempool_init()

```
int eba_internals_mempool_init (
            void  )
```

Initializes the memory pool for the EBA module.

eba_internals_mempool_init - Initialize the memory pool for the EBA module.

This function creates a generic memory pool with 8-byte alignment, allocates a contiguous block of DMA-coherent memory to back that pool, registers the allocated memory block with the pool, and initializes the spinlock protecting the global tracking list.

Return: 0 if the memory pool is successfully initialized, 1 otherwise.

This function creates a generic memory pool with an 8-byte alignment, allocates a contiguous block of memory (using kmalloc) to back that pool, registers the allocated memory block with the pool, and initializes the spinlock protecting the global tracking list.

**Returns**

0 if the memory pool is successfully initialized, 1 otherwise.

Definition at line 22 of file eba_internals.c.

```
{
    unsigned long align = 8;    /* Set allocation alignment to 8 bytes */
    unsigned long phys_addr;    /* Dummy physical address for vmalloc'd memory */

    /* Create the generic memory pool.
     * The second parameter (-1) specifies flags; here, no special flags are needed.
     */
    eba_pool = gen_pool_create(align, -1);
    if (!eba_pool) {
        EBA_ERR("EBA: Failed to create memory pool\n");
        return 1;
    }

    /* Allocate a contiguous block of virtual memory using vmalloc.
     * This method supports very large allocations (subject to system limits) but
     * the returned memory is not physically contiguous.
     */
    eba_pool_mem = vmalloc(MEMORY_POOL_SIZE);
    if (!eba_pool_mem) {
        EBA_ERR("EBA: Failed to allocate memory for the pool\n");
        gen_pool_destroy(eba_pool);
        return 1;
    }

    /* Since vmalloc'd memory is not physically contiguous,
     * we cannot reliably compute a physical base address.
     * We pass a dummy value (e.g., 0) to gen_pool_add_virt().
     */
    phys_addr = 0;
    gen_pool_add_virt(eba_pool, (unsigned long)eba_pool_mem, phys_addr, MEMORY_POOL_SIZE, -1);

    /* Initialize the spinlock to protect the global tracking list */
    spin_lock_init(&eba_buffer_list_lock);

    EBA_INFO("EBA: Memory pool initialized successfully\n");
    return 0;
}
```

#### 4.3.3.7 eba_internals_read()

```
int eba_internals_read (
            void * data_out,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_internals_read - Read data from an allocated buffer. @data_out: Pointer to the destination where read data should be stored. @buff_id: Identifier of the buffer (virtual address returned by eba_internals_malloc). @off: Offset (in bytes) within the buffer from which reading should start. @size: Number of bytes to read.

This function locates the allocated buffer corresponding to @buff_id, verifies that the read operation (starting at offset @off for @size bytes) is within bounds, and copies data from the buffer into the provided output pointer.

Return: 0 on success, or a negative error code (e.g., -EINVAL) if the operation fails.

Definition at line 321 of file eba_internals.c.

```
{
    struct eba_buffer *buf;
    int found = 0;

    if (!data_out)
    {
     EBA_ERR("eba_internals_read: data pointer is NULL\n");
     return -EINVAL;
    }

    /* Locate the tracking structure using buff_id (the allocated virtual address) */
    spin_lock(&eba_buffer_list_lock);
```

```
    list_for_each_entry(buf, &eba_buffer_list, node) {
        if (buf->address == buff_id) {
            found = 1;
            break;
        }
    }
    spin_unlock(&eba_buffer_list_lock);

    if (!found)
    {
        EBA_ERR("eba_internals_read: Buffer with id 0x%llx not found\n", buff_id);
        return -EINVAL;  /* Buffer not found */
    }

    spin_lock(&buf->lock);
        /* Check if the read operation is within bounds */
    if (off + size > buf->size)
    {
        EBA_ERR("eba_internals_read: Read out of bounds: off(%llu) + size(%llu) > buf->size(%llu)\n", off, size, buf->size);
        spin_unlock(&buf->lock);
        return -EINVAL;
    }

    /* Copy data from the allocated buffer at the specified offset into data_out */
    memcpy(data_out, (void *)(buf->address + off), size);
    spin_unlock(&buf->lock);
    EBA_INFO("EBA: Read %llu bytes at offset %llu on buf %llu\n",size,off,buff_id);
    return 0;
}
```

#### 4.3.3.8 eba_internals_rw_stress_test()

```
int eba_internals_rw_stress_test (
            void  )
```

eba_internals_rw_stress_test - Stress test for read/write operations on the EBA memory pool.

This function allocates multiple buffers using eba_internals_malloc(), writes a known data pattern into each buffer with eba_internals_write(), reads the data back using eba_internals_read(), verifies that the read data matches the written data, and then deallocates the buffers using eba_internals_free().

Return: 0 on success, or a negative error code if any operation fails.

Definition at line 377 of file eba_internals.c.

```
{
    const int num_allocs = 100;  /* Number of buffers to allocate for the test */
    void **allocations;          /* Array to store allocated buffer pointers */
    int *sizes;                  /* Array to store the sizes for each buffer */
    int i, ret = 0;
    unsigned char *pattern = NULL;
    unsigned char *read_back = NULL;

    /* Allocate arrays to track allocated buffers and their sizes */
    allocations = kmalloc_array(num_allocs, sizeof(void *), GFP_KERNEL);
    if (!allocations)
        return -ENOMEM;

    sizes = kmalloc_array(num_allocs, sizeof(int), GFP_KERNEL);
    if (!sizes) {
        kfree(allocations);
        return -ENOMEM;
    }

    /* Initialize the sizes and set each allocation pointer to NULL */
    for (i = 0; i < num_allocs; i++) {
        sizes[i] = 16 + ((i * 31) % 1024);  /* Buffer size will vary between 16 and about 1040 bytes */
        allocations[i] = NULL;
    }

    /* Allocation and test phase */
    for (i = 0; i < num_allocs; i++) {
        /* Allocate a buffer with a lifetime of 10 seconds */
        allocations[i] = eba_internals_malloc(sizes[i], 10);
        if (!allocations[i]) {
            EBA_ERR("EBA-RW-STRESS: Allocation %d of size %d failed\n", i, sizes[i]);
```

```
            ret = -ENOMEM;
            goto cleanup;
        }

        /* Allocate temporary buffers for writing and reading */
        pattern = kmalloc(sizes[i], GFP_KERNEL);
        if (!pattern) {
            ret = -ENOMEM;
            goto cleanup;
        }
        read_back = kmalloc(sizes[i], GFP_KERNEL);
        if (!read_back) {
            kfree(pattern);
            ret = -ENOMEM;
            goto cleanup;
        }

        /*
         * Fill the pattern buffer with a test pattern.
         * For example, store a sequence of incrementing values modulo 256.
         */
        {
            int j;
            for (j = 0; j < sizes[i]; j++)
                pattern[j] = (unsigned char)(j & 0xFF);
        }

        /* Write the test pattern into the allocated buffer */
        ret = eba_internals_write(pattern, (uint64_t)allocations[i], 0, sizes[i]);
        if (ret != 0) {
            EBA_ERR("EBA-RW-STRESS: Write failed on buffer %d\n", i);
            kfree(pattern);
            kfree(read_back);
            goto cleanup;
        }

        /* Read back the data into the read_back buffer */
        ret = eba_internals_read(read_back, (uint64_t)allocations[i], 0, sizes[i]);
        if (ret != 0) {
            EBA_ERR("EBA-RW-STRESS: Read failed on buffer %d\n", i);
            kfree(pattern);
            kfree(read_back);
            goto cleanup;
        }

        /* Compare the written pattern with the read-back data */
        if (memcmp(pattern, read_back, sizes[i]) != 0) {
            EBA_ERR("EBA-RW-STRESS: Data mismatch on buffer %d\n", i);
            ret = -EINVAL;
            kfree(pattern);
            kfree(read_back);
            goto cleanup;
        }

        /* Free the temporary buffers */
        kfree(pattern);
        kfree(read_back);

        EBA_INFO("EBA-RW-STRESS: Buffer %d (size %d) verified successfully\n", i, sizes[i]);
    }

    EBA_INFO("EBA-RW-STRESS: All read/write operations completed successfully\n");
    ret = 0;

cleanup:
    /* Free all allocated buffers */
    for (i = 0; i < num_allocs; i++) {
        if (allocations[i])
            eba_internals_free(allocations[i]);
    }
    kfree(allocations);
    kfree(sizes);
    return ret;
}
```

#### 4.3.3.9 eba_internals_write()

```
int eba_internals_write (
            const void * data,
            uint64_t buff_id,
```

```
            uint64_t off,
            uint64_t size )
```

eba_internals_write - Write data into an allocated buffer. @data: Pointer to the data to write. @buff_id: Identifier of the buffer (virtual address returned by eba_internals_malloc). @off: Offset (in bytes) within the buffer at which writing should begin. @size: Number of bytes to write.

This function locates the allocated buffer corresponding to @buff_id, verifies that the write operation (starting at offset @off for @size bytes) is within bounds, and copies the data into the buffer.

Return: 0 on success, or a negative error code (e.g., -EINVAL) if the operation fails.

Definition at line 276 of file eba_internals.c.

```
00277 {
00278     struct eba_buffer *buf;
00279     int found = 0;
00280
00281     if (!data)
00282     {
00283           EBA_ERR("eba_internals_write: data pointer is NULL\n");
00284          return -EINVAL;
00285     }
00286
00287     /* Locate the tracking structure using buff_id (the allocated virtual address) */
00288     spin_lock(&eba_buffer_list_lock);
00289     list_for_each_entry(buf, &eba_buffer_list, node) {
00290          if (buf->address == buff_id) {
00291               found = 1;
00292               break;
00293          }
00294     }
00295     spin_unlock(&eba_buffer_list_lock);
00296
00297     if (!found)
00298     {
00299           EBA_ERR("eba_internals_write: Buffer with id 0x%llx not found\n", buff_id);
00300          return -EINVAL;  /* Buffer not found */
00301     }
00302      spin_lock(&buf->lock);
00303     /* Check if the write operation is within bounds */
00304     if (off + size > buf->size)
00305     {
00306           EBA_ERR("eba_internals_write: Write out of bounds: off(%llu) + size(%llu) >
      buf->size(%llu)\n", off, size, buf->size);
00307           spin_unlock(&buf->lock);
00308          return -EINVAL;
00309     }
00310
00311     /* Copy the data into the allocated buffer at the specified offset */
00312     memcpy((void *)(buf->address + off), data, size);
00313     spin_unlock(&buf->lock);
00314     EBA_INFO("EBA: Written %llu bytes at offset %llu on buf %llu\n",size,off,buff_id);
00315     return 0;
00316 }
```

## 4.4 eba_internals.h

Go to the documentation of this file.

```
00001
00011 #ifndef EBA_INTERNALS_H
00012 #define EBA_INTERNALS_H
00013 #include <linux/slab.h>
00014 #include <linux/genalloc.h>
00015 #include <linux/printk.h>
00016 #include <linux/spinlock.h>
00017 #include <linux/list.h>
00018 #include <linux/ktime.h>
00019 #include <asm/io.h>
00020 #include <linux/vmalloc.h>
00021 #include <linux/string.h>
00022 #include <linux/errno.h>
00024 #define MEMORY_POOL_SIZE (1024 * 1024)*1000
00025
00037 struct eba_buffer {
00038   spinlock_t lock;
```

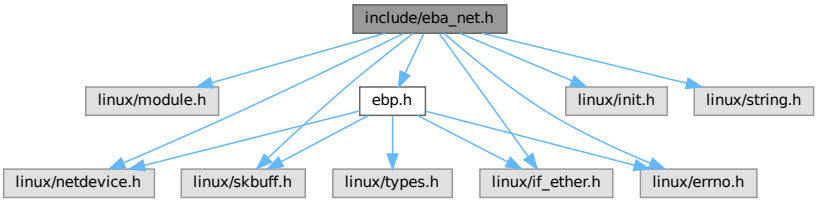
```
  uint8_t type;
  uint64_t address;
  uint64_t size;
  ktime_t expires;
  struct list_head node;
};

int eba_internals_mempool_init(void);

void *eba_internals_malloc(uint64_t size, uint64_t life_time);

int eba_internals_free(void *ptr);

void eba_internals_mempool_free(void);

int eba_internals_mem_stress_test(void);

int eba_internals_write(const void *data, uint64_t buff_id, uint64_t off, uint64_t size);

int eba_internals_read(void *data_out, uint64_t buff_id, uint64_t off, uint64_t size);

int eba_internals_rw_stress_test(void);

void eba_check_expired_buffers(void);
#endif
```

## 4.5 include/eba_net.h File Reference

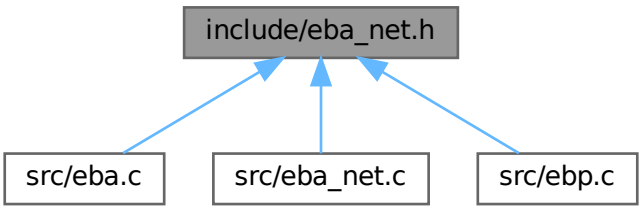
EBA Networking Interface Header.

```
#include <linux/module.h>
#include <linux/netdevice.h>
#include <linux/skbuff.h>
#include <linux/if_ether.h>
#include <linux/init.h>
#include <linux/errno.h>
#include <linux/string.h>
#include "ebp.h"
```

Include dependency graph for eba_net.h:



This graph shows which files directly or indirectly include this file:




**Functions**

- int send_raw_ethernet_packet (const unsigned char ∗payload, size_t payload_len, const unsigned char ∗dst_mac, int protocol, const char ∗ifname)
- int eba_net_get_max_mtu (const char ∗ifname, int ∗max_mtu)
- int eba_net_set_mtu (const char ∗ifname, int new_mtu)
- char ∗ build_discover_req_packet (uint16_t mtu, uint64_t ∗out_len)
- char ∗ build_discover_ack_packet (uint64_t buffer_id, uint64_t ∗out_len)
- char ∗ build_invoke_req_packet (uint32_t iid, uint32_t opid, const char ∗args, uint64_t args_len, const char ∗payload, uint64_t payload_len, uint64_t ∗out_len)
- char ∗ build_invoke_ack_packet (uint8_t status, uint64_t ∗out_len)
- int send_discover_req_packet (uint16_t mtu, const unsigned char dest_mac[6], const char ∗ifname)
- int send_discover_ack_packet (uint64_t buffer_id, const unsigned char dest_mac[6], const char ∗ifname)
- int send_invoke_req_packet (uint32_t iid, uint32_t opid, const char ∗args, uint64_t args_len, const void ∗payload, uint64_t payload_len, const unsigned char dest_mac[6], const char ∗ifname)
- int send_invoke_ack_packet (uint8_t status, const unsigned char dest_mac[6], const char ∗ifname)

## 4.5.1 Detailed Description

EBA Networking Interface Header.

This header defines the networking functions for the EBA system. It includes routines for sending raw Ethernet packets, retrieving and setting network interface MTU values, and building and transmitting various EBA protocol packets (e.g. Discover Request/ACK, Invoke Request/ACK). These interfaces facilitate EBA communication over the network.

Definition in file eba_net.h.

## 4.5.2 Function Documentation

### 4.5.2.1 build_discover_ack_packet()

```
char * build_discover_ack_packet (
            uint64_t buffer_id,
            uint64_t * out_len )
```

build_discover_ack_packet - Build a Discover Acknowledgment packet. @buffer_id: 64-bit identifier to embed in the ACK packet. @out_len: Pointer to a variable that returns the total packet length.

This function builds and returns a newly allocated Discover-Ack packet. The caller is responsible for freeing the packet.

Return: Pointer to the allocated packet on success, or NULL on failure.

Definition at line 134 of file eba_net.c.

```
{
    uint64_t len = sizeof(struct ebp_discover_ack);
    struct ebp_discover_ack *ack = kmalloc(len, GFP_KERNEL);
    if (!ack)
        return NULL;
    ack->header.msgType = EBP_MSG_DISCOVER_ACK;
    ack->buffer_id = cpu_to_be64(buffer_id);
    *out_len = len;
    return (char *)ack;
}
```

#### 4.5.2.2 build_discover_req_packet()

```
char * build_discover_req_packet (
            uint16_t mtu,
            uint64_t * out_len )
```

build_discover_req_packet - Build a Discover Request packet. @mtu: MTU value to embed in the discover request. @out_len: Pointer to a variable that returns the total packet length.

This function builds and returns a newly allocated Discover Request packet. The caller is responsible for freeing the allocated packet.

Return: Pointer to the packet on success, or NULL on failure.

Definition at line 122 of file eba_net.c.

```
{
    uint64_t len = sizeof(struct ebp_discover_req);
    struct ebp_discover_req *req = kmalloc(len, GFP_KERNEL);
    if (!req)
        return NULL;
    req->header.msgType = EBP_MSG_DISCOVER;
    req->mtu = htons(mtu);
    *out_len = len;
    return (char *)req;
}
```

#### 4.5.2.3 build_invoke_ack_packet()

```
char * build_invoke_ack_packet (
            uint8_t status,
            uint64_t * out_len )
```

build_invoke_ack_packet - Build an Invoke Acknowledgment packet. @status: 8-bit status code (enum INVOKE_↩ STATUS) to embed in the ACK. @out_len: Pointer to a variable that returns the total packet length.

This function builds and returns a newly allocated Invoke-Ack packet. The caller is responsible for freeing the allocated packet.

Return: Pointer to the allocated packet on success, or NULL on failure.

Definition at line 170 of file eba_net.c.

```
{
    uint64_t len = sizeof(struct ebp_invoke_ack);
    struct ebp_invoke_ack *ack = kmalloc(len, GFP_KERNEL);
    if (!ack)
        return NULL;
    ack->header.msgType = EBP_MSG_INVOKE_ACK;
    ack->status = status;
    *out_len = len;
    return (char *)ack;
}
```

#### 4.5.2.4 build_invoke_req_packet()

```
char * build_invoke_req_packet (
            uint32_t iid,
            uint32_t opid,
            const char * args,
            uint64_t args_len,
            const char * payload,
            uint64_t payload_len,
            uint64_t * out_len )
```

build_invoke_req_packet - Build an Invoke Request packet. @iid: 32-bit Invocation ID. @opid: 32-bit Operation ID (e.g., EBP_OP_WRITE, EBP_OP_READ). @args: Pointer to the invoke argument data. @args_len: Length (in bytes) of the invoke argument data. @payload: Pointer to additional payload data appended after the arguments. @payload_len: Length (in bytes) of the payload data. @out_len: Pointer to a variable that returns the total packet length.

The packet is built with the following layout:

[ebp_invoke_req header] | [args data] | [payload data]

The caller is responsible for freeing the returned packet.

Return: Pointer to the allocated packet on success, or NULL on failure.

Definition at line 146 of file eba_net.c.

```
00150 {
00151     uint64_t total_len = sizeof(struct ebp_invoke_req) + args_len + payload_len;
00152     char *buf = kmalloc(total_len, GFP_KERNEL);
00153     if (!buf)
00154         return NULL;
00155
00156     struct ebp_invoke_req *req = (struct ebp_invoke_req *)buf;
00157     req->header.msgType = EBP_MSG_INVOKE;
00158     req->iid = htonl(iid);
00159     req->opid = htonl(opid);
00160     /* Set args_len as the sum of the write header length and the payload length */
00161     req->args_len = cpu_to_be64(args_len + payload_len);
00162     if (args && args_len > 0)
00163         memcpy(buf + sizeof(struct ebp_invoke_req), args, args_len);
00164     if (payload && payload_len > 0)
00165         memcpy(buf + sizeof(struct ebp_invoke_req) + args_len, payload, payload_len);
00166     *out_len = total_len;
00167     return buf;
00168 }
```

#### 4.5.2.5 eba_net_get_max_mtu()

```
int eba_net_get_max_mtu (
            const char * ifname,
            int * max_mtu )
```

eba_net_get_max_mtu - Retrieve the maximum MTU supported by the specified network device. @ifname: Network device name (e.g., "eth0"). @max_mtu: Pointer to an integer where the maximum MTU will be stored.

Return: 0 on success or a negative error code on failure.

Definition at line 65 of file eba_net.c.

```
00066 {
00067     struct net_device *dev;
00068
00069     if (!ifname || !max_mtu)
00070         return -EINVAL;
00071
00072     /* Look up the network device by name in the initial network namespace */
```

```
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("EBA_NET: Could not find device %s\n", ifname);
        return -ENODEV;
    }

    /* Some drivers set max_mtu in the net_device structure */
    *max_mtu = dev->max_mtu;
    EBA_INFO("EBA_NET: Device %s current MTU: %d, Maximum supported MTU: %d\n",
            dev->name, dev->mtu, dev->max_mtu);

    dev_put(dev);
    return 0;
}
```

#### 4.5.2.6 eba_net_set_mtu()

```
int eba_net_set_mtu (
            const char * ifname,
            int new_mtu )
```

eba_net_set_mtu - Change the MTU of the specified network device. @ifname: Network device name (e.g., "eth0"). @new_mtu: The new MTU value to set.

Return: 0 on success or a negative error code on failure.

Definition at line 88 of file eba_net.c.

```
{
    struct net_device *dev;
    int ret = 0;

    if (!ifname)
        return -EINVAL;

    /* Look up the network device by name */
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("EBA_NET: Could not find device %s\n", ifname);
        return -ENODEV;
    }

    /* Check if the device's driver provides the ndo_change_mtu callback */
    if (dev->netdev_ops && dev->netdev_ops->ndo_change_mtu) {
        ret = dev->netdev_ops->ndo_change_mtu(dev, new_mtu);
        if (ret) {
            EBA_ERR("EBA_NET: Failed to change MTU on %s to %d, error: %d\n",
                    dev->name, new_mtu, ret);
        } else {
            EBA_INFO("EBA_NET: Successfully changed MTU on %s to %d\n",
                     dev->name, new_mtu);
        }
    } else {
        EBA_ERR("EBA_NET: Device %s does not support MTU change\n", dev->name);
        ret = -EOPNOTSUPP;
    }

    dev_put(dev);
    return ret;
}
```

#### 4.5.2.7 send_discover_ack_packet()

```
int send_discover_ack_packet (
            uint64_t buffer_id,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_discover_ack_packet - Build and send an EBP_MSG_DISCOVER_ACK packet. @buffer_id: 64-bit buffer identifier to embed in the ACK. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name.

Return: 0 on success or a negative error code on failure.

Definition at line 206 of file eba_net.c.

```
{
    uint64_t pkt_len = 0;
    char *packet = build_discover_ack_packet(buffer_id, &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_discover_ack_packet: build_discover_ack_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_discover_ack_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_discover_ack_packet: Sent EBP_MSG_DISCOVER_ACK to %pM via %s (buffer_id=0x%llx)\n",
             dest_mac, ifname, (unsigned long long)buffer_id);
    return 0;
}
```

#### 4.5.2.8 send_discover_req_packet()

```
int send_discover_req_packet (
            uint16_t mtu,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_discover_req_packet - Build and send an EBP_MSG_DISCOVER packet. @mtu: MTU value to embed in the discover request. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name (e.g., "enp0s8").

Return: 0 on success or a negative error code on failure.

Definition at line 182 of file eba_net.c.

```
{
    uint64_t pkt_len = 0;
    char *packet = build_discover_req_packet(mtu, &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_discover_req_packet: build_discover_req_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_discover_req_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_discover_req_packet: Sent EBP_MSG_DISCOVER to %pM via %s (MTU = %u)\n",
             dest_mac, ifname, mtu);
    return 0;
}
```

#### 4.5.2.9 send_invoke_ack_packet()

```
int send_invoke_ack_packet (
            uint8_t status,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_invoke_ack_packet - Build and send an EBP_MSG_INVOKE_ACK response packet. @status: Status code to embed in the ACK. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name.

Return: 0 on success or a negative error code on failure.

Definition at line 262 of file eba_net.c.

```
{
    uint64_t pkt_len = 0;
    char *packet = build_invoke_ack_packet(status, &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_invoke_ack_packet: build_invoke_ack_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_invoke_ack_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_invoke_ack_packet: Sent EBP_MSG_INVOKE_ACK (status=0x%02x) to %pM via %s\n",
             status, dest_mac, ifname);
    return 0;
}
```

#### 4.5.2.10 send_invoke_req_packet()

```
int send_invoke_req_packet (
            uint32_t iid,
            uint32_t opid,
            const char * args,
            uint64_t args_len,
            const void * payload,
            uint64_t payload_len,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_invoke_req_packet - Build and send an EBP_MSG_INVOKE request packet. @iid: Invocation ID (caller-defined). @opid: Operation ID (e.g., EBP_OP_WRITE, EBP_OP_READ). @args: Pointer to the arguments buffer. @args_len: Length of the arguments buffer. @payload: Pointer to optional payload data appended after the arguments. @payload_len: Length of the optional payload. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name.

This is a generic sender for an Invoke Request. Specialized helper functions, such as those implementing remote allocations or writes, may also call build_invoke_req_packet() directly.

Return: 0 on success or a negative error code on failure.

Definition at line 230 of file eba_net.c.

```
{
    uint64_t pkt_len = 0;
    char *packet = build_invoke_req_packet(iid, opid,
                                           args, args_len,
                                           payload, payload_len,
                                           &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_invoke_req_packet: build_invoke_req_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_invoke_req_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
```

```
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_invoke_req_packet: Sent EBP_MSG_INVOKE (IID=%u, OPID=%u) to %pM via %s\n",
             iid, opid, dest_mac, ifname);
    return 0;
}
```

#### 4.5.2.11 send_raw_ethernet_packet()

```
int send_raw_ethernet_packet (
            const unsigned char * payload,
            size_t payload_len,
            const unsigned char * dst_mac,
            int protocol,
            const char * ifname )
```

Definition at line 3 of file eba_net.c.

```
{
    struct net_device *dev = NULL;
    struct sk_buff *skb = NULL;
    struct ethhdr *eth;
    int ret;

    /* Look up the network device by name */
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("send_raw_ethernet_packet: Could not find device %s\n", ifname);
        return -ENODEV;
    }
    /* Calculate total length: Ethernet header + payload */
    size_t packet_len = ETH_HLEN + payload_len;

    /*
     * Allocate an sk_buff.
     */
    skb = alloc_skb(packet_len + dev->needed_tailroom, GFP_KERNEL);
    if (!skb) {
        EBA_ERR("send_raw_ethernet_packet: Failed to allocate sk_buff\n");
        dev_put(dev);
        return -ENOMEM;
    }

    /* Reserve headroom for the Ethernet header */
    skb_reserve(skb, ETH_HLEN);

    /* Set the device for the packet */
    skb->dev = dev;
    skb->protocol = htons(protocol);
    skb->pkt_type = PACKET_OUTGOING;

    /* Build the Ethernet header */
    eth = (struct ethhdr *)skb_push(skb, ETH_HLEN);
    /* Fill in destination MAC address (provided as argument) */
    memcpy(eth->h_dest, dst_mac, ETH_ALEN);
    /* Use the device's own MAC address as source */
    memcpy(eth->h_source, dev->dev_addr, ETH_ALEN);
    /* Set the protocol field */
    eth->h_proto = htons(protocol);

    /* Append payload data to the packet */
    if (payload && payload_len > 0) {
        void *payload_ptr = skb_put(skb, payload_len);
        memcpy(payload_ptr, payload, payload_len);
    }
    /* Queue the packet for transmission */
    ret = dev_queue_xmit(skb);
    if (ret < 0) {
        EBA_ERR("send_raw_ethernet_packet: Packet transmission failed: %d\n", ret);
        /* In case of error, the kernel should free the skb */
    } else {
        EBA_INFO("send_raw_ethernet_packet: Packet transmitted successfully on %s\n", ifname);
    }
    /* Release the reference to the network device */
    dev_put(dev);
    return ret;
}
```

## 4.6 eba_net.h

Go to the documentation of this file.

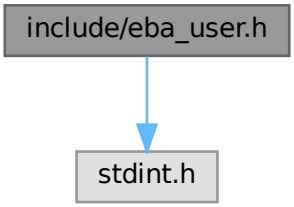
```
#ifndef EBA_NET
#define EBA_NET
#include <linux/module.h>
#include <linux/netdevice.h>
#include <linux/skbuff.h>
#include <linux/if_ether.h>
#include <linux/init.h>
#include <linux/errno.h>
#include <linux/string.h>
#include "ebp.h"

int send_raw_ethernet_packet(const unsigned char *payload, size_t payload_len,const unsigned char *dst_mac,int protocol,const char *ifname);

int eba_net_get_max_mtu(const char *ifname, int *max_mtu);

int eba_net_set_mtu(const char *ifname, int new_mtu);

char *build_discover_req_packet(uint16_t mtu, uint64_t *out_len);

char *build_discover_ack_packet(uint64_t buffer_id, uint64_t *out_len);

char *build_invoke_req_packet(uint32_t iid, uint32_t opid,
    const char *args, uint64_t args_len,
    const char *payload, uint64_t payload_len,
    uint64_t *out_len);

char *build_invoke_ack_packet(uint8_t status, uint64_t *out_len);

int send_discover_req_packet(uint16_t mtu, const unsigned char dest_mac[6], const char *ifname);

int send_discover_ack_packet(uint64_t buffer_id, const unsigned char dest_mac[6], const char *ifname);

int send_invoke_req_packet(uint32_t iid, uint32_t opid,
    const char *args, uint64_t args_len,
    const void *payload, uint64_t payload_len,
    const unsigned char dest_mac[6],
    const char *ifname);



int send_invoke_ack_packet(uint8_t status,
                           const unsigned char dest_mac[6],
                           const char *ifname);

#endif
```

## 4.7 include/eba_user.h File Reference

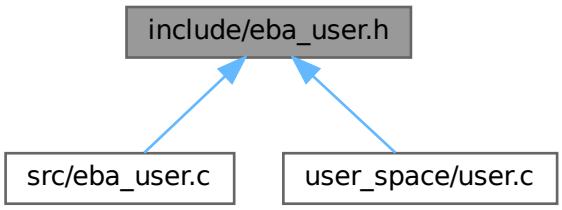
EBA User API Header.

`#include <stdint.h>`

Include dependency graph for eba_user.h:



This graph shows which files directly or indirectly include this file:




**Functions**

- uint64_t eba_alloc (uint64_t size, uint64_t life_time, uint8_t type)
- int eba_write (const void ∗data, uint64_t buff_id, uint64_t off, uint64_t size)
- int eba_read (void ∗data_out, uint64_t buff_id, uint64_t off, uint64_t size)
- int eba_remote_alloc (uint64_t size, uint64_t life_time, uint64_t local_buff_id, const char mac[6])
- int eba_remote_write (uint64_t buff_id, uint64_t offset, uint64_t size, const char ∗payload, const char mac[6])
- int eba_remote_read (uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset, uint64_t src_offset, uint64_t size, const char mac[6])

## 4.7.1 Detailed Description

EBA User API Header.

This header declares the user-space API functions for local and remote buffer operations within the EBA system. The API includes interfaces for buffer allocation, read and write operations on locally allocated buffers, and remote operations executed via IOCTL. These functions abstract the lower-level details from user-space applications.

Definition in file eba_user.h.

## 4.7.2 Function Documentation

### 4.7.2.1 eba_alloc()

```
uint64_t eba_alloc (
            uint64_t size,
            uint64_t life_time,
            uint8_t type )
```

eba_alloc - Request a buffer from the local node. @size: The size (in bytes) for the buffer allocation. @life_time: The duration of the allocation in seconds; 0 indicates "infinite". @type: The type of memory requested (depends on node specifications).

Return: The allocated buffer ID on success, or 0 on failure.

Definition at line 31 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_alloc_data alloc;

    alloc.size = size;
    alloc.life_time = life_time;
    alloc.buff_id = 0;

    fd = open_eba_device();
    if (fd < 0)
        return 0;

    ret = ioctl(fd, EBA_IOCTL_ALLOC, &alloc);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_ALLOC)");
        return 0;
    }

    return alloc.buff_id;
}
```

#### 4.7.2.2 eba_read()

```
int eba_read (
            void * data_out,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_read - Read data from a locally allocated buffer. @data_out: Pointer to the destination where data should be copied. @buff_id: Identifier of the target local buffer. @off: The offset (in bytes) within the buffer to begin reading. @size: The number of bytes to read.

Return: 0 on success, or 1 on failure.

Definition at line 79 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_rw_data rw;

    rw.buff_id = buf_id;
    rw.off = off;
    rw.size = size;
    rw.user_addr = (uint64_t)data_out;

    fd = open_eba_device();
    if (fd < 0)
        return 1;

    ret = ioctl(fd, EBA_IOCTL_READ, &rw);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_READ)");
        return 1;
    }
    return 0;
}
```

#### 4.7.2.3 eba_remote_alloc()

```
int eba_remote_alloc (
            uint64_t size,
            uint64_t life_time,
            uint64_t local_buff_id,
            const char mac[6] )
```

eba_remote_alloc - Request a buffer allocation on a remote node via IOCTL. @size: The size (in bytes) of the requested buffer. @life_time: The life time for the allocation in seconds; 0 indicates "infinite". @local_buff_id: The local buffer identifier where the remote buffer ID will be stored. @mac: The MAC address of the remote node.

Return: 0 on success, or a negative error code on failure.

Definition at line 104 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_remote_alloc remote;

    /* Fill in the remote_alloc structure */
    memset(&remote, 0, sizeof(remote));
    remote.size = size;
    remote.life_time = life_time;
    remote.buffer_id = local_buff_id;
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_ALLOC, &remote);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_REMOTE_ALLOC)");
        return 1;
    }
    return 0;
}
```

#### 4.7.2.4 eba_remote_read()

```
int eba_remote_read (
            uint64_t dst_buffer_id,
            uint64_t src_buffer_id,
            uint64_t dst_offset,
            uint64_t src_offset,
            uint64_t size,
            const char mac[6] )
```

eba_remote_read - Read data from a remote allocated buffer via IOCTL. @dst_buffer_id: Local destination buffer identifier. @src_buffer_id: Remote source buffer identifier. @dst_offset: The offset (in bytes) within the destination buffer where data should be written. @src_offset: The offset (in bytes) within the source buffer where reading begins. @size: The number of bytes to read. @mac: The MAC address of the remote node.

Return: 0 on success, or a negative error code on failure.

Definition at line 169 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_remote_read remote;

    /* Fill in the remote_alloc structure */
    remote.dst_buffer_id = dst_buffer_id;
    remote.src_buffer_id= src_buffer_id;
    remote.dst_offset = dst_offset;
    remote.src_offset = src_offset;
    remote.size = size;
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_READ, &remote);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_REMOTE_READ)");
        return 1;
    }
    return 0;
}
```

#### 4.7.2.5 eba_remote_write()

```
int eba_remote_write (
            uint64_t buff_id,
            uint64_t offset,
            uint64_t size,
            const char * payload,
            const char mac[6] )
```

eba_remote_write - Write data to a remote allocated buffer via IOCTL. @buff_id: The identifier of the remote buffer. @offset: The offset (in bytes) within the remote buffer where writing should begin. @size: The number of bytes of the payload to write. @payload: Pointer to the data payload. @mac: The MAC address of the remote node.

Return: 0 on success, or a negative error code on failure.

Definition at line 132 of file eba_user.c.

```
00133 {
00134     int fd, ret;
00135     struct eba_remote_write remote;
00136
00137     /* Fill in the remote_alloc structure */
00138     remote.buff_id = buff_id;
00139     remote.offset = offset;
00140     remote.size = size;
00141     remote.payload = malloc(remote.size);
00142     if(remote.payload == 0)
00143     {
00144         perror("malloc failed");
00145         return 1;
00146     }
00147     memcpy((void *)remote.payload, payload, remote.size);
00148     memcpy((void *)remote.mac, mac, 6);
00149
00150     fd = open("/dev/eba", O_RDWR);
00151     if (fd < 0) {
00152         free(remote.payload);
00153         perror("open(/dev/eba)");
00154         return 1;
00155     }
00156
00157     ret = ioctl(fd, EBA_IOCTL_REMOTE_WRITE, &remote);
00158     close(fd);
00159
00160     if (ret < 0) {
00161         free(remote.payload);
00162         perror("ioctl(EBA_IOCTL_REMOTE_WRITE)");
00163         return 1;
00164     }
00165     free(remote.payload);
00166     return 0;
00167 }
```

#### 4.7.2.6 eba_write()

```
int eba_write (
            const void * data,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_write - Write data to a locally allocated buffer. @data: Pointer to the data to be written. @buff_id: Identifier of the target local buffer. @off: The offset (in bytes) within the buffer to begin writing. @size: The number of bytes to write.

Return: 0 on success, or 1 on failure.

Definition at line 55 of file eba_user.c.

```
00056 {
00057     int fd, ret;
```

```
    struct eba_rw_data rw;

    rw.buff_id = buf_id;
    rw.off = off;
    rw.size = size;
    rw.user_addr = (uint64_t)data;

    fd = open_eba_device();
    if (fd < 0)
        return 1;

    ret = ioctl(fd, EBA_IOCTL_WRITE, &rw);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_WRITE)");
        return 1;
    }
    return 0;
}
```

## 4.8 eba_user.h

Go to the documentation of this file.

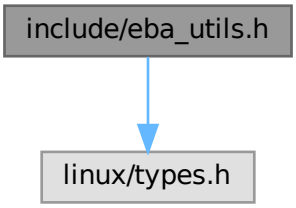
```
#ifndef EBA_USER_H
#define EBA_USER_H
#include <stdint.h>
uint64_t eba_alloc(uint64_t size, uint64_t life_time,uint8_t type);

int eba_write(const void* data, uint64_t buff_id, uint64_t off, uint64_t size);

int eba_read(void* data_out, uint64_t buff_id, uint64_t off, uint64_t size);

/*
 * The following remote API functions use IOCTL calls to communicate with
 * kernel-space routines. The MAC address parameters should eventually be
 * replaced with node IDs.
 */

int eba_remote_alloc(uint64_t size, uint64_t life_time, uint64_t local_buff_id,const char mac[6]/*
TODO modify it to be come node*/);

int eba_remote_write(uint64_t buff_id, uint64_t offset, uint64_t size,const char* payload ,const char
mac[6]/* TODO modify it to be come node*/);

int eba_remote_read(uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset,uint64_t
src_offset ,uint64_t size,const char mac[6]/* TODO modify it to be come node*/);

#endif
```

## 4.9 include/eba_utils.h File Reference

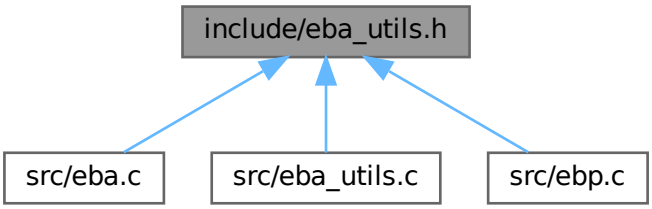
EBA Utilities for File and Buffer Operations.

```
#include <linux/types.h>
```

Include dependency graph for eba_utils.h:




This graph shows which files directly or indirectly include this file:




**Functions**

- int eba_utils_file_to_buf (const char *filepath, uint64_t buff_id)
- int eba_utils_buf_to_file (uint64_t buff_id, uint64_t size, const char *filepath)
- int eba_export_node_specs (void)
- int test_eba_utils_file_to_buf (void)
- int test_eba_utils_buf_to_file (void)
- int test_eba_export_node_specs (void)

### 4.9.1 Detailed Description

EBA Utilities for File and Buffer Operations.

This header provides utility functions to facilitate transferring data between files on disk and EBA buffers. It includes routines to read an entire file into a pre-allocated buffer, write the contents of a buffer to a file, and export node-specific data to disk. Additionally, test functions are provided to verify the correctness of these operations.

Definition in file eba_utils.h.

### 4.9.2 Function Documentation

#### 4.9.2.1 eba_export_node_specs()

```
int eba_export_node_specs (
            void  )
```

eba_export_node_specs - Export node specification buffers to files.

Iterates over the global node_infos[] array and for each node that has a nonzero node_specs pointer (i.e. an EBA buffer address), this function:

1. Determines the size of the EBA buffer.
2. Writes the contents of the buffer to a file located at /var/lib/eba/node_X_specs, where X is the node index.

Return: 0 on success, or a negative error code on failure.

eba_export_node_specs - Exports each node's node_specs buffer into a file.

Definition at line 154 of file eba_utils.c.

```
00155 {
00156     int i;
00157     char filepath[256];
00158
00159     EBA_INFO("Exporting node_specs buffers to files...\n");
00160
00161     for (i = 0; i < MAX_NODE_COUNT; i++) {
00162         if (node_infos[i].id != -1 && node_infos[i].node_specs != 0) {
00163             struct eba_buffer *buf;
00164             int found = 0;
00165             uint64_t buff_id = node_infos[i].node_specs;
00166
00167             /* Acquire global lock, search the buffer list for buff_id */
00168             spin_lock(&eba_buffer_list_lock);
00169             list_for_each_entry(buf, &eba_buffer_list, node) {
00170                 if (buf->address == buff_id) {
00171                     found = 1;
00172                     break;
00173                 }
00174             }
00175             spin_unlock(&eba_buffer_list_lock);
00176
00177             if (!found) {
00178                 EBA_ERR("Node %d has node_specs=0x%llx, but no matching buffer\n",
00179                         i, buff_id);
00180                 continue;
00181             }
00182
00183             /* Build the path like "/var/lib/eba/node_2_specs" */
00184             snprintf(filepath, sizeof(filepath),
00185                      "/var/lib/eba/node_%d_specs", i);
00186
00187             /* Dump the entire buffer to that file */
00188             eba_utils_buf_to_file(buff_id, buf->size, filepath);
00189         }
00190     }
00191
00192     return 0;
00193 }
```

#### 4.9.2.2 eba_utils_buf_to_file()

```
int eba_utils_buf_to_file (
            uint64_t buff_id,
            uint64_t size,
            const char * filepath )
```

eba_utils_buf_to_file - Write the entire contents of an EBA buffer to a file. @buff_id: Identifier of the EBA buffer containing the data. @size: The size of the data in the buffer. @filepath: Absolute path to the destination file on disk (e.g. "/var/lib/eba/dump.eba").

This function writes the content of the EBA buffer, starting from the beginning and spanning @size bytes, to the file specified by @filepath.

Return: 0 on success, or a negative error code on failure.

eba_utils_buf_to_file - Writes data from an EBA buffer to a file. @buff_id: EBA buffer pointer @size: How many bytes to read from the buffer @filepath: Where to write on disk

Returns 0 on success, negative error code on failure.

Definition at line 92 of file eba_utils.c.

```
00093 {
00094     struct file *filp;
00095     loff_t pos = 0;
00096     char *kbuf = NULL;
00097     ssize_t bytes_written;
00098     int ret;
00099
00100     if (size == 0) {
00101         EBA_ERR("eba_utils_buf_to_file: zero size, nothing to write\n");
00102         return -EINVAL;
00103     }
00104
00105     /* Allocate a kernel buffer of 'size' to read from EBA */
00106     kbuf = kmalloc(size, GFP_KERNEL);
00107     if (!kbuf)
00108         return -ENOMEM;
00109
00110     /* Read from EBA buffer into kbuf */
00111     ret = eba_internals_read(kbuf, buff_id, 0, size);
00112     if (ret < 0) {
00113         EBA_ERR("eba_internals_read() failed on buff_id 0x%llx\n", buff_id);
00114         goto out_free;
00115     }
00116
00117     /* Open or create the file (truncate if exists) */
00118     filp = filp_open(filepath, O_WRONLY | O_CREAT | O_TRUNC, 0600);
00119     if (IS_ERR(filp)) {
00120         EBA_ERR("Failed to open file %s for writing, err=%ld\n",
00121                 filepath, PTR_ERR(filp));
00122         ret = PTR_ERR(filp);
00123         filp = NULL;
00124         goto out_free;
00125     }
00126
00127     /* Write kbuf to file */
00128     bytes_written = kernel_write(filp, kbuf, size, &pos);
00129     if (bytes_written < 0 || bytes_written != size) {
00130         EBA_ERR("Failed to write entire buffer to file %s, bytes_written=%zd\n",
00131                 filepath, bytes_written);
00132         if (bytes_written >= 0)
00133             ret = -EIO;
00134         else
00135             ret = bytes_written;
00136         goto out_close;
00137     }
00138
00139     EBA_INFO("Wrote %llu bytes from EBA buffer 0x%llx to file %s\n",
00140              size, buff_id, filepath);
00141     ret = 0; /* success */
00142
00143 out_close:
00144     if (filp)
00145         filp_close(filp, NULL);
00146 out_free:
00147     kfree(kbuf);
00148     return ret;
00149 }
```

#### 4.9.2.3 eba_utils_file_to_buf()

```
int eba_utils_file_to_buf (
            const char * filepath,
            uint64_t buff_id )
```

eba_utils_file_to_buf - Read an entire file into a pre-allocated EBA buffer. @filepath: Absolute path to the file on disk (e.g. "/var/lib/eba/somefile.eba"). @buff_id: Identifier of the pre-allocated buffer.

This function reads the complete contents of the specified file and copies it into the EBA buffer identified by @buff_id.

Return: 0 on success, 1 on error.

eba_utils_file_to_buf - Reads an entire file into an existing EBA buffer. @filepath: Path to the file on disk @buff_id: The EBA buffer ID (already allocated).

Returns 0 on success, 1 on error.

Definition at line 23 of file eba_utils.c.

```
{
    struct file *filp;
    loff_t pos = 0;
    loff_t file_size;
    char *kbuf = NULL;
    ssize_t bytes_read;
    int ret = 1;  /* default to error */

    /* Open the file read-only */
    filp = filp_open(filepath, O_RDONLY, 0);
    if (IS_ERR(filp)) {
        EBA_ERR("Failed to open file %s, err=%ld\n", filepath, PTR_ERR(filp));
        return 1;
    }

    /* Get file size via i_size_read() */
    file_size = i_size_read(file_inode(filp));
    if (file_size <= 0) {
        EBA_ERR("File %s is empty or error reading size\n", filepath);
        goto out_close;
    }

    /* Safety check if extremely large */
    if (file_size > ULONG_MAX) {
        EBA_ERR("File %s is too large (%lld bytes)\n", filepath, file_size);
        goto out_close;
    }

    /* Allocate a kernel buffer to read the file contents into */
    kbuf = kmalloc(file_size, GFP_KERNEL);
    if (!kbuf) {
        EBA_ERR("Failed to allocate temp buffer\n");
        goto out_close;
    }

    /* Read the entire file into kbuf */
    bytes_read = kernel_read(filp, kbuf, file_size, &pos);
    if (bytes_read < 0 || bytes_read != file_size) {
        EBA_ERR("Failed to read entire file %s, bytes_read=%zd\n",
                filepath, bytes_read);
        goto out_free;
    }

    /* Write that data into the EBA buffer buff_id at offset=0 */
    if (eba_internals_write(kbuf, buff_id, 0, file_size) < 0) {
        EBA_ERR("eba_internals_write() failed for file %s\n", filepath);
        goto out_free;
    }

    EBA_INFO("Read %lld bytes from file %s into EBA buffer 0x%llx\n",
             file_size, filepath, buff_id);
    ret = 0; /* success */

out_free:
    kfree(kbuf);
out_close:
    filp_close(filp, NULL);
    return ret;
}
```

#### 4.9.2.4 test_eba_export_node_specs()

```
int test_eba_export_node_specs (
            void  )
```

test_eba_export_node_specs - Self-test for eba_export_node_specs function.

Return: 0 on success, or a non-zero error code on failure.

Definition at line 281 of file eba_utils.c.

```
{
    int node_index = 0;  /* We'll pick node 0 for the test */
    void *buff_id;
    char test_data[] = "Hello from node_0_specs!\n";
    int ret;

    EBA_INFO("=== test_eba_export_node_specs: START ===\n");

    /* 1. Mark node_infos[node_index] as valid */
    node_infos[node_index].id = node_index; /* or some other ID != -1 */

    /* 2. Allocate a buffer to hold the specs data */
    buff_id = eba_internals_malloc(1024, 0); /* 1KB for demonstration */
    if (!buff_id) {
        EBA_ERR("Failed to allocate node_specs buffer for node %d\n", node_index);
        return 1;
    }

    /* 3. Write some data into that buffer */
    ret = eba_internals_write(test_data, (uint64_t)buff_id, 0, strlen(test_data));
    if (ret < 0) {
        EBA_ERR("Failed to write test data into node_specs buffer\n");
        eba_internals_free(buff_id);
        return 1;
    }

    /* 4. Store this buffer in node_infos[node_index].node_specs */
    node_infos[node_index].node_specs = (uint64_t)buff_id;

    /*
     * 5. Call eba_export_node_specs()
     *    This should write the entire buffer content to /var/lib/eba/node_0_specs.
     *    Make sure /var/lib/eba/ directory exists with writable permissions for root.
     */
    ret = eba_export_node_specs();
    if (ret != 0) {
        EBA_ERR("eba_export_node_specs returned error %d\n", ret);
        /* We won't free the buffer here so you can inspect if needed, but normally you might. */
        return 1;
    }

    EBA_INFO("Check /var/lib/eba/node_0_specs to see if it contains: \"%s\"\n", test_data);

    /*
     * 6. (Optional) If you want to clean up this buffer immediately:
     */
    // eba_internals_free(buff_id);
    // node_infos[node_index].node_specs = 0;
    // node_infos[node_index].id = -1;

    EBA_INFO("=== test_eba_export_node_specs: END ===\n");
    return 0; /* success */
}
```

#### 4.9.2.5 test_eba_utils_buf_to_file()

```
int test_eba_utils_buf_to_file (
            void  )
```

test_eba_utils_buf_to_file - Self-test for eba_utils_buf_to_file function.

Return: 0 on success, or a non-zero error code on failure.

Definition at line 236 of file eba_utils.c.

```
{
    void *buff_id;
    char pattern[512];
    int i, ret;

    EBA_INFO("=== test_eba_utils_buf_to_file: START ===\n");

    /* 1. Allocate a 512-byte buffer */
    buff_id = eba_internals_malloc(512, 0);
    if (!buff_id) {
        EBA_ERR("Failed to allocate EBA buffer in test_eba_utils_buf_to_file\n");
        return 1;
    }

    /* 2. Fill local array 'pattern' with some data to demonstrate writing */
    for (i = 0; i < (int)sizeof(pattern); i++) {
        pattern[i] = (char)('A' + (i % 26)); /* 'A'..'Z' repeated */
    }

    /* 3. Write this pattern to the EBA buffer */
    ret = eba_internals_write(pattern, (uint64_t)buff_id, 0, sizeof(pattern));
    if (ret < 0) {
        EBA_ERR("Failed to write pattern to EBA buffer\n");
        eba_internals_free(buff_id);
        return 1;
    }

    /* 4. Now write the EBA buffer to a file: /tmp/test_output */
    ret = eba_utils_buf_to_file((uint64_t)buff_id, sizeof(pattern), "/tmp/test_output");
    if (ret < 0) {
        EBA_ERR("eba_utils_buf_to_file failed, ret=%d\n", ret);
        eba_internals_free(buff_id);
        return 1;
    }

    EBA_INFO("Check /tmp/test_output to see if the 512-byte pattern is there.\n");

    /* 5. Free the buffer */
    eba_internals_free(buff_id);

    EBA_INFO("=== test_eba_utils_buf_to_file: END ===\n");
    return 0; /* success */
}
```

#### 4.9.2.6 test_eba_utils_file_to_buf()

```
int test_eba_utils_file_to_buf (
            void  )
```

test_eba_utils_file_to_buf - Self-test for eba_utils_file_to_buf function.

Return: 0 on success, or a non-zero error code on failure.

Definition at line 195 of file eba_utils.c.

```
{
    void *buff_id;
    int ret;
    char tmpbuf[64]; /* small stack buffer to read/print a piece of data */

    EBA_INFO("=== test_eba_utils_file_to_buf: START ===\n");

    /* 1. Allocate a buffer of 512 bytes with no expiration (lifetime=0). */
    buff_id = eba_internals_malloc(512, 0);
    if (!buff_id) {
        EBA_ERR("Failed to allocate EBA buffer in test_eba_utils_file_to_buf\n");
        return 1;
    }

    /* 2. Read from /tmp/test_input into this buffer.
     *    Make sure /tmp/test_input exists and has <=512 bytes, or adjust code.
     */
    ret = eba_utils_file_to_buf("/tmp/test_input", (uint64_t)buff_id);
    if (ret != 0) {
        EBA_ERR("eba_utils_file_to_buf failed (ret=%d)\n", ret);
        eba_internals_free(buff_id);
        return 1;
    }

    /* 3. Optionally read back a small portion of data from the buffer and print it */
```

```
    memset(tmpbuf, 0, sizeof(tmpbuf));
    if (eba_internals_read(tmpbuf, (uint64_t)buff_id, 0, sizeof(tmpbuf) - 1) == 0) {
        EBA_INFO("First 63 bytes from EBA buffer: \"%s\"\n", tmpbuf);
    } else {
        EBA_ERR("Failed to read back from EBA buffer\n");
    }

    /* 4. Free the EBA buffer */
    eba_internals_free(buff_id);

    EBA_INFO("=== test_eba_utils_file_to_buf: END ===\n");
    return 0; /* success */
}
```

## 4.10 eba_utils.h

Go to the documentation of this file.

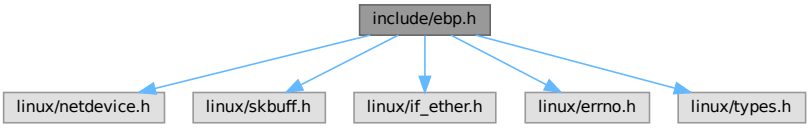
```
#include <linux/types.h>
int eba_utils_file_to_buf(const char *filepath, uint64_t buff_id);

int eba_utils_buf_to_file(uint64_t buff_id, uint64_t size, const char *filepath);


int eba_export_node_specs(void);

int test_eba_utils_file_to_buf(void);

int test_eba_utils_buf_to_file(void);


int test_eba_export_node_specs(void);
```

## 4.11 include/ebp.h File Reference

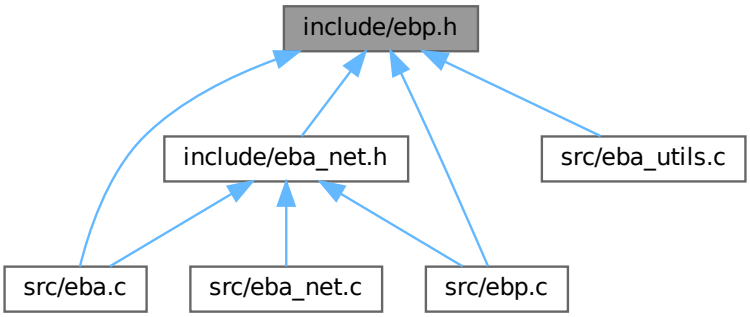
EBA Protocol Header File.

```
#include <linux/netdevice.h>
#include <linux/skbuff.h>
#include <linux/if_ether.h>
#include <linux/errno.h>
#include <linux/types.h>
```

Include dependency graph for ebp.h:



This graph shows which files directly or indirectly include this file:




### Classes

- struct node_info
- struct invoke_tracker
- struct op_entry
- struct ebp_op_write_args
- struct ebp_op_alloc_args
- struct ebp_op_read_args
- struct ebp_header
- struct ebp_discover_req
- struct ebp_discover_ack
- struct ebp_invoke_req
- struct ebp_invoke_ack

### Macros

- #define EBP_ETHERTYPE 0xEBA0
- #define EBP_NODE_SPECS_MAX_SIZE 512
- #define EBP_NODE_SPECS_MAX_LIFE_TIME 0
- #define MAX_NODE_COUNT 10
- #define MAX_INVOKE_COUNT 20
- #define MAX_OP_COUNT 5

### Typedefs

- typedef int(∗ ebp_op_t) (const void ∗args, uint64_t arg_len, const char mac[6])

  *Function pointer type for EBA operations.*

### Enumerations

- enum INVOKE_STATUS {
  INVOKE_QUEUED = 0 , INVOKE_IN_PROGRESS , INVOKE_COMPLETED , INVOKE_FAILED ,
  INVOKE_DEFAULT }
- enum EBP_MSG { EBP_MSG_DISCOVER = 0x01 , EBP_MSG_INVOKE , EBP_MSG_DISCOVER_ACK ,
  EBP_MSG_INVOKE_ACK }
- enum EBP_OP_IDS { EBP_OP_ALLOC = 1 , EBP_OP_READ , EBP_OP_WRITE }

### Functions

- int node_info_array_init (void)
- int invoke_tracker_array_init (void)
- int op_entry_array_init (void)
- struct ebp_op_write_args __attribute__ ((packed))
- int ebp_handle_packets (struct sk_buff ∗skb, struct net_device ∗dev, struct packet_type ∗pt, struct net_device ∗orig_dev)
- void ebp_init (void)
- void ebp_exit (void)
- int ebp_op_alloc (const void ∗args, uint64_t arg_len, const char mac[6])
- int ebp_op_write (const void ∗args, uint64_t arg_len, const char mac[6])
- int ebp_op_read (const void ∗args, uint64_t arg_len, const char mac[6])
- int ebp_register_op (uint32_t op_id, ebp_op_t fn)
- int ebp_invoke_op (uint32_t op_id, const void ∗args, uint64_t arg_len, const char mac[6])
- int ebp_ops_init (void)
- void print_op_entries (void)
- int ebp_remote_alloc (uint64_t size, uint64_t life_time, uint64_t local_buff_id, const char mac[6])
- int ebp_remote_write (uint64_t buff_id, uint64_t offset, uint64_t size, const char ∗payload, const char mac[6])
- int ebp_remote_read (uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset, uint64_t src_offset, uint64_t size, const char mac[6])
- int ebp_register_node (uint16_t mtu, const char mac[6], uint64_t node_specs)
- void print_node_infos (void)

### Variables

- uint64_t buff_id
- uint64_t offset
- uint64_t size
- uint64_t life_time
- uint64_t buffer_id
- uint64_t dst_buffer_id
- uint64_t src_buffer_id
- uint64_t dst_offset
- uint64_t src_offset
- uint8_t msgType
- struct ebp_header header
- uint16_t mtu
- uint32_t iid
- uint32_t opid
- uint64_t args_len
- uint8_t args [ ]
- uint8_t status

## 4.11.1 Detailed Description

EBA Protocol Header File.

This header defines the data structures, constants, and function prototypes required to implement the EB protocol for remote buffer allocation, read/write operations, node discovery, and operation invocation.

The EB protocol uses custom Ethernet frame types for communication between nodes.

Definition in file ebp.h.

## 4.11.2 Macro Definition Documentation

### 4.11.2.1 EBP_ETHERTYPE

```
#define EBP_ETHERTYPE 0xEBA0
```

EBA EtherType used to identify EBA frames in Ethernet communication.

Definition at line 21 of file ebp.h.

### 4.11.2.2 EBP_NODE_SPECS_MAX_LIFE_TIME

```
#define EBP_NODE_SPECS_MAX_LIFE_TIME 0
```

Default maximum lifetime for node specifications buffer.

Definition at line 25 of file ebp.h.

### 4.11.2.3 EBP_NODE_SPECS_MAX_SIZE

```
#define EBP_NODE_SPECS_MAX_SIZE 512
```

Maximum size for the node specification buffer.

Definition at line 23 of file ebp.h.

### 4.11.2.4 MAX_INVOKE_COUNT

```
#define MAX_INVOKE_COUNT 20
```

Maximum number of invocation entries allowed.

Definition at line 29 of file ebp.h.

### 4.11.2.5 MAX_NODE_COUNT

```
#define MAX_NODE_COUNT 10
```

Maximum number of nodes that can be registered.

Definition at line 27 of file ebp.h.

### 4.11.2.6 MAX_OP_COUNT

```
#define MAX_OP_COUNT 5
```

Maximum number of operation entries allowed in the op_entries array.

Definition at line 31 of file ebp.h.

### 4.11.3 Typedef Documentation

#### 4.11.3.1 ebp_op_t

```
typedef int(* ebp_op_t) (const void *args, uint64_t arg_len, const char mac[6])
```

Function pointer type for EBA operations.

This function pointer is used by each op_entry to reference an operation function. Every EBA operation function must accept a pointer to a raw arguments buffer, the length of the arguments, and the destination MAC address.

@args Pointer to the raw arguments buffer. @arg_len Length of the arguments in bytes. @mac Destination MAC address.

@Returns 0 on success, or a negative error code on failure.

Definition at line 90 of file ebp.h.

### 4.11.4 Enumeration Type Documentation

#### 4.11.4.1 EBP_MSG

```
enum EBP_MSG
```

enum EBP_MSG - Enumerates the different message types used in the EBA protocol. @EBP_MSG_DISCOVER: Message to initiate node discovery. @EBP_MSG_INVOKE: Message to request an operation invocation. @EBP↩_MSG_DISCOVER_ACK: Response message for a discovery request. @EBP_MSG_INVOKE_ACK: Acknowledgment message for an invocation request.

**Enumerator**

| | |
|---|---|
| EBP_MSG_DISCOVER | |
| EBP_MSG_INVOKE | |
| EBP_MSG_DISCOVER_ACK | |
| EBP_MSG_INVOKE_ACK | |

Definition at line 151 of file ebp.h.

```
00152 {
00153     EBP_MSG_DISCOVER = 0x01,
00154     EBP_MSG_INVOKE,
00155     EBP_MSG_DISCOVER_ACK,
00156     EBP_MSG_INVOKE_ACK
00157 };
```

#### 4.11.4.2 EBP_OP_IDS

```
enum EBP_OP_IDS
```

enum EBP_OP_IDS - Enumerates exposed EBA operation identifiers.

These identifiers are used to map functions to operations in the op_entry array. @EBP_OP_ALLOC: Operation ID for buffer allocation. @EBP_OP_READ: Operation ID for reading from a buffer. @EBP_OP_WRITE: Operation ID for writing to a buffer.

**Enumerator**

| | |
|---|---|
| EBP_OP_ALLOC | |
| EBP_OP_READ | |
| EBP_OP_WRITE | |

Definition at line 167 of file ebp.h.

```
00167                 {
00168     EBP_OP_ALLOC = 1,
00169     EBP_OP_READ,
00170     EBP_OP_WRITE
00171 };
```

#### 4.11.4.3 INVOKE_STATUS

```
enum INVOKE_STATUS
```

enum INVOKE_STATUS - Possible statuses for an invocation. @INVOKE_QUEUED: Invocation is queued for processing. @INVOKE_IN_PROGRESS: Invocation is currently being processed. @INVOKE_COMPLETED: Invocation has completed successfully. @INVOKE_FAILED: Invocation has failed. @INVOKE_DEFAULT: Default state for an uninitialized invocation.

**Enumerator**

| | |
|---|---|
| INVOKE_QUEUED | |
| INVOKE_IN_PROGRESS | |
| INVOKE_COMPLETED | |
| INVOKE_FAILED | |
| INVOKE_DEFAULT | |

Definition at line 41 of file ebp.h.

```
00041                     {
00042     INVOKE_QUEUED = 0,
00043     INVOKE_IN_PROGRESS,
00044     INVOKE_COMPLETED,
00045     INVOKE_FAILED,
00046     INVOKE_DEFAULT
00047 };
```

### 4.11.5 Function Documentation

#### 4.11.5.1 __attribute__()

```
struct ebp_invoke_ack __attribute__ (
              (packed)  )
```

#### 4.11.5.2 ebp_exit()

```
void ebp_exit (
              void  )
```

ebp_exit - Unregister the EBA packet handler from the Linux networking stack.

Definition at line 178 of file ebp.c.

```
00179 {
00180     dev_remove_pack(&ebp_packet_type);
00181 }
```

#### 4.11.5.3 ebp_handle_packets()

```
int ebp_handle_packets (
            struct sk_buff * skb,
            struct net_device * dev,
            struct packet_type * pt,
            struct net_device * orig_dev )
```

ebp_handle_packets - Callback for processing received Ethernet frames. @skb: Pointer to the socket buffer containing the received packet. @dev: Network device that received the packet. @pt: Pointer to the packet_type structure associated with the packet. @orig_dev: Original network device.

This function is registered with the networking stack to handle Ethernet frames with the EBP_ETHERTYPE protocol. It processes the received packet and returns an appropriate status code.

**Returns**

NET_RX_SUCCESS on successful processing, or an error code if processing fails.

Definition at line 17 of file ebp.c.

```
{
    struct ebp_header *eba_hdr;
    unsigned char *payload;
    int payload_len;
    struct ethhdr *eth;

    if (!skb)
    {
        EBA_ERR("ebp_handle_packets: Packet is NULL\n");
        return NET_RX_DROP;
    }

    /* Verify packet is long enough for an Ethernet header + EBA header */
    if (skb->len < ETH_HLEN + sizeof(struct ebp_header))
    {
        EBA_ERR("ebp_handle_packets: Packet too short on device %s, length: %u\n",dev->name, skb->len);
        kfree_skb(skb);
        return NET_RX_DROP;
    }

    /* Get the Ethernet header */
    eth = eth_hdr(skb);
    EBA_INFO("ebp_handle_packets: Source MAC address: %pM\n", eth->h_source);

    /* EBA header */
    payload = skb->data;
    payload_len = skb->len;
    eba_hdr = (struct ebp_header *)payload;

    EBA_INFO("ebp_handle_packets: Received packet on %s, protocol: 0x%04x, length: %u, EBA msgType: 0x%02x\n",dev->name, ntohs(skb->protocol), skb->len, eba_hdr->msgType);

    switch (eba_hdr->msgType)
    {
    case EBP_MSG_DISCOVER: /* 0x01 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_discover_req))
        {
            EBA_ERR("ebp_handle_packets: Discover Request packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_discover_req *disc = (struct ebp_discover_req *)eba_hdr;
            EBA_INFO("ebp_handle_packets: Discover Request: MTU = %u\n", ntohs(disc->mtu));
            //allocate node specs buffer
            void* node_specs = eba_internals_malloc(EBP_NODE_SPECS_MAX_SIZE,EBP_NODE_SPECS_MAX_LIFE_TIME);
            //register the new node
            //handle errors lol
            int ret = ebp_register_node(ntohs(disc->mtu),eth->h_source,(uint64_t)node_specs);
            if(ret < 0)
            {
                EBA_ERR("ebp_handle_packets: register node failed\n");
                kfree_skb(skb);
```

```
                return NET_RX_DROP;
            }
            //send the discover ack with the allocated buffer to enable remote node to share its data.
            /*int ret = send_discover_ack_packet((uint64_t)node_specs,eth->h_source,"enp0s8");
            if(ret < 0)
            {
                EBA_ERR("ebp_handle_packets: send discover ack packet failed\n");
                kfree_skb(skb);
                return NET_RX_DROP;
            }*/
        }
        break;
    case EBP_MSG_DISCOVER_ACK: /* 0x03 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_discover_ack))
        {
            EBA_ERR("ebp_handle_packets: Discover Ack packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_discover_ack *ack = (struct ebp_discover_ack *)eba_hdr;
            uint64_t buff_id = be64_to_cpu(ack->buffer_id);
            EBA_INFO("ebp_handle_packets: Discover Ack: buffer_id = 0x%llu \n", buff_id);
            //todo handle mtu
            /*int rc = eba_utils_file_to_buf("/var/lib/eba/node_local.eba",(uint64_t)local_specs);
            if (rc < 0) {
                EBA_ERR("ebp_handle_packets: Failed to read node_local.eba, ret=%d\n", rc);
                kfree_skb(skb);
                return NET_RX_DROP;
            } else {
                EBA_INFO("ebp_handle_packets: Read %llu bytes from node_local.eba, sending to remote\n", (uint64_t)4096);
                uint64_t offset = 0;
                for(int i = 0 ; i < 4 ; i++)
                {
                    rc = ebp_remote_write(buff_id, offset, (uint64_t)1000, (char*) (local_specs) + offset, eth->h_source);
                    if (rc < 0) {
                        EBA_ERR("ebp_handle_packets: Failed to ebp_remote_write() node_local.eba content, ret=%d\n", rc);
                    }
                    offset+= 1000;
                }
            }
        */
        }
        break;
    case EBP_MSG_INVOKE: /* 0x02 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_invoke_req))
        {
            EBA_ERR("ebp_handle_packets: Invoke Request packet too short\n");
        }
        else
        {
            struct ebp_invoke_req *inv = (struct ebp_invoke_req *)eba_hdr;
            uint32_t iid = ntohl(inv->iid);
            uint32_t opid = ntohl(inv->opid);
            uint64_t args_len = be64_to_cpu(inv->args_len);

            EBA_INFO("ebp_handle_packets: Invoke Request: IID = %u, OpID = %u, args_len = %llu\n",iid, opid, args_len);
            int ret = ebp_invoke_op(opid, inv->args, args_len, eth->h_source);
            if (ret < 0) {
                EBA_ERR("ebp_handle_packets: ebp_invoke_op(opid=%u) failed ret=%d\n", opid, ret);
                kfree_skb(skb);
                return NET_RX_DROP;
                /* Possibly build & send an error ack or something... TODO */
            } else {
                /* Possibly build an ack for success... TODO */
            }

        }
        break;
    case EBP_MSG_INVOKE_ACK: /* 0x04 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_invoke_ack))
        {
            EBA_ERR("ebp_handle_packets: Invoke Ack packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_invoke_ack *inv_ack = (struct ebp_invoke_ack *)eba_hdr;
            EBA_INFO("ebp_handle_packets: Invoke Ack: status = 0x%02x TODO \n ", inv_ack->status);
        }
        break;
```

```
    default:
        EBA_ERR("ebp_handle_packets: Unknown EBA msgType: 0x%02x\n", eba_hdr->msgType);
        kfree_skb(skb);
        return NET_RX_DROP;
        break;
    }

    kfree_skb(skb);
    return NET_RX_SUCCESS;
}
```

#### 4.11.5.4 ebp_init()

```
void ebp_init (
            void  )
```

ebp_init - Register the EBA packet handler with the Linux networking stack and all other struct arrays related to ebp.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 166 of file ebp.c.

```
{
    dev_add_pack(&ebp_packet_type);
    node_info_array_init();
    invoke_tracker_array_init();
    op_entry_array_init();
    ebp_ops_init();
    local_specs = eba_internals_malloc(4096,0);
    char mac[6] = {0xff,0xff,0xff,0xff,0xff,0xff};
    send_discover_req_packet(69,mac,"enp0s8");
    print_node_infos();
}
```

#### 4.11.5.5 ebp_invoke_op()

```
int ebp_invoke_op (
            uint32_t op_id,
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_invoke_op - Invoke a registered EBA operation. @op_id: Operation identifier to look up. @args: Pointer to the raw argument data. @arg_len: Length of the argument data in bytes. @mac: Destination MAC address of the invoking node.

This function searches for the operation associated with the provided op_id and calls the corresponding function.

**Returns**

The return value of the invoked operation, or a negative error code if not found.

Definition at line 283 of file ebp.c.

```
{
    int i;
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_id == op_id) {
            EBA_INFO("Found op_id %u in slot %d, calling handler...\n", op_id, i);
            return op_entries[i].op_ptr(args, arg_len,mac);
        }
    }
    EBA_ERR("ebp_invoke_op: No matching op_id %u found\n", op_id);
    return -EINVAL;
}
```

#### 4.11.5.6 ebp_op_alloc()

```
int ebp_op_alloc (
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_op_alloc - Handle a remote buffer allocation request. @args: Pointer to the argument provided in the invoke packet. @arg_len: Length of the argument. @mac: Destination MAC address of the invoking node.

This function processes a remote allocation request by parsing the provided arguments and attempting to allocate a buffer accordingly.

**Returns**

0 on success, or 1 on failure.

Definition at line 338 of file ebp.c.

```
00339 {
00340     if (!args || arg_len < sizeof(struct ebp_op_alloc_args)) {
00341         EBA_ERR("ebp_op_alloc: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",
00342                 arg_len, sizeof(struct ebp_op_alloc_args), args);
00343         return -EINVAL;
00344     }
00345
00346     const struct ebp_op_alloc_args *alloc_args = args;
00347     EBA_INFO("ebp_op_alloc: Allocation request received: size = %llu, life_time = %llu, receive
      bufferID = %llu\n",
00348             alloc_args->size, alloc_args->life_time, alloc_args->buffer_id);
00349
00350     void *new_buf = eba_internals_malloc(alloc_args->size, alloc_args->life_time);
00351     if (!new_buf) {
00352         EBA_ERR("ebp_op_alloc: Allocation failed for size %llu\n", alloc_args->size);
00353         return -ENOMEM;
00354     }
00355     uint64_t buf_id = (uint64_t)new_buf;
00356     ebp_remote_write(alloc_args->buffer_id,0,sizeof(buf_id),(char *)&buf_id,mac);
00357     return 0;
00358 }
```

#### 4.11.5.7 ebp_op_read()

```
int ebp_op_read (
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_op_read - Handle a remote read operation request. @args: Pointer to the argument blob from the invoke packet. @arg_len: Length of the argument data. @mac: Destination MAC address of the invoking node.

This function processes a request to read data from a remote buffer.

**Returns**

0 on success, or 1 on failure.

Definition at line 359 of file ebp.c.

```
{
    /* Verify that a valid argument is provided and that it's at least as large as our header. */
    if (!args || arg_len < sizeof(struct ebp_op_read_args)) {
        EBA_ERR("ebp_op_read: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",arg_len, sizeof(struct ebp_op_read_args), args);
        return -EINVAL;
    }
    const struct ebp_op_read_args *rd_args = args;

    EBA_INFO("ebp_op_read: dest_buffer_id = %llu, src_buffer_id = %llu, dst_offset = %llu, src_offset = %llu, size = %llu\n",rd_args->dst_buffer_id,rd_args->src_buffer_id,
         rd_args->dst_offset,rd_args->src_offset, rd_args->size);
    // TODO here inquue to the invoke queue, if it is queued send an ack
    void* read_data = kmalloc(rd_args->size,GFP_KERNEL);
    int ret = eba_internals_read(read_data, rd_args->src_buffer_id, rd_args->src_offset, rd_args->size);
    ebp_remote_write(rd_args->dst_buffer_id,rd_args->dst_offset,rd_args->size,(const char*)read_data,mac);

    kfree(read_data);
    //build packet and send it then free the read_data
    // TODO send the invoke with ( write request to the concerned node with read_data as the payload !).

    if (ret < 0) {
        EBA_ERR("ebp_op_read: eba_internals_read() failed with error %d\n", ret);
        return ret;
    }

    EBA_INFO("ebp_op_read: Successfully read %llu bytes from src_buffer_id %llu into dest_buffer_id %llu\n", rd_args->size, rd_args->src_buffer_id, rd_args->dst_buffer_id);
    return 0;
}
```

#### 4.11.5.8 ebp_op_write()

```
int ebp_op_write (
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_op_write - Handle a remote write operation request. @args: Pointer to the argument from the invoke packet. @arg_len: Length of the argument data. @mac: Destination MAC address of the invoking node.

This function processes a request to write data to a remote buffer.

**Returns**

0 on success, or 1 on failure.

Definition at line 310 of file ebp.c.

```
{
    /* Check that args is not NULL and that its is large enough for our fixed-size header */
    if (!args || arg_len < sizeof(struct ebp_op_write_args))
    {
        EBA_ERR("ebp_op_write: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",arg_len, sizeof(struct ebp_op_write_args), args);
        return -EINVAL;
    }

    const struct ebp_op_write_args *wr = args;
    uint64_t header_size = sizeof(struct ebp_op_write_args);

    if (arg_len < header_size + wr->size)
    {
        EBA_ERR("ebp_op_write: args too short for payload data: arg_len = %llu, header_size = %llu, expected payload size = %llu\n",
                arg_len, header_size, wr->size);
```

```
        return -EINVAL;
    }

    const uint8_t *payload = (const uint8_t *)args + header_size;

    EBA_INFO("ebp_op_write: buff_id = %llx offset = %llu size = %llu\n",wr->buff_id, wr->offset, wr->size);
     // TODO here inqueue to the invoke queue and send an ack
    //print_hex_dump(KERN_INFO, "ebp_op_write payload :", DUMP_PREFIX_OFFSET, 16, 1,payload, wr->size, true);
    send_invoke_ack_packet(INVOKE_QUEUED,mac,"enp0s8");
    return eba_internals_write(payload, wr->buff_id, wr->offset, wr->size);
}
```

#### 4.11.5.9 ebp_ops_init()

```
int ebp_ops_init (
            void  )
```

ebp_ops_init - Initialize and register the default EBA operations.

This function registers the default set of EBA operations (alloc, read, write) into the global op_entries array.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 298 of file ebp.c.

```
{
    int ret = 0;
    if (ebp_register_op(EBP_OP_ALLOC,ebp_op_alloc) < 0)
        ret = -1;
    if (ebp_register_op(EBP_OP_WRITE,ebp_op_write) < 0)
        ret = -1;
    if (ebp_register_op(EBP_OP_READ,ebp_op_read) < 0)
        ret = -1;
    return ret;
}
```

#### 4.11.5.10 ebp_register_node()

```
int ebp_register_node (
            uint16_t mtu,
            const char mac[6],
            uint64_t node_specs )
```

ebp_register_node - Register a new node in the global node_infos array. @mtu: Maximum Transmission Unit (MTU) for the node. @mac: MAC address of the node. @node_specs: Identifier for the buffer containing node specification details.

This function checks if a node with the specified MAC address is already registered. If not, it adds the node to the node_infos array, increments the global node count, and returns 0. If the node is already registered, it returns -EEXIST. If there is no space available, it returns -ENOSPC.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 427 of file ebp.c.

```
{
    int free_slot = -1;
    int i;

    /* Find the first free slot (indicated by id == -1) */
    for (i = 0; i < MAX_NODE_COUNT; i++) {
        if (node_infos[i].id == INVALID_NODE_ID) {
            free_slot = i;
            break;
        }
    }
    if (free_slot == -1) {
        EBA_ERR("ebp_register_node: No space left to register a new node.\n");
        return -ENOSPC;
    }
    /* Register the node in the found free slot */
    node_infos[free_slot].id = nodes_count;
    node_infos[free_slot].mtu = mtu;
    memcpy(node_infos[free_slot].mac, mac, 6);
    node_infos[free_slot].node_specs = node_specs;
    EBA_INFO("ebp_register_node: Registered node with NodeID=%d (slot %d), MAC=%pM, MTU=%u, node_specs=%llu\n",
            node_infos[free_slot].id, free_slot, mac, mtu, node_specs);
    nodes_count++;
    print_node_infos();
    return 0;
}
```

#### 4.11.5.11 ebp_register_op()

```
int ebp_register_op (
            uint32_t op_id,
            ebp_op_t fn )
```

Definition at line 252 of file ebp.c.

```
{
    int i;

    if (!fn) {
        EBA_ERR("eba_register_op: null function pointer\n");
        return -EINVAL;
    }

    /* Check if this op_id is already registered */
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_id == op_id) {
            EBA_ERR("eba_register_op: op_id %u already exists (slot %d)\n", op_id, i);
            return -EEXIST;
        }
    }

    /* Find a free entry */
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_ptr == NULL && op_entries[i].op_id == 0) {
            op_entries[i].op_id  = op_id;
            op_entries[i].op_ptr = fn;
            EBA_INFO("Registered op_id %u in slot %d\n", op_id, i);
            return 0;
        }
    }
    EBA_ERR("eba_register_op: No space for new op_id %u\n", op_id);
    return -ENOSPC;
}
```

#### 4.11.5.12 ebp_remote_alloc()

```
int ebp_remote_alloc (
            uint64_t size,
```

```
            uint64_t life_time,
            uint64_t local_buff_id,
            const char mac[6] )
```

ebp_remote_alloc - Request a buffer allocation from a remote node. @size: Size of the buffer to allocate. @life_↩time: Lifetime of the buffer allocation. @local_buff_id: Local buffer identifier where the allocated buffer's ID will be stored. @mac: MAC address of the remote node (to be modified to a node ID in the future).

This function sends a remote allocation request to a distant node.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 387 of file ebp.c.

```
{
    struct ebp_op_alloc_args alloc_args = {
        .size       = size,
        .life_time  = life_time,
        .buffer_id  = local_buff_id
    };
    send_invoke_req_packet(0x1234,EBP_OP_ALLOC,(char *)&alloc_args,sizeof(alloc_args),NULL,0,mac,"enp0s8");
    EBA_INFO("ebp_remote_alloc: Sent EBP_OP_ALLOC request with local_buf_id = %llu\n",local_buff_id);
    return 0;

}
```

#### 4.11.5.13 ebp_remote_read()

```
int ebp_remote_read (
            uint64_t dst_buffer_id,
            uint64_t src_buffer_id,
            uint64_t dst_offset,
            uint64_t src_offset,
            uint64_t size,
            const char mac[6] )
```

ebp_remote_read - Read data from a remote pre-allocated buffer into a local buffer. @dst_buffer_id: Local buffer identifier where the data will be stored. @src_buffer_id: Remote buffer identifier to read data from. @dst_offset: Byte offset within the local buffer. @src_offset: Byte offset within the remote buffer where reading should start. @size: Number of bytes to read. @mac: MAC address of the remote node (to be modified to a node ID in the future).

This function sends a request to read data from a remote node's buffer into a local buffer.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 413 of file ebp.c.

```
{
    struct ebp_op_read_args read_args = {
        .dst_buffer_id = dst_buffer_id,
        .src_buffer_id = src_buffer_id,
        .dst_offset    = dst_offset,
        .src_offset    = src_offset,
        .size          = size
    };
    send_invoke_req_packet(0x1234,EBP_OP_READ,(char *)&read_args,sizeof(read_args),NULL,0,mac,"enp0s8");
    EBA_INFO("ebp_remote_read: Sent EBP_OP_READ request from  %llu to %llu\n",src_buffer_id,dst_buffer_id);
    return 0;

}
```

#### 4.11.5.14 ebp_remote_write()

```
int ebp_remote_write (
            uint64_t buff_id,
            uint64_t offset,
            uint64_t size,
            const char * payload,
            const char mac[6] )
```

ebp_remote_write - Write data to a remote pre-allocated buffer. @buff_id: Remote buffer identifier. @offset: Byte offset in the remote buffer where writing should begin. @size: Size of the data payload to write. @payload: Pointer to the data payload. @mac: MAC address of the remote node (to be modified to a node ID in the future).

This function sends a request to write data to a remote node's buffer.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 400 of file ebp.c.

```
00401 {
00402     struct ebp_op_write_args write_args = {
00403         .buff_id = buff_id,
00404         .offset = offset,
00405         .size = size
00406     };
00407     send_invoke_req_packet(0x1234,EBP_OP_WRITE,(char*)&write_args,sizeof(write_args),payload,
      write_args.size,mac,"enp0s8");
00408     EBA_INFO("ebp_remote_write: Sent EBP_OP_WRITE request on buffer %llu\n",buff_id);
00409     return 0;
00410
00411 }
```

#### 4.11.5.15 invoke_tracker_array_init()

```
int invoke_tracker_array_init (
            void )
```

invoke_tracker_array_init - Initialize an array of invoke_tracker structures.

This function initializes each element in the invoke_tracker array by setting:

- iid to 0,
- pid to 0,
- done to false,
- status to INVOKE_DEFAULT.

**Returns**

0 on success, or -EINVAL if the provided array pointer is NULL.

Definition at line 229 of file ebp.c.

```
00230 {
00231     uint64_t i;
00232     for (i = 0; i < MAX_INVOKE_COUNT; i++)
00233     {
00234         invoke_trackers[i].iid = 0;
00235         invoke_trackers[i].pid = 0;
00236         invoke_trackers[i].done = false;
00237         invoke_trackers[i].status = INVOKE_DEFAULT;
00238     }
00239     return 0;
00240 }
```

#### 4.11.5.16 node_info_array_init()

```
int node_info_array_init (
            void  )
```

node_info_array_init - Initialize an array of node_info structures.

This function initializes each element in the node_info array by setting:

- id to 0,
- mtu to 0,
- mac to zero,
- node_specs to 0.

**Returns**

0 on success, or -EINVAL if the provided array pointer is NULL.

Definition at line 215 of file ebp.c.

```
00216 {
00217     uint64_t i;
00218     for (i = 0; i < MAX_NODE_COUNT; i++)
00219     {
00220         node_infos[i].id = INVALID_NODE_ID;
00221         node_infos[i].mtu = 0;
00222         /* Zero out the 6-byte MAC address */
00223         memset(node_infos[i].mac, 0, sizeof(node_infos[i].mac));
00224         node_infos[i].node_specs = 0;
00225     }
00226     return 0;
00227 }
```

#### 4.11.5.17 op_entry_array_init()

```
int op_entry_array_init (
            void  )
```

op_entry_array_init - Initialize an array of op_entry structures.

This function initializes each element in the op_entry array by setting:

- op_id to 0,
- op_ptr to NULL.

**Returns**

0 on success, or -EINVAL if the provided array pointer is NULL.

Definition at line 242 of file ebp.c.

```
00243 {
00244     uint64_t i;
00245     for (i = 0; i < MAX_OP_COUNT; i++)
00246     {
00247         op_entries[i].op_id = 0;
00248         op_entries[i].op_ptr = NULL;
00249     }
00250     return 0;
00251 }
```

#### 4.11.5.18 print_node_infos()

```
void print_node_infos (
            void  )
```

Definition at line 189 of file ebp.c.

```
00190 {
00191     int i;
00192     EBA_INFO("=== node_infos Array Dump ===\n");
00193     for (i = 0; i < MAX_NODE_COUNT; i++) {
00194         if (node_infos[i].id != INVALID_NODE_ID) {
00195             EBA_INFO("Slot %d: NodeID=%d, MTU=%u, MAC=%pM, node_specs=%llu\n",
00196                     i,
00197                     node_infos[i].id,
00198                     node_infos[i].mtu,
00199                     node_infos[i].mac,
00200                     node_infos[i].node_specs);
00201         }
00202     }
00203     EBA_INFO("============================\n");
00204 }
```

#### 4.11.5.19 print_op_entries()

```
void print_op_entries (
            void  )
```

print_op_entries - Display the current list of operation entries.

This utility function prints the registered operation entries for debugging and verification.

Definition at line 205 of file ebp.c.

```
00206 {
00207     int i;
00208     EBA_INFO("=== op_entries Array Dump ===\n");
00209     for (i = 0; i < MAX_OP_COUNT; i++) {
00210         EBA_INFO("Slot %d: op_id = %u, op_ptr = %p\n",
00211                  i, op_entries[i].op_id, op_entries[i].op_ptr);
00212     }
00213     EBA_INFO("============================\n");
00214 }
```

### 4.11.6 Variable Documentation

#### 4.11.6.1 args

```
uint8_t args[]
```

Definition at line 4 of file ebp.h.

#### 4.11.6.2 args_len

```
uint64_t args_len
```

Definition at line 3 of file ebp.h.

#### 4.11.6.3 buff_id

```
uint64_t buff_id
```

Definition at line 0 of file ebp.h.

#### 4.11.6.4 buffer_id

```
uint64_t buffer_id
```

Definition at line 2 of file ebp.h.

#### 4.11.6.5 dst_buffer_id

```
uint64_t dst_buffer_id
```

Definition at line 0 of file ebp.h.

#### 4.11.6.6 dst_offset

```
uint64_t dst_offset
```

Definition at line 2 of file ebp.h.

#### 4.11.6.7 header

```
struct ebp_header header
```

Definition at line 0 of file ebp.h.

#### 4.11.6.8 iid

```
uint32_t iid
```

Definition at line 1 of file ebp.h.

#### 4.11.6.9 life_time

```
uint64_t life_time
```

Definition at line 1 of file ebp.h.

#### 4.11.6.10 msgType

```
uint8_t msgType
```

Definition at line 0 of file ebp.h.

#### 4.11.6.11 mtu

```
uint16_t mtu
```

Definition at line 1 of file ebp.h.

#### 4.11.6.12 offset

```
uint64_t offset
```

Definition at line 1 of file ebp.h.

#### 4.11.6.13 opid

```
uint32_t opid
```

Definition at line 2 of file ebp.h.

#### 4.11.6.14 size

```
uint64_t size
```

Definition at line 2 of file ebp.h.

#### 4.11.6.15 src_buffer_id

```
uint64_t src_buffer_id
```

Definition at line 1 of file ebp.h.

#### 4.11.6.16 src_offset

```
uint64_t src_offset
```

Definition at line 3 of file ebp.h.

#### 4.11.6.17 status

```
uint8_t status
```

Definition at line 1 of file ebp.h.

## 4.12 ebp.h

Go to the documentation of this file.

```
00001
00011 #ifndef EBP_H
00012 #define EBP_H
00013 #include <linux/netdevice.h>
00014 #include <linux/skbuff.h>
00015 #include <linux/if_ether.h>
00016 #include <linux/errno.h>
00017 #include <linux/types.h>
00018 /* EBA Protocol Constants */
00019
00021 #define EBP_ETHERTYPE 0xEBA0
00023 #define EBP_NODE_SPECS_MAX_SIZE 512
00025 #define EBP_NODE_SPECS_MAX_LIFE_TIME 0
00027 #define MAX_NODE_COUNT    10
00029 #define MAX_INVOKE_COUNT  20
00031 #define MAX_OP_COUNT      5
00032
00041 enum INVOKE_STATUS {
00042     INVOKE_QUEUED = 0,
00043     INVOKE_IN_PROGRESS,
00044     INVOKE_COMPLETED,
00045     INVOKE_FAILED,
00046     INVOKE_DEFAULT
00047 };
00048
00056 struct node_info {
00057     uint16_t id;
00058     uint16_t mtu;
00059     char mac[6];
00060     uint64_t node_specs;
00061 };
00062
00070 struct invoke_tracker {
00071     uint32_t iid;
00072     pid_t pid;
00073     bool done;
00074     enum INVOKE_STATUS status;
00075 };
00076
00090 typedef int (*ebp_op_t)(const void *args, uint64_t arg_len, const char mac[6]);
00091
00100 struct op_entry {
00101     uint32_t          op_id;
00102     ebp_op_t  op_ptr;
00103
00104 };
00105
00117 int node_info_array_init(void);
00118
00130 int invoke_tracker_array_init(void);
00131
00141 int op_entry_array_init(void);
00142
00143 /* EBA Message Types */
00151 enum EBP_MSG
00152 {
00153     EBP_MSG_DISCOVER = 0x01,
00154     EBP_MSG_INVOKE,
00155     EBP_MSG_DISCOVER_ACK,
00156     EBP_MSG_INVOKE_ACK
00157 };
00158
00167 enum EBP_OP_IDS {
00168     EBP_OP_ALLOC = 1,
00169     EBP_OP_READ,
00170     EBP_OP_WRITE
00171 };
00172
00181 struct ebp_op_write_args {
00182     uint64_t buff_id;
00183     uint64_t offset;
00184     uint64_t size;
00185 }__attribute__((packed));
00186
00187
00196 struct ebp_op_alloc_args {
00197     uint64_t size;
00198     uint64_t life_time;
00199     uint64_t buffer_id;
00200 } __attribute__((packed));
00201
00202
```

```
struct ebp_op_read_args {
    uint64_t dst_buffer_id;
    uint64_t src_buffer_id;
    uint64_t dst_offset;
    uint64_t src_offset;
    uint64_t size;
} __attribute__((packed));


struct ebp_header {
    uint8_t msgType;
} __attribute__((packed));

struct ebp_discover_req {
    struct ebp_header header;
    uint16_t mtu;
} __attribute__((packed));

struct ebp_discover_ack {
    struct ebp_header header;
    uint64_t buffer_id;
} __attribute__((packed));

struct ebp_invoke_req {
    struct ebp_header header;
    uint32_t iid;
    uint32_t opid;
    uint64_t args_len;
    uint8_t args[];
} __attribute__((packed));

struct ebp_invoke_ack {
    struct ebp_header header;
    uint8_t status;
} __attribute__((packed));


int ebp_handle_packets(struct sk_buff *skb, struct net_device *dev, struct packet_type *pt, struct net_device *orig_dev);

void ebp_init(void);

void ebp_exit(void);

int ebp_op_alloc(const void *args, uint64_t arg_len, const char mac[6]);

int ebp_op_write(const void *args, uint64_t arg_len, const char mac[6]);

int ebp_op_read(const void *args, uint64_t arg_len, const char mac[6]);

int ebp_register_op(uint32_t op_id, ebp_op_t fn);

int ebp_invoke_op(uint32_t op_id, const void *args, uint64_t arg_len, const char mac[6]);

int ebp_ops_init(void);

void print_op_entries(void);

int ebp_remote_alloc(uint64_t size, uint64_t life_time, uint64_t local_buff_id,const char mac[6]/* TODO modify it to be come node*/);

int ebp_remote_write(uint64_t buff_id, uint64_t offset, uint64_t size,const char* payload ,const char mac[6]/* TODO modify it to be come node*/);

int ebp_remote_read(uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset,uint64_t src_offset ,uint64_t size,const char mac[6]/* TODO modify it to be come node*/);

int ebp_register_node(uint16_t mtu, const char mac[6], uint64_t node_specs);

/*
    TODO:
    For this node ebp implementation :
        Implement the invoke queue
        Implement the full node discovery
        ( modify the node_info struct to have the args structure so when we respond
        we build the corresponding packet perfectly )
        Add buffer types. ( disk )
        Handle MTU for each operation, make sure it uses the min(local mtu,remote mtu).
        TODO finish the discovery
*/
void print_node_infos(void);
#endif
```

## 4.13 src/eba.c File Reference

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/uaccess.h>
#include <linux/slab.h>
#include <linux/string.h>
#include <linux/timer.h>
#include "eba_internals.h"
#include "eba.h"
#include "eba_net.h"
#include "ebp.h"
#include "eba_utils.h"
```

Include dependency graph for eba.c:

### Macros

- #define DEBUG

### Functions

- static long eba_ioctl (struct file ∗file, unsigned int cmd, unsigned long arg)
- static void eba_timer_callback (struct timer_list ∗t)
- static int eba_open (struct inode ∗inode, struct file ∗file)
- static int eba_release (struct inode ∗inode, struct file ∗file)
- static int __init eba_module_init (void)
- static void __exit eba_module_exit (void)
- module_init (eba_module_init)
- module_exit (eba_module_exit)
- MODULE_LICENSE ("GPL")
- MODULE_AUTHOR ("Nader BEN AMMAR")
- MODULE_DESCRIPTION ("EBA Kernel Module with IOCTL Support")
- MODULE_VERSION ("0.1")

### Variables

- static dev_t eba_devno
- static struct cdev eba_cdev
- static struct class ∗ eba_class = NULL
- static struct timer_list eba_timer
- static const struct file_operations eba_fops

## 4.13.1 Macro Definition Documentation

### 4.13.1.1 DEBUG

```
#define DEBUG
```

Definition at line 12 of file eba.c.

## 4.13.2 Function Documentation

### 4.13.2.1 eba_ioctl()

```
static long eba_ioctl (
            struct file * file,
            unsigned int cmd,
            unsigned long arg )  [static]
```

Definition at line 33 of file eba.c.

```
00034  {
00035      int ret = 0;
00036      struct eba_alloc_data alloc_data;
00037      struct eba_rw_data rw_data;
00038      struct eba_remote_alloc remote_alloc;
00039      struct eba_remote_write remote_write;
00040      struct eba_remote_read remote_read;
00041      void *kbuf; /* Temporary kernel buffer for data transfers */
00042
00043      switch (cmd) {
00044      case EBA_IOCTL_ALLOC:
00045          /* Copy the allocation parameters from user space */
00046          if (copy_from_user(&alloc_data, (void __user *)arg, sizeof(alloc_data)))
00047              return -EFAULT;
00048
00049          alloc_data.buff_id = (uint64_t)eba_internals_malloc(alloc_data.size,
00050                                                              alloc_data.life_time);
00051          if (!alloc_data.buff_id)
00052              ret = -ENOMEM;
00053
00054          /* Return the allocation info to user space */
00055          if (copy_to_user((void __user *)arg, &alloc_data, sizeof(alloc_data)))
00056              ret = -EFAULT;
00057          break;
00058
00059      case EBA_IOCTL_WRITE:
00060          if (copy_from_user(&rw_data, (void __user *)arg, sizeof(rw_data)))
00061              return -EFAULT;
00062          kbuf = kmalloc(rw_data.size, GFP_KERNEL);
00063          if (!kbuf)
00064              return -ENOMEM;
00065          if (copy_from_user(kbuf, (void __user *)(rw_data.user_addr), rw_data.size)) {
00066              kfree(kbuf);
00067              return -EFAULT;
00068          }
00069          ret = eba_internals_write(kbuf, rw_data.buff_id, rw_data.off, rw_data.size);
00070          kfree(kbuf);
00071          break;
00072
00073      case EBA_IOCTL_READ:
00074          if (copy_from_user(&rw_data, (void __user *)arg, sizeof(rw_data)))
00075              return -EFAULT;
00076          kbuf = kmalloc(rw_data.size, GFP_KERNEL);
00077          if (!kbuf)
00078              return -ENOMEM;
00079          ret = eba_internals_read(kbuf, rw_data.buff_id, rw_data.off, rw_data.size);
00080          if (ret) {
00081              kfree(kbuf);
00082              return ret;
00083          }
00084          if (copy_to_user((void __user *)(rw_data.user_addr), kbuf, rw_data.size))
00085              ret = -EFAULT;
00086          kfree(kbuf);
00087          break;
00088
```

```
        case EBA_IOCTL_REMOTE_ALLOC:
            if (copy_from_user(&remote_alloc, (void __user *)arg, sizeof(remote_alloc)))
                return -EFAULT;
            ret = ebp_remote_alloc(remote_alloc.size,remote_alloc.life_time,remote_alloc.buffer_id,remote_alloc.mac);
            break;

        case EBA_IOCTL_REMOTE_WRITE:
            if (copy_from_user(&remote_write, (void __user *)arg, sizeof(remote_write)))
                return -EFAULT;
            ret = ebp_remote_write(remote_write.buff_id,remote_write.offset,remote_write.size,remote_write.payload,remote_write.mac);
            break;
        case EBA_IOCTL_REMOTE_READ:
            if (copy_from_user(&remote_read, (void __user *)arg, sizeof(remote_read)))
                return -EFAULT;
            ret = ebp_remote_read(remote_read.dst_buffer_id,remote_read.src_buffer_id,remote_read.dst_offset,remote_read.src_offset,remote

            break;

            default:
            ret = -ENOTTY;
            break;
        }
        return ret;
    }
```

#### 4.13.2.2 eba_module_exit()

```
static void __exit eba_module_exit (
            void ) [static]
```

Definition at line 197 of file eba.c.

```
    {
        del_timer_sync(&eba_timer);
        eba_internals_mempool_free();
        device_destroy(eba_class, eba_devno);
        class_destroy(eba_class);
        cdev_del(&eba_cdev);
        unregister_chrdev_region(eba_devno, 1);
        ebp_exit();
        EBA_INFO("Module unloaded\n");
    }
```

#### 4.13.2.3 eba_module_init()

```
static int __init eba_module_init (
            void ) [static]
```

Definition at line 146 of file eba.c.

```
    {
        int ret;
        /* Allocate a character device region */
        ret = alloc_chrdev_region(&eba_devno, 0, 1, "eba");
        if (ret < 0)
            goto err_alloc;

        /* Initialize and add the character device */
        cdev_init(&eba_cdev, &eba_fops);
        eba_cdev.owner = THIS_MODULE;
        ret = cdev_add(&eba_cdev, eba_devno, 1);
        if (ret)
            goto err_cdev;

        /* Create a device class using the single-argument version of class_create() */
        eba_class = class_create("eba_class");
        if (IS_ERR(eba_class)) {
            ret = PTR_ERR(eba_class);
            goto err_class;
        }
        /* Create the device node /dev/eba */
        device_create(eba_class, NULL, eba_devno, NULL, "eba");

        /* Initialize the memory pool */
        ret = eba_internals_mempool_init();
```

```
    if (ret) {
        EBA_ERR("Mempool Init failed\n");
        goto err_mempool;
    }
    // Initialize the timer:
    timer_setup(&eba_timer, eba_timer_callback, 0);
    // Schedule the timer for the first time
    mod_timer(&eba_timer, jiffies + msecs_to_jiffies(EBA_CLEAN_BUFFER_CALLBACK_TIMER));

    ebp_init();
    EBA_INFO("Module loaded\n");
    return 0;

err_mempool:
    device_destroy(eba_class, eba_devno);
    class_destroy(eba_class);
err_class:
    cdev_del(&eba_cdev);
err_cdev:
    unregister_chrdev_region(eba_devno, 1);
err_alloc:
    return ret;
}
```

#### 4.13.2.4 eba_open()

```
static int eba_open (
            struct inode * inode,
            struct file * file )  [static]
```

Definition at line 123 of file eba.c.

```
{
    if (!try_module_get(THIS_MODULE))
        return -ENODEV;
    return 0;
}
```

#### 4.13.2.5 eba_release()

```
static int eba_release (
            struct inode * inode,
            struct file * file )  [static]
```

Definition at line 130 of file eba.c.

```
{
    module_put(THIS_MODULE);
    return 0;
}
```

#### 4.13.2.6 eba_timer_callback()

```
static void eba_timer_callback (
            struct timer_list * t )  [static]
```

Definition at line 117 of file eba.c.

```
{
    eba_check_expired_buffers();
    mod_timer(&eba_timer, jiffies + msecs_to_jiffies(EBA_CLEAN_BUFFER_CALLBACK_TIMER)); // reschedule
for next second
}
```

#### 4.13.2.7 MODULE_AUTHOR()

```
MODULE_AUTHOR (
            "Nader BEN AMMAR"  )
```

#### 4.13.2.8 MODULE_DESCRIPTION()

```
MODULE_DESCRIPTION (
            "EBA Kernel Module with IOCTL Support"  )
```

#### 4.13.2.9 module_exit()

```
module_exit (
            eba_module_exit  )
```

#### 4.13.2.10 module_init()

```
module_init (
            eba_module_init  )
```

#### 4.13.2.11 MODULE_LICENSE()

```
MODULE_LICENSE (
            "GPL"  )
```

#### 4.13.2.12 MODULE_VERSION()

```
MODULE_VERSION (
            "0.1"  )
```

### 4.13.3 Variable Documentation

#### 4.13.3.1 eba_cdev

```
struct cdev eba_cdev  [static]
```

Definition at line 29 of file eba.c.

#### 4.13.3.2 eba_class

```
struct class* eba_class = NULL  [static]
```

Definition at line 30 of file eba.c.

#### 4.13.3.3 eba_devno

```
dev_t eba_devno  [static]
```

Definition at line 28 of file eba.c.

#### 4.13.3.4 eba_fops

```
const struct file_operations eba_fops  [static]
```

**Initial value:**

```
= {
     .owner = THIS_MODULE,
     .open = eba_open,
     .release = eba_release,
     .unlocked_ioctl = eba_ioctl,
 }
```

Definition at line 138 of file eba.c.

```
00138                                                   {
00139      .owner = THIS_MODULE,
00140      .open = eba_open,
00141      .release = eba_release,
00142      .unlocked_ioctl = eba_ioctl,
00143  };
```

#### 4.13.3.5 eba_timer

```
struct timer_list eba_timer  [static]
```

Definition at line 114 of file eba.c.

## 4.14 eba.c

Go to the documentation of this file.

```
00001 /*
00002  * eba.c - EBA Kernel Module with IOCTL Support
00003  *
00004  * This module demonstrates a simple character device that supports IOCTL commands.
00005  * It registers a device node (/dev/eba) so that user programs can interact with the
00006  * memory pool via IOCTL calls for allocation, read, and write operations.
00007  *
00008  * Author: Nader BEN AMMAR
00009  * License: GPL
00010  * Version: 1.0
00011  */
00012  #define DEBUG
00013  #include <linux/module.h>       /* Core header for loading LKMs into the kernel */
00014  #include <linux/init.h>         /* Macros: __init, __exit */
00015  #include <linux/fs.h>           /* File operations, dev_t, etc. */
00016  #include <linux/cdev.h>         /* Character device registration */
00017  #include <linux/device.h>       /* Device class creation */
00018  #include <linux/uaccess.h>      /* copy_to_user(), copy_from_user() */
00019  #include <linux/slab.h>         /* kmalloc() and kfree() */
00020  #include <linux/string.h>       /* memcpy(), memcmp() */
00021  #include <linux/timer.h>
00022  #include "eba_internals.h"      /* Internal memory pool functions */
00023  #include "eba.h"                /* Public definitions, IOCTL commands, structures */
00024  #include "eba_net.h"
00025  #include "ebp.h"
00026  #include "eba_utils.h"
00027  /* Global variables for character device registration */
00028  static dev_t eba_devno;
00029  static struct cdev eba_cdev;
00030  static struct class *eba_class = NULL;
00031
00032  /* IOCTL handler: dispatches commands to allocation, read, and write operations */
00033  static long eba_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
00034  {
00035      int ret = 0;
00036      struct eba_alloc_data alloc_data;
00037      struct eba_rw_data rw_data;
00038      struct eba_remote_alloc remote_alloc;
00039      struct eba_remote_write remote_write;
00040      struct eba_remote_read remote_read;
00041      void *kbuf; /* Temporary kernel buffer for data transfers */
00042
00043      switch (cmd) {
```

```
    case EBA_IOCTL_ALLOC:
        /* Copy the allocation parameters from user space */
        if (copy_from_user(&alloc_data, (void __user *)arg, sizeof(alloc_data)))
            return -EFAULT;

        alloc_data.buff_id = (uint64_t)eba_internals_malloc(alloc_data.size,
                                                            alloc_data.life_time);
        if (!alloc_data.buff_id)
            ret = -ENOMEM;

        /* Return the allocation info to user space */
        if (copy_to_user((void __user *)arg, &alloc_data, sizeof(alloc_data)))
            ret = -EFAULT;
        break;

    case EBA_IOCTL_WRITE:
        if (copy_from_user(&rw_data, (void __user *)arg, sizeof(rw_data)))
            return -EFAULT;
        kbuf = kmalloc(rw_data.size, GFP_KERNEL);
        if (!kbuf)
            return -ENOMEM;
        if (copy_from_user(kbuf, (void __user *)(rw_data.user_addr), rw_data.size)) {
            kfree(kbuf);
            return -EFAULT;
        }
        ret = eba_internals_write(kbuf, rw_data.buff_id, rw_data.off, rw_data.size);
        kfree(kbuf);
        break;

    case EBA_IOCTL_READ:
        if (copy_from_user(&rw_data, (void __user *)arg, sizeof(rw_data)))
            return -EFAULT;
        kbuf = kmalloc(rw_data.size, GFP_KERNEL);
        if (!kbuf)
            return -ENOMEM;
        ret = eba_internals_read(kbuf, rw_data.buff_id, rw_data.off, rw_data.size);
        if (ret) {
            kfree(kbuf);
            return ret;
        }
        if (copy_to_user((void __user *)(rw_data.user_addr), kbuf, rw_data.size))
            ret = -EFAULT;
        kfree(kbuf);
        break;

    case EBA_IOCTL_REMOTE_ALLOC:
        if (copy_from_user(&remote_alloc, (void __user *)arg, sizeof(remote_alloc)))
            return -EFAULT;
        ret = ebp_remote_alloc(remote_alloc.size,remote_alloc.life_time,remote_alloc.buffer_id,remote_alloc.mac);
        break;

    case EBA_IOCTL_REMOTE_WRITE:
        if (copy_from_user(&remote_write, (void __user *)arg, sizeof(remote_write)))
            return -EFAULT;
        ret = ebp_remote_write(remote_write.buff_id,remote_write.offset,remote_write.size,remote_write.payload,remote_write.mac);
        break;
    case EBA_IOCTL_REMOTE_READ:
        if (copy_from_user(&remote_read, (void __user *)arg, sizeof(remote_read)))
            return -EFAULT;
        ret = ebp_remote_read(remote_read.dst_buffer_id,remote_read.src_buffer_id,remote_read.dst_offset,remote_read.src_offset,remote
        break;

        default:
        ret = -ENOTTY;
        break;
    }
    return ret;
}

/* Timer for checking expired buffers */
static struct timer_list eba_timer;

/* Timer callback function */
static void eba_timer_callback(struct timer_list *t)
{
    eba_check_expired_buffers();
    mod_timer(&eba_timer, jiffies + msecs_to_jiffies(EBA_CLEAN_BUFFER_CALLBACK_TIMER)); // reschedule for next second
}

static int eba_open(struct inode *inode, struct file *file)
{
    if (!try_module_get(THIS_MODULE))
        return -ENODEV;
```

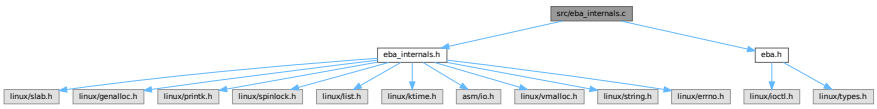
```
    return 0;
}

static int eba_release(struct inode *inode, struct file *file)
{
    module_put(THIS_MODULE);
    return 0;
}


/* File operations structure */
static const struct file_operations eba_fops = {
    .owner = THIS_MODULE,
    .open = eba_open,
    .release = eba_release,
    .unlocked_ioctl = eba_ioctl,
};

/* Module initialization: register character device and initialize memory pool */
static int __init eba_module_init(void)
{
    int ret;
    /* Allocate a character device region */
    ret = alloc_chrdev_region(&eba_devno, 0, 1, "eba");
    if (ret < 0)
        goto err_alloc;

    /* Initialize and add the character device */
    cdev_init(&eba_cdev, &eba_fops);
    eba_cdev.owner = THIS_MODULE;
    ret = cdev_add(&eba_cdev, eba_devno, 1);
    if (ret)
        goto err_cdev;

    /* Create a device class using the single-argument version of class_create() */
    eba_class = class_create("eba_class");
    if (IS_ERR(eba_class)) {
        ret = PTR_ERR(eba_class);
        goto err_class;
    }
    /* Create the device node /dev/eba */
    device_create(eba_class, NULL, eba_devno, NULL, "eba");

    /* Initialize the memory pool */
    ret = eba_internals_mempool_init();
    if (ret) {
        EBA_ERR("Mempool Init failed\n");
        goto err_mempool;
    }
    // Initialize the timer:
    timer_setup(&eba_timer, eba_timer_callback, 0);
    // Schedule the timer for the first time
    mod_timer(&eba_timer, jiffies + msecs_to_jiffies(EBA_CLEAN_BUFFER_CALLBACK_TIMER));

    ebp_init();
    EBA_INFO("Module loaded\n");
    return 0;

err_mempool:
    device_destroy(eba_class, eba_devno);
    class_destroy(eba_class);
err_class:
    cdev_del(&eba_cdev);
err_cdev:
    unregister_chrdev_region(eba_devno, 1);
err_alloc:
    return ret;
}

/* Module cleanup: free memory pool and unregister character device */
static void __exit eba_module_exit(void)
{
    del_timer_sync(&eba_timer);
    eba_internals_mempool_free();
    device_destroy(eba_class, eba_devno);
    class_destroy(eba_class);
    cdev_del(&eba_cdev);
    unregister_chrdev_region(eba_devno, 1);
    ebp_exit();
    EBA_INFO("Module unloaded\n");
}
module_init(eba_module_init);
module_exit(eba_module_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Nader BEN AMMAR");
MODULE_DESCRIPTION("EBA Kernel Module with IOCTL Support");
```

```
MODULE_VERSION("0.1");
```

## 4.15 src/eba_internals.c File Reference

```
#include "eba_internals.h"
#include "eba.h"
```

Include dependency graph for eba_internals.c:




### Functions

- LIST_HEAD (eba_buffer_list)
- int eba_internals_mempool_init (void)

  *Initializes the memory pool for the EBA module.*
- void ∗ eba_internals_malloc (uint64_t size, uint64_t life_time)

  *Allocates a block of memory from the EBA memory pool.*
- int eba_internals_free (void ∗ptr)

  *Frees a previously allocated memory block from the EBA memory pool.*
- void eba_internals_mempool_free (void)

  *Frees the entire memory pool for the EBA module.*
- int eba_internals_mem_stress_test (void)

  *Performs a stress test on the EBA memory pool.*
- int eba_internals_write (const void ∗data, uint64_t buff_id, uint64_t off, uint64_t size)
- int eba_internals_read (void ∗data_out, uint64_t buff_id, uint64_t off, uint64_t size)
- int eba_internals_rw_stress_test (void)
- void eba_check_expired_buffers (void)

### Variables

- spinlock_t eba_buffer_list_lock
- struct gen_pool ∗ eba_pool = NULL
- void ∗ eba_pool_mem = NULL

### 4.15.1 Function Documentation

#### 4.15.1.1 eba_check_expired_buffers()

```
void eba_check_expired_buffers (
            void  )
```

eba_check_expired_buffers - Free buffers that have passed their expiration time.

This function iterates over the global tracking list of allocated buffers. If a buffer's expiration time is nonzero (i.e., its lifetime is not infinite) and the current time is equal to or greater than the expiration time, the buffer is removed from the list, freed from the generic memory pool, and its tracking structure is deallocated.

**Note**

: This function should be called periodically from a timer or workqueue to reclaim expired buffers.

Definition at line 484 of file eba_internals.c.

```
{
    struct eba_buffer *buf, *tmp;
    ktime_t now = ktime_get_real_seconds();
    // Acquire the spinlock before iterating the list.
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry_safe(buf, tmp, &eba_buffer_list, node) {
        // If expires is 0, the lifetime is infinite, so skip.
        if (buf->expires != 0 && now >= buf->expires) {
            EBA_INFO("EBA: Buffer at %llu expired (current time %lld >= expires %lld)\n",buf->address, now,buf->expires);
            // Remove the entry from the tracking list
            list_del(&buf->node);
            // Free the allocated memory from the gen_pool
            gen_pool_free(eba_pool, buf->address, buf->size);
            // Free the tracking structure itself
            kfree(buf);
        }
    }
    spin_unlock(&eba_buffer_list_lock);
}
```

#### 4.15.1.2 eba_internals_free()

```
int eba_internals_free (
            void * ptr )
```

Frees a previously allocated memory block from the EBA memory pool.

This function looks up the tracking structure associated with the provided pointer, removes it from the global tracking list, frees the memory block from the gen_pool, and then frees the tracking structure.

**Parameters**

| | |
|---|---|
| *ptr* | The pointer to the memory block to free. |

**Returns**

0 if the memory was successfully freed, or -1 if the pointer is invalid or not found.

Definition at line 137 of file eba_internals.c.

```
{
    struct eba_buffer *buf;
    int found = 0;

    /* Check if the memory pool is initialized and if the pointer is valid */
    if (!eba_pool || !ptr)
        return -1;

    /* Lock the global list before searching for the tracking structure */
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
        if (buf->address == (unsigned long)ptr) {
            list_del(&buf->node);
            found = 1;
            break;
        }
    }
    spin_unlock(&eba_buffer_list_lock);

    /* If no tracking structure is found, log an error and return -1 */
    if (!found) {
        EBA_ERR("EBA: Attempt to free unknown memory at %llu\n", (uint64_t)ptr);
        return -1;
    }
```

```

    /* Free the memory from the gen_pool using the stored size */
    gen_pool_free(eba_pool, (unsigned long)ptr, buf->size);

    /* Free the tracking structure */
    kfree(buf);

    EBA_INFO("EBA: Freed memory at %llu, size: %llu bytes\n", (uint64_t)ptr, buf->size);
    return 0;
}
```

#### 4.15.1.3 eba_internals_malloc()

```
void * eba_internals_malloc (
            uint64_t size,
            uint64_t life_time )
```

Allocates a block of memory from the EBA memory pool.

This function allocates a block of memory from the generic memory pool, creates a tracking structure to record the allocation information (including the allocated pointer, size, and lifetime), and adds the tracking structure to a global list.

**Parameters**

| | |
|---|---|
| *size* | The number of bytes to allocate. |
| *life_time* | The lifetime of the allocation in seconds. |

**Returns**

A pointer to the allocated memory on success, or NULL if the allocation fails.

Definition at line 73 of file eba_internals.c.

```
{
    unsigned long addr;
    void *ptr;
    struct eba_buffer *buf;

    /* Check if the memory pool is initialized */
    if (!eba_pool)
        return NULL;

    /* Allocate a block of memory from the gen_pool */
    addr = gen_pool_alloc(eba_pool, size);
    if (!addr) {
        EBA_ERR("EBA: Allocation failed for size %llu\n", size);
        return NULL;
    }
    ptr = (void *)addr;
     /* Zero out the allocated memory */
     memset(ptr, 0, size);
    /* Allocate and initialize the tracking structure */
    buf = kmalloc(sizeof(*buf), GFP_KERNEL);
    if (!buf) {
        EBA_ERR("EBA: Tracking structure allocation failed\n");
        gen_pool_free(eba_pool, addr, size);
        return NULL;
    }

    spin_lock_init(&buf->lock);
    buf->type = 0;    /* Set default type (customize if needed) */

    /* Store the allocated pointer */
    buf->address = (unsigned long)ptr;
    buf->size = size;

    /* Set the expiration time using the provided lifetime (in seconds) */
     if (life_time != 0)
         buf->expires = ktime_get_real_seconds() + life_time;
     else
```

```
        buf->expires = 0;
    /* Initialize the list head for tracking */
    INIT_LIST_HEAD(&buf->node);

    /* Add the tracking structure to the global list in a thread-safe manner */
    spin_lock(&eba_buffer_list_lock);
    list_add(&buf->node, &eba_buffer_list);
    spin_unlock(&eba_buffer_list_lock);

    EBA_INFO("EBA: Allocated memory at %llu, size: %llu bytes, lifetime: %llu sec\n",
            (uint64_t)ptr, size, life_time);

    return ptr;
}
```

#### 4.15.1.4 eba_internals_mem_stress_test()

```
int eba_internals_mem_stress_test (
            void  )
```

Performs a stress test on the EBA memory pool.

This function repeatedly allocates and frees memory blocks using eba_internals_malloc() and eba_internals_free() to verify that the memory pool is robust and that allocations/deallocations work correctly.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 222 of file eba_internals.c.

```
{
    const int num_allocs = 100;  /* Number of allocations for the stress test */
    void **allocations;          /* Array to store pointers to allocated memory */
    int *sizes;                  /* Array to store the sizes for each allocation */
    int i, ret;

    /* Dynamically allocate arrays for tracking allocations and their sizes */
    allocations = kmalloc_array(num_allocs, sizeof(void *), GFP_KERNEL);
    if (!allocations)
        return -ENOMEM;

    sizes = kmalloc_array(num_allocs, sizeof(int), GFP_KERNEL);
    if (!sizes) {
        kfree(allocations);
        return -ENOMEM;
    }

    /* Initialize sizes and set each allocation pointer to NULL */
    for (i = 0; i < num_allocs; i++) {
        sizes[i] = 16 + ((i * 31) % 1024);  /* Example size: between 16 and ~1040 bytes */
        allocations[i] = NULL;
    }

    /* Allocation Phase: Allocate memory chunks and verify each allocation */
    for (i = 0; i < num_allocs; i++) {
        allocations[i] = eba_internals_malloc(sizes[i], 10);  /* Lifetime set to 10 sec */
        if (!allocations[i]) {
            EBA_ERR("EBA-STRESS: Allocation %d of size %d failed\n", i, sizes[i]);
            ret = -ENOMEM;
            goto cleanup;
        }
    }
    EBA_INFO("EBA-STRESS: Allocation phase completed successfully\n");

    /* De-allocation Phase: Free all allocated memory chunks */
    for (i = 0; i < num_allocs; i++) {
        ret = eba_internals_free(allocations[i]);
        if (ret != 0) {
            EBA_ERR("EBA-STRESS: Freeing allocation %d failed\n", i);
            goto cleanup;
        }
    }
    EBA_INFO("EBA-STRESS: De-allocation phase completed successfully\n");

    ret = 0;  /* If all allocations and frees succeed, return success */

cleanup:
    /* Free the tracking arrays */
    kfree(allocations);
    kfree(sizes);
    return ret;
}
```

#### 4.15.1.5 eba_internals_mempool_free()

```
void eba_internals_mempool_free (
            void  )
```

Frees the entire memory pool for the EBA module.

This function cleans up by removing and freeing all tracking structures in the global list, destroying the generic memory pool, and releasing the backing memory that was allocated via vmalloc.

Definition at line 179 of file eba_internals.c.

```
{
    struct eba_buffer *buf, *tmp;
    int outstanding = 0;

    // Count outstanding allocations
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
        outstanding++;
    }
    EBA_INFO("EBA: %d outstanding buffers before cleanup\n", outstanding);
    if (outstanding > 0) {
        list_for_each_entry_safe(buf, tmp, &eba_buffer_list, node) {
            EBA_INFO("EBA: Forcing free of buffer at %llu, size: %llu bytes\n",
                    buf->address, buf->size);
            list_del(&buf->node);
            gen_pool_free(eba_pool, buf->address, buf->size);
            kfree(buf);
        }
    }

    // Now destroy the gen_pool and free the backing memory
    if (eba_pool) {
        gen_pool_destroy(eba_pool);
        eba_pool = NULL;
    }
    if (eba_pool_mem) {
        vfree(eba_pool_mem);
        eba_pool_mem = NULL;
    }
    spin_unlock(&eba_buffer_list_lock);
    EBA_INFO("EBA: Memory pool freed successfully\n");
}
```

#### 4.15.1.6 eba_internals_mempool_init()

```
int eba_internals_mempool_init (
            void  )
```

Initializes the memory pool for the EBA module.

This function creates a generic memory pool with an 8-byte alignment, allocates a contiguous block of memory (using kmalloc) to back that pool, registers the allocated memory block with the pool, and initializes the spinlock protecting the global tracking list.

**Returns**

0 if the memory pool is successfully initialized, 1 otherwise.

Definition at line 22 of file eba_internals.c.

```
{
    unsigned long align = 8;    /* Set allocation alignment to 8 bytes */
    unsigned long phys_addr;    /* Dummy physical address for vmalloc'd memory */

    /* Create the generic memory pool.
     * The second parameter (-1) specifies flags; here, no special flags are needed.
     */
    eba_pool = gen_pool_create(align, -1);
    if (!eba_pool) {
```

```
        EBA_ERR("EBA: Failed to create memory pool\n");
        return 1;
    }

    /* Allocate a contiguous block of virtual memory using vmalloc.
     * This method supports very large allocations (subject to system limits) but
     * the returned memory is not physically contiguous.
     */
    eba_pool_mem = vmalloc(MEMORY_POOL_SIZE);
    if (!eba_pool_mem) {
        EBA_ERR("EBA: Failed to allocate memory for the pool\n");
        gen_pool_destroy(eba_pool);
        return 1;
    }

    /* Since vmalloc'd memory is not physically contiguous,
     * we cannot reliably compute a physical base address.
     * We pass a dummy value (e.g., 0) to gen_pool_add_virt().
     */
    phys_addr = 0;
    gen_pool_add_virt(eba_pool, (unsigned long)eba_pool_mem, phys_addr, MEMORY_POOL_SIZE, -1);

    /* Initialize the spinlock to protect the global tracking list */
    spin_lock_init(&eba_buffer_list_lock);

    EBA_INFO("EBA: Memory pool initialized successfully\n");
    return 0;
}
```

#### 4.15.1.7 eba_internals_read()

```
int eba_internals_read (
            void * data_out,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_internals_read - Read data from an allocated buffer. @data_out: Pointer to the destination where read data should be stored. @buff_id: Identifier of the buffer (virtual address returned by eba_internals_malloc). @off: Offset (in bytes) within the buffer from which reading should start. @size: Number of bytes to read.

This function locates the allocated buffer corresponding to @buff_id, verifies that the read operation (starting at offset @off for @size bytes) is within bounds, and copies data from the buffer into the provided output pointer.

Return: 0 on success, or a negative error code (e.g., -EINVAL) if the operation fails.

Definition at line 321 of file eba_internals.c.

```
{
    struct eba_buffer *buf;
    int found = 0;

    if (!data_out)
    {
     EBA_ERR("eba_internals_read: data pointer is NULL\n");
     return -EINVAL;
    }

    /* Locate the tracking structure using buff_id (the allocated virtual address) */
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
        if (buf->address == buff_id) {
            found = 1;
            break;
        }
    }
    spin_unlock(&eba_buffer_list_lock);

    if (!found)
    {
         EBA_ERR("eba_internals_read: Buffer with id 0x%llx not found\n", buff_id);
         return -EINVAL;  /* Buffer not found */
    }

    spin_lock(&buf->lock);
        /* Check if the read operation is within bounds */
```

```
    if (off + size > buf->size)
    {
        EBA_ERR("eba_internals_read: Read out of bounds: off(%llu) + size(%llu) > buf->size(%llu)\n", off, size, buf->size);
        spin_unlock(&buf->lock);
        return -EINVAL;
    }

    /* Copy data from the allocated buffer at the specified offset into data_out */
    memcpy(data_out, (void *)(buf->address + off), size);
    spin_unlock(&buf->lock);
    EBA_INFO("EBA: Read %llu bytes at offset %llu on buf %llu\n",size,off,buff_id);
    return 0;
}
```

#### 4.15.1.8 eba_internals_rw_stress_test()

```
int eba_internals_rw_stress_test (
            void )
```

eba_internals_rw_stress_test - Stress test for read/write operations on the EBA memory pool.

This function allocates multiple buffers using eba_internals_malloc(), writes a known data pattern into each buffer with eba_internals_write(), reads the data back using eba_internals_read(), verifies that the read data matches the written data, and then deallocates the buffers using eba_internals_free().

Return: 0 on success, or a negative error code if any operation fails.

Definition at line 377 of file eba_internals.c.

```
{
    const int num_allocs = 100;  /* Number of buffers to allocate for the test */
    void **allocations;          /* Array to store allocated buffer pointers */
    int *sizes;                  /* Array to store the sizes for each buffer */
    int i, ret = 0;
    unsigned char *pattern = NULL;
    unsigned char *read_back = NULL;

    /* Allocate arrays to track allocated buffers and their sizes */
    allocations = kmalloc_array(num_allocs, sizeof(void *), GFP_KERNEL);
    if (!allocations)
        return -ENOMEM;

    sizes = kmalloc_array(num_allocs, sizeof(int), GFP_KERNEL);
    if (!sizes) {
        kfree(allocations);
        return -ENOMEM;
    }

    /* Initialize the sizes and set each allocation pointer to NULL */
    for (i = 0; i < num_allocs; i++) {
        sizes[i] = 16 + ((i * 31) % 1024);  /* Buffer size will vary between 16 and about 1040 bytes */
        allocations[i] = NULL;
    }

    /* Allocation and test phase */
    for (i = 0; i < num_allocs; i++) {
        /* Allocate a buffer with a lifetime of 10 seconds */
        allocations[i] = eba_internals_malloc(sizes[i], 10);
        if (!allocations[i]) {
            EBA_ERR("EBA-RW-STRESS: Allocation %d of size %d failed\n", i, sizes[i]);
            ret = -ENOMEM;
            goto cleanup;
        }

        /* Allocate temporary buffers for writing and reading */
        pattern = kmalloc(sizes[i], GFP_KERNEL);
        if (!pattern) {
            ret = -ENOMEM;
            goto cleanup;
        }
        read_back = kmalloc(sizes[i], GFP_KERNEL);
        if (!read_back) {
            kfree(pattern);
            ret = -ENOMEM;
            goto cleanup;
        }
```

```

        /*
         * Fill the pattern buffer with a test pattern.
         * For example, store a sequence of incrementing values modulo 256.
         */
        {
            int j;
            for (j = 0; j < sizes[i]; j++)
                pattern[j] = (unsigned char)(j & 0xFF);
        }

        /* Write the test pattern into the allocated buffer */
        ret = eba_internals_write(pattern, (uint64_t)allocations[i], 0, sizes[i]);
        if (ret != 0) {
             EBA_ERR("EBA-RW-STRESS: Write failed on buffer %d\n", i);
             kfree(pattern);
             kfree(read_back);
             goto cleanup;
        }

        /* Read back the data into the read_back buffer */
        ret = eba_internals_read(read_back, (uint64_t)allocations[i], 0, sizes[i]);
        if (ret != 0) {
             EBA_ERR("EBA-RW-STRESS: Read failed on buffer %d\n", i);
             kfree(pattern);
             kfree(read_back);
             goto cleanup;
        }

        /* Compare the written pattern with the read-back data */
        if (memcmp(pattern, read_back, sizes[i]) != 0) {
             EBA_ERR("EBA-RW-STRESS: Data mismatch on buffer %d\n", i);
             ret = -EINVAL;
             kfree(pattern);
             kfree(read_back);
             goto cleanup;
        }

        /* Free the temporary buffers */
        kfree(pattern);
        kfree(read_back);

        EBA_INFO("EBA-RW-STRESS: Buffer %d (size %d) verified successfully\n", i, sizes[i]);
    }

    EBA_INFO("EBA-RW-STRESS: All read/write operations completed successfully\n");
    ret = 0;

cleanup:
    /* Free all allocated buffers */
    for (i = 0; i < num_allocs; i++) {
        if (allocations[i])
            eba_internals_free(allocations[i]);
    }
    kfree(allocations);
    kfree(sizes);
    return ret;
}
```

#### 4.15.1.9 eba_internals_write()

```
int eba_internals_write (
            const void * data,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_internals_write - Write data into an allocated buffer. @data: Pointer to the data to write. @buff_id: Identifier of the buffer (virtual address returned by eba_internals_malloc). @off: Offset (in bytes) within the buffer at which writing should begin. @size: Number of bytes to write.

This function locates the allocated buffer corresponding to @buff_id, verifies that the write operation (starting at offset @off for @size bytes) is within bounds, and copies the data into the buffer.

Return: 0 on success, or a negative error code (e.g., -EINVAL) if the operation fails.

Definition at line 276 of file eba_internals.c.

```
{
    struct eba_buffer *buf;
    int found = 0;

    if (!data)
    {
          EBA_ERR("eba_internals_write: data pointer is NULL\n");
         return -EINVAL;
    }

    /* Locate the tracking structure using buff_id (the allocated virtual address) */
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
         if (buf->address == buff_id) {
              found = 1;
              break;
         }
    }
    spin_unlock(&eba_buffer_list_lock);

    if (!found)
    {
          EBA_ERR("eba_internals_write: Buffer with id 0x%llx not found\n", buff_id);
         return -EINVAL;  /* Buffer not found */
    }
     spin_lock(&buf->lock);
    /* Check if the write operation is within bounds */
    if (off + size > buf->size)
    {
          EBA_ERR("eba_internals_write: Write out of bounds: off(%llu) + size(%llu) > buf->size(%llu)\n", off, size, buf->size);
          spin_unlock(&buf->lock);
         return -EINVAL;
    }

    /* Copy the data into the allocated buffer at the specified offset */
    memcpy((void *)(buf->address + off), data, size);
    spin_unlock(&buf->lock);
    EBA_INFO("EBA: Written %llu bytes at offset %llu on buf %llu\n",size,off,buff_id);
    return 0;
}
```

#### 4.15.1.10 LIST_HEAD()

```
LIST_HEAD (
            eba_buffer_list  )
```

### 4.15.2 Variable Documentation

#### 4.15.2.1 eba_buffer_list_lock

```
spinlock_t eba_buffer_list_lock
```

Definition at line 6 of file eba_internals.c.

#### 4.15.2.2 eba_pool

```
struct gen_pool* eba_pool = NULL
```

Definition at line 9 of file eba_internals.c.

#### 4.15.2.3 eba_pool_mem

```
void* eba_pool_mem = NULL
```

Definition at line 11 of file eba_internals.c.

## 4.16 eba_internals.c

Go to the documentation of this file.

```
#include "eba_internals.h"
#include "eba.h"
/* Global list of currently allocated buffers */
LIST_HEAD(eba_buffer_list);
/* Global lock to protect the eba_buffer_list */
spinlock_t eba_buffer_list_lock;

/* The genpool instance used for memory allocation */
struct gen_pool *eba_pool = NULL;
/* The raw memory backing the pool */
void *eba_pool_mem = NULL;
int eba_internals_mempool_init(void)
{
    unsigned long align = 8;    /* Set allocation alignment to 8 bytes */
    unsigned long phys_addr;    /* Dummy physical address for vmalloc'd memory */

    /* Create the generic memory pool.
     * The second parameter (-1) specifies flags; here, no special flags are needed.
     */
    eba_pool = gen_pool_create(align, -1);
    if (!eba_pool) {
        EBA_ERR("EBA: Failed to create memory pool\n");
        return 1;
    }

    /* Allocate a contiguous block of virtual memory using vmalloc.
     * This method supports very large allocations (subject to system limits) but
     * the returned memory is not physically contiguous.
     */
    eba_pool_mem = vmalloc(MEMORY_POOL_SIZE);
    if (!eba_pool_mem) {
        EBA_ERR("EBA: Failed to allocate memory for the pool\n");
        gen_pool_destroy(eba_pool);
        return 1;
    }

    /* Since vmalloc'd memory is not physically contiguous,
     * we cannot reliably compute a physical base address.
     * We pass a dummy value (e.g., 0) to gen_pool_add_virt().
     */
    phys_addr = 0;
    gen_pool_add_virt(eba_pool, (unsigned long)eba_pool_mem, phys_addr, MEMORY_POOL_SIZE, -1);

    /* Initialize the spinlock to protect the global tracking list */
    spin_lock_init(&eba_buffer_list_lock);

    EBA_INFO("EBA: Memory pool initialized successfully\n");
    return 0;
}
void *eba_internals_malloc(uint64_t size, uint64_t life_time)
{
    unsigned long addr;
    void *ptr;
    struct eba_buffer *buf;

    /* Check if the memory pool is initialized */
    if (!eba_pool)
        return NULL;

    /* Allocate a block of memory from the gen_pool */
    addr = gen_pool_alloc(eba_pool, size);
    if (!addr) {
        EBA_ERR("EBA: Allocation failed for size %llu\n", size);
        return NULL;
    }
    ptr = (void *)addr;
     /* Zero out the allocated memory */
     memset(ptr, 0, size);
    /* Allocate and initialize the tracking structure */
    buf = kmalloc(sizeof(*buf), GFP_KERNEL);
    if (!buf) {
        EBA_ERR("EBA: Tracking structure allocation failed\n");
        gen_pool_free(eba_pool, addr, size);
        return NULL;
    }

    spin_lock_init(&buf->lock);
    buf->type = 0;    /* Set default type (customize if needed) */

    /* Store the allocated pointer */
    buf->address = (unsigned long)ptr;
    buf->size = size;
```

```c

    /* Set the expiration time using the provided lifetime (in seconds) */
     if (life_time != 0)
          buf->expires = ktime_get_real_seconds() + life_time;
     else
          buf->expires = 0;
    /* Initialize the list head for tracking */
    INIT_LIST_HEAD(&buf->node);

    /* Add the tracking structure to the global list in a thread-safe manner */
    spin_lock(&eba_buffer_list_lock);
    list_add(&buf->node, &eba_buffer_list);
    spin_unlock(&eba_buffer_list_lock);

    EBA_INFO("EBA: Allocated memory at %llu, size: %llu bytes, lifetime: %llu sec\n",
            (uint64_t)ptr, size, life_time);

    return ptr;
}

int eba_internals_free(void *ptr)
{
    struct eba_buffer *buf;
    int found = 0;

    /* Check if the memory pool is initialized and if the pointer is valid */
    if (!eba_pool || !ptr)
         return -1;

    /* Lock the global list before searching for the tracking structure */
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
         if (buf->address == (unsigned long)ptr) {
              list_del(&buf->node);
              found = 1;
              break;
         }
    }
    spin_unlock(&eba_buffer_list_lock);

    /* If no tracking structure is found, log an error and return -1 */
    if (!found) {
         EBA_ERR("EBA: Attempt to free unknown memory at %llu\n", (uint64_t)ptr);
         return -1;
    }

    /* Free the memory from the gen_pool using the stored size */
    gen_pool_free(eba_pool, (unsigned long)ptr, buf->size);

    /* Free the tracking structure */
    kfree(buf);

    EBA_INFO("EBA: Freed memory at %llu, size: %llu bytes\n", (uint64_t)ptr, buf->size);
    return 0;
}

void eba_internals_mempool_free(void)
{
    struct eba_buffer *buf, *tmp;
    int outstanding = 0;

    // Count outstanding allocations
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
         outstanding++;
    }
    EBA_INFO("EBA: %d outstanding buffers before cleanup\n", outstanding);
    if (outstanding > 0) {
         list_for_each_entry_safe(buf, tmp, &eba_buffer_list, node) {
              EBA_INFO("EBA: Forcing free of buffer at %llu, size: %llu bytes\n",
                      buf->address, buf->size);
              list_del(&buf->node);
              gen_pool_free(eba_pool, buf->address, buf->size);
              kfree(buf);
         }
    }

    // Now destroy the gen_pool and free the backing memory
    if (eba_pool) {
         gen_pool_destroy(eba_pool);
         eba_pool = NULL;
    }
    if (eba_pool_mem) {
         vfree(eba_pool_mem);
         eba_pool_mem = NULL;
    }
    spin_unlock(&eba_buffer_list_lock);
```

```
    EBA_INFO("EBA: Memory pool freed successfully\n");
}

int eba_internals_mem_stress_test(void)
{
    const int num_allocs = 100;  /* Number of allocations for the stress test */
    void **allocations;          /* Array to store pointers to allocated memory */
    int *sizes;                  /* Array to store the sizes for each allocation */
    int i, ret;

    /* Dynamically allocate arrays for tracking allocations and their sizes */
    allocations = kmalloc_array(num_allocs, sizeof(void *), GFP_KERNEL);
    if (!allocations)
        return -ENOMEM;

    sizes = kmalloc_array(num_allocs, sizeof(int), GFP_KERNEL);
    if (!sizes) {
        kfree(allocations);
        return -ENOMEM;
    }

    /* Initialize sizes and set each allocation pointer to NULL */
    for (i = 0; i < num_allocs; i++) {
        sizes[i] = 16 + ((i * 31) % 1024);  /* Example size: between 16 and ~1040 bytes */
        allocations[i] = NULL;
    }

    /* Allocation Phase: Allocate memory chunks and verify each allocation */
    for (i = 0; i < num_allocs; i++) {
        allocations[i] = eba_internals_malloc(sizes[i], 10);  /* Lifetime set to 10 sec */
        if (!allocations[i]) {
            EBA_ERR("EBA-STRESS: Allocation %d of size %d failed\n", i, sizes[i]);
            ret = -ENOMEM;
            goto cleanup;
        }
    }
    EBA_INFO("EBA-STRESS: Allocation phase completed successfully\n");

    /* De-allocation Phase: Free all allocated memory chunks */
    for (i = 0; i < num_allocs; i++) {
        ret = eba_internals_free(allocations[i]);
        if (ret != 0) {
            EBA_ERR("EBA-STRESS: Freeing allocation %d failed\n", i);
            goto cleanup;
        }
    }
    EBA_INFO("EBA-STRESS: De-allocation phase completed successfully\n");

    ret = 0;  /* If all allocations and frees succeed, return success */

cleanup:
    /* Free the tracking arrays */
    kfree(allocations);
    kfree(sizes);
    return ret;
}

int eba_internals_write(const void *data, uint64_t buff_id, uint64_t off, uint64_t size)
{
    struct eba_buffer *buf;
    int found = 0;

    if (!data)
    {
         EBA_ERR("eba_internals_write: data pointer is NULL\n");
        return -EINVAL;
    }

    /* Locate the tracking structure using buff_id (the allocated virtual address) */
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry(buf, &eba_buffer_list, node) {
        if (buf->address == buff_id) {
            found = 1;
            break;
        }
    }
    spin_unlock(&eba_buffer_list_lock);

    if (!found)
    {
         EBA_ERR("eba_internals_write: Buffer with id 0x%llx not found\n", buff_id);
        return -EINVAL;  /* Buffer not found */
    }
     spin_lock(&buf->lock);
    /* Check if the write operation is within bounds */
    if (off + size > buf->size)
    {
```

```
            EBA_ERR("eba_internals_write: Write out of bounds: off(%llu) + size(%llu) > buf->size(%llu)\n", off, size, buf->size);
            spin_unlock(&buf->lock);
           return -EINVAL;
     }

     /* Copy the data into the allocated buffer at the specified offset */
     memcpy((void *)(buf->address + off), data, size);
     spin_unlock(&buf->lock);
     EBA_INFO("EBA: Written %llu bytes at offset %llu on buf %llu\n",size,off,buff_id);
     return 0;
}

/*
 * eba_internals_read - Reads data from an allocated buffer.
 */
int eba_internals_read(void *data_out, uint64_t buff_id, uint64_t off, uint64_t size)
{
     struct eba_buffer *buf;
     int found = 0;

     if (!data_out)
     {
      EBA_ERR("eba_internals_read: data pointer is NULL\n");
      return -EINVAL;
     }

     /* Locate the tracking structure using buff_id (the allocated virtual address) */
     spin_lock(&eba_buffer_list_lock);
     list_for_each_entry(buf, &eba_buffer_list, node) {
          if (buf->address == buff_id) {
               found = 1;
               break;
          }
     }
     spin_unlock(&eba_buffer_list_lock);

     if (!found)
     {
            EBA_ERR("eba_internals_read: Buffer with id 0x%llx not found\n", buff_id);
            return -EINVAL;  /* Buffer not found */
     }

     spin_lock(&buf->lock);
          /* Check if the read operation is within bounds */
     if (off + size > buf->size)
     {
            EBA_ERR("eba_internals_read: Read out of bounds: off(%llu) + size(%llu) > buf->size(%llu)\n", off, size, buf->size);
            spin_unlock(&buf->lock);
            return -EINVAL;
     }

     /* Copy data from the allocated buffer at the specified offset into data_out */
     memcpy(data_out, (void *)(buf->address + off), size);
     spin_unlock(&buf->lock);
     EBA_INFO("EBA: Read %llu bytes at offset %llu on buf %llu\n",size,off,buff_id);
     return 0;
}

/*
 * eba_internals_rw_stress_test - Performs a stress test for read/write operations.
 *
 * For each allocation in the test, this function:
 *   1. Allocates a buffer using eba_internals_malloc() with a given size and lifetime.
 *   2. Creates a test pattern.
 *   3. Uses eba_internals_write() to write the pattern into the allocated buffer.
 *   4. Reads back the data using eba_internals_read() into a temporary buffer.
 *   5. Compares the read data with the original pattern to verify correctness.
 *   6. Frees the allocated buffer with eba_internals_free().
 *
 * The test is performed for a predetermined number of iterations.
 */
int eba_internals_rw_stress_test(void)
{
     const int num_allocs = 100;  /* Number of buffers to allocate for the test */
     void **allocations;          /* Array to store allocated buffer pointers */
     int *sizes;                  /* Array to store the sizes for each buffer */
     int i, ret = 0;
     unsigned char *pattern = NULL;
     unsigned char *read_back = NULL;

     /* Allocate arrays to track allocated buffers and their sizes */
     allocations = kmalloc_array(num_allocs, sizeof(void *), GFP_KERNEL);
     if (!allocations)
          return -ENOMEM;

```

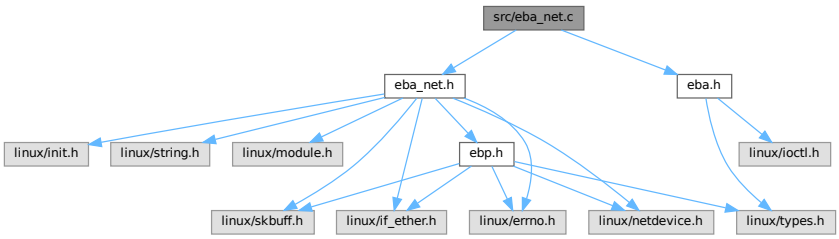
```
    sizes = kmalloc_array(num_allocs, sizeof(int), GFP_KERNEL);
    if (!sizes) {
        kfree(allocations);
        return -ENOMEM;
    }

    /* Initialize the sizes and set each allocation pointer to NULL */
    for (i = 0; i < num_allocs; i++) {
        sizes[i] = 16 + ((i * 31) % 1024);  /* Buffer size will vary between 16 and about 1040 bytes */
        allocations[i] = NULL;
    }

    /* Allocation and test phase */
    for (i = 0; i < num_allocs; i++) {
        /* Allocate a buffer with a lifetime of 10 seconds */
        allocations[i] = eba_internals_malloc(sizes[i], 10);
        if (!allocations[i]) {
            EBA_ERR("EBA-RW-STRESS: Allocation %d of size %d failed\n", i, sizes[i]);
            ret = -ENOMEM;
            goto cleanup;
        }

        /* Allocate temporary buffers for writing and reading */
        pattern = kmalloc(sizes[i], GFP_KERNEL);
        if (!pattern) {
            ret = -ENOMEM;
            goto cleanup;
        }
        read_back = kmalloc(sizes[i], GFP_KERNEL);
        if (!read_back) {
            kfree(pattern);
            ret = -ENOMEM;
            goto cleanup;
        }

        /*
         * Fill the pattern buffer with a test pattern.
         * For example, store a sequence of incrementing values modulo 256.
         */
        {
            int j;
            for (j = 0; j < sizes[i]; j++)
                pattern[j] = (unsigned char)(j & 0xFF);
        }

        /* Write the test pattern into the allocated buffer */
        ret = eba_internals_write(pattern, (uint64_t)allocations[i], 0, sizes[i]);
        if (ret != 0) {
            EBA_ERR("EBA-RW-STRESS: Write failed on buffer %d\n", i);
            kfree(pattern);
            kfree(read_back);
            goto cleanup;
        }

        /* Read back the data into the read_back buffer */
        ret = eba_internals_read(read_back, (uint64_t)allocations[i], 0, sizes[i]);
        if (ret != 0) {
            EBA_ERR("EBA-RW-STRESS: Read failed on buffer %d\n", i);
            kfree(pattern);
            kfree(read_back);
            goto cleanup;
        }

        /* Compare the written pattern with the read-back data */
        if (memcmp(pattern, read_back, sizes[i]) != 0) {
            EBA_ERR("EBA-RW-STRESS: Data mismatch on buffer %d\n", i);
            ret = -EINVAL;
            kfree(pattern);
            kfree(read_back);
            goto cleanup;
        }

        /* Free the temporary buffers */
        kfree(pattern);
        kfree(read_back);

        EBA_INFO("EBA-RW-STRESS: Buffer %d (size %d) verified successfully\n", i, sizes[i]);
    }

    EBA_INFO("EBA-RW-STRESS: All read/write operations completed successfully\n");
    ret = 0;

cleanup:
    /* Free all allocated buffers */
    for (i = 0; i < num_allocs; i++) {
        if (allocations[i])
```

```
            eba_internals_free(allocations[i]);
    }
    kfree(allocations);
    kfree(sizes);
    return ret;
}

void eba_check_expired_buffers(void)
{
    struct eba_buffer *buf, *tmp;
    ktime_t now = ktime_get_real_seconds();
    // Acquire the spinlock before iterating the list.
    spin_lock(&eba_buffer_list_lock);
    list_for_each_entry_safe(buf, tmp, &eba_buffer_list, node) {
        // If expires is 0, the lifetime is infinite, so skip.
        if (buf->expires != 0 && now >= buf->expires) {
            EBA_INFO("EBA: Buffer at %llu expired (current time %lld >= expires %lld)\n",buf->address, now,buf->expires);
            // Remove the entry from the tracking list
            list_del(&buf->node);
            // Free the allocated memory from the gen_pool
            gen_pool_free(eba_pool, buf->address, buf->size);
            // Free the tracking structure itself
            kfree(buf);
        }
    }
    spin_unlock(&eba_buffer_list_lock);
}
```

## 4.17 src/eba_net.c File Reference

```
#include "eba_net.h"
#include "eba.h"
```

Include dependency graph for eba_net.c:




### Functions

- int send_raw_ethernet_packet (const unsigned char ∗payload, size_t payload_len, const unsigned char ∗dst_mac, int protocol, const char ∗ifname)
- int eba_net_get_max_mtu (const char ∗ifname, int ∗max_mtu)
- int eba_net_set_mtu (const char ∗ifname, int new_mtu)
- char ∗ build_discover_req_packet (uint16_t mtu, uint64_t ∗out_len)
- char ∗ build_discover_ack_packet (uint64_t buffer_id, uint64_t ∗out_len)
- char ∗ build_invoke_req_packet (uint32_t iid, uint32_t opid, const char ∗args, uint64_t args_len, const char ∗payload, uint64_t payload_len, uint64_t ∗out_len)
- char ∗ build_invoke_ack_packet (uint8_t status, uint64_t ∗out_len)
- int send_discover_req_packet (uint16_t mtu, const unsigned char dest_mac[6], const char ∗ifname)
- int send_discover_ack_packet (uint64_t buffer_id, const unsigned char dest_mac[6], const char ∗ifname)
- int send_invoke_req_packet (uint32_t iid, uint32_t opid, const char ∗args, uint64_t args_len, const void ∗payload, uint64_t payload_len, const unsigned char dest_mac[6], const char ∗ifname)
- int send_invoke_ack_packet (uint8_t status, const unsigned char dest_mac[6], const char ∗ifname)

### 4.17.1 Function Documentation

#### 4.17.1.1 build_discover_ack_packet()

```
char * build_discover_ack_packet (
            uint64_t buffer_id,
            uint64_t * out_len )
```

build_discover_ack_packet - Build a Discover Acknowledgment packet. @buffer_id: 64-bit identifier to embed in the ACK packet. @out_len: Pointer to a variable that returns the total packet length.

This function builds and returns a newly allocated Discover-Ack packet. The caller is responsible for freeing the packet.

Return: Pointer to the allocated packet on success, or NULL on failure.

Definition at line 134 of file eba_net.c.

```
00135 {
00136     uint64_t len = sizeof(struct ebp_discover_ack);
00137     struct ebp_discover_ack *ack = kmalloc(len, GFP_KERNEL);
00138     if (!ack)
00139         return NULL;
00140     ack->header.msgType = EBP_MSG_DISCOVER_ACK;
00141     ack->buffer_id = cpu_to_be64(buffer_id);
00142     *out_len = len;
00143     return (char *)ack;
00144 }
```

#### 4.17.1.2 build_discover_req_packet()

```
char * build_discover_req_packet (
            uint16_t mtu,
            uint64_t * out_len )
```

build_discover_req_packet - Build a Discover Request packet. @mtu: MTU value to embed in the discover request. @out_len: Pointer to a variable that returns the total packet length.

This function builds and returns a newly allocated Discover Request packet. The caller is responsible for freeing the allocated packet.

Return: Pointer to the packet on success, or NULL on failure.

Definition at line 122 of file eba_net.c.

```
00123 {
00124     uint64_t len = sizeof(struct ebp_discover_req);
00125     struct ebp_discover_req *req = kmalloc(len, GFP_KERNEL);
00126     if (!req)
00127         return NULL;
00128     req->header.msgType = EBP_MSG_DISCOVER;
00129     req->mtu = htons(mtu);
00130     *out_len = len;
00131     return (char *)req;
00132 }
```

#### 4.17.1.3 build_invoke_ack_packet()

```
char * build_invoke_ack_packet (
            uint8_t status,
            uint64_t * out_len )
```

build_invoke_ack_packet - Build an Invoke Acknowledgment packet. @status: 8-bit status code (enum INVOKE_↩ STATUS) to embed in the ACK. @out_len: Pointer to a variable that returns the total packet length.

This function builds and returns a newly allocated Invoke-Ack packet. The caller is responsible for freeing the allocated packet.

Return: Pointer to the allocated packet on success, or NULL on failure.

Definition at line 170 of file eba_net.c.

```
00171 {
00172     uint64_t len = sizeof(struct ebp_invoke_ack);
00173     struct ebp_invoke_ack *ack = kmalloc(len, GFP_KERNEL);
00174     if (!ack)
00175         return NULL;
00176     ack->header.msgType = EBP_MSG_INVOKE_ACK;
00177     ack->status = status;
00178     *out_len = len;
00179     return (char *)ack;
00180 }
```

#### 4.17.1.4 build_invoke_req_packet()

```
char * build_invoke_req_packet (
            uint32_t iid,
            uint32_t opid,
            const char * args,
            uint64_t args_len,
            const char * payload,
            uint64_t payload_len,
            uint64_t * out_len )
```

build_invoke_req_packet - Build an Invoke Request packet. @iid: 32-bit Invocation ID. @opid: 32-bit Operation ID (e.g., EBP_OP_WRITE, EBP_OP_READ). @args: Pointer to the invoke argument data. @args_len: Length (in bytes) of the invoke argument data. @payload: Pointer to additional payload data appended after the arguments. @payload_len: Length (in bytes) of the payload data. @out_len: Pointer to a variable that returns the total packet length.

The packet is built with the following layout:

[ebp_invoke_req header] | [args data] | [payload data]

The caller is responsible for freeing the returned packet.

Return: Pointer to the allocated packet on success, or NULL on failure.

Definition at line 146 of file eba_net.c.

```
00150 {
00151     uint64_t total_len = sizeof(struct ebp_invoke_req) + args_len + payload_len;
00152     char *buf = kmalloc(total_len, GFP_KERNEL);
00153     if (!buf)
00154         return NULL;
00155
00156     struct ebp_invoke_req *req = (struct ebp_invoke_req *)buf;
00157     req->header.msgType = EBP_MSG_INVOKE;
00158     req->iid = htonl(iid);
00159     req->opid = htonl(opid);
00160     /* Set args_len as the sum of the write header length and the payload length */
00161     req->args_len = cpu_to_be64(args_len + payload_len);
00162     if (args && args_len > 0)
00163         memcpy(buf + sizeof(struct ebp_invoke_req), args, args_len);
00164     if (payload && payload_len > 0)
00165         memcpy(buf + sizeof(struct ebp_invoke_req) + args_len, payload, payload_len);
00166     *out_len = total_len;
00167     return buf;
00168 }
```

#### 4.17.1.5 eba_net_get_max_mtu()

```
int eba_net_get_max_mtu (
            const char * ifname,
            int * max_mtu )
```

eba_net_get_max_mtu - Retrieve the maximum MTU supported by the specified network device. @ifname: Network device name (e.g., "eth0"). @max_mtu: Pointer to an integer where the maximum MTU will be stored.

Return: 0 on success or a negative error code on failure.

Definition at line 65 of file eba_net.c.

```
00066 {
00067     struct net_device *dev;
00068
00069     if (!ifname || !max_mtu)
00070         return -EINVAL;
00071
00072     /* Look up the network device by name in the initial network namespace */
00073     dev = dev_get_by_name(&init_net, ifname);
00074     if (!dev) {
00075         EBA_ERR("EBA_NET: Could not find device %s\n", ifname);
00076         return -ENODEV;
00077     }
00078
00079     /* Some drivers set max_mtu in the net_device structure */
00080     *max_mtu = dev->max_mtu;
00081     EBA_INFO("EBA_NET: Device %s current MTU: %d, Maximum supported MTU: %d\n",
00082             dev->name, dev->mtu, dev->max_mtu);
00083
00084     dev_put(dev);
00085     return 0;
00086 }
```

#### 4.17.1.6 eba_net_set_mtu()

```
int eba_net_set_mtu (
            const char * ifname,
            int new_mtu )
```

eba_net_set_mtu - Change the MTU of the specified network device. @ifname: Network device name (e.g., "eth0"). @new_mtu: The new MTU value to set.

Return: 0 on success or a negative error code on failure.

Definition at line 88 of file eba_net.c.

```
00089 {
00090     struct net_device *dev;
00091     int ret = 0;
00092
00093     if (!ifname)
00094         return -EINVAL;
00095
00096     /* Look up the network device by name */
00097     dev = dev_get_by_name(&init_net, ifname);
00098     if (!dev) {
00099         EBA_ERR("EBA_NET: Could not find device %s\n", ifname);
00100         return -ENODEV;
00101     }
00102
00103     /* Check if the device's driver provides the ndo_change_mtu callback */
00104     if (dev->netdev_ops && dev->netdev_ops->ndo_change_mtu) {
00105         ret = dev->netdev_ops->ndo_change_mtu(dev, new_mtu);
00106         if (ret) {
00107             EBA_ERR("EBA_NET: Failed to change MTU on %s to %d, error: %d\n",
00108                     dev->name, new_mtu, ret);
00109         } else {
00110             EBA_INFO("EBA_NET: Successfully changed MTU on %s to %d\n",
00111                      dev->name, new_mtu);
00112         }
00113     } else {
00114         EBA_ERR("EBA_NET: Device %s does not support MTU change\n", dev->name);
00115         ret = -EOPNOTSUPP;
00116     }
00117
00118     dev_put(dev);
00119     return ret;
00120 }
```

#### 4.17.1.7 send_discover_ack_packet()

```
int send_discover_ack_packet (
            uint64_t buffer_id,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_discover_ack_packet - Build and send an EBP_MSG_DISCOVER_ACK packet. @buffer_id: 64-bit buffer identifier to embed in the ACK. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name.

Return: 0 on success or a negative error code on failure.

Definition at line 206 of file eba_net.c.

```
00207 {
00208     uint64_t pkt_len = 0;
00209     char *packet = build_discover_ack_packet(buffer_id, &pkt_len);
00210     int ret;
00211
00212     if (!packet) {
00213         EBA_ERR("send_discover_ack_packet: build_discover_ack_packet() failed.\n");
00214         return -ENOMEM;
00215     }
00216
00217     ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
00218     if (ret < 0) {
00219         EBA_ERR("send_discover_ack_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
00220         kfree(packet);
00221         return ret;
00222     }
00223
00224     kfree(packet);
00225     EBA_INFO("send_discover_ack_packet: Sent EBP_MSG_DISCOVER_ACK to %pM via %s (buffer_id=0x%llx)\n",
00226             dest_mac, ifname, (unsigned long long)buffer_id);
00227     return 0;
00228 }
```

#### 4.17.1.8 send_discover_req_packet()

```
int send_discover_req_packet (
            uint16_t mtu,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_discover_req_packet - Build and send an EBP_MSG_DISCOVER packet. @mtu: MTU value to embed in the discover request. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name (e.g., "enp0s8").

Return: 0 on success or a negative error code on failure.

Definition at line 182 of file eba_net.c.

```
00183 {
00184     uint64_t pkt_len = 0;
00185     char *packet = build_discover_req_packet(mtu, &pkt_len);
00186     int ret;
00187
00188     if (!packet) {
00189         EBA_ERR("send_discover_req_packet: build_discover_req_packet() failed.\n");
00190         return -ENOMEM;
00191     }
00192
00193     ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
00194     if (ret < 0) {
00195         EBA_ERR("send_discover_req_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
00196         kfree(packet);
00197         return ret;
00198     }
00199
00200     kfree(packet);
00201     EBA_INFO("send_discover_req_packet: Sent EBP_MSG_DISCOVER to %pM via %s (MTU = %u)\n",
00202             dest_mac, ifname, mtu);
00203     return 0;
00204 }
```

#### 4.17.1.9 send_invoke_ack_packet()

```
int send_invoke_ack_packet (
            uint8_t status,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_invoke_ack_packet - Build and send an EBP_MSG_INVOKE_ACK response packet. @status: Status code to embed in the ACK. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name.

Return: 0 on success or a negative error code on failure.

Definition at line 262 of file eba_net.c.

```
00265 {
00266     uint64_t pkt_len = 0;
00267     char *packet = build_invoke_ack_packet(status, &pkt_len);
00268     int ret;
00269
00270     if (!packet) {
00271         EBA_ERR("send_invoke_ack_packet: build_invoke_ack_packet() failed.\n");
00272         return -ENOMEM;
00273     }
00274
00275     ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
00276     if (ret < 0) {
00277         EBA_ERR("send_invoke_ack_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
00278         kfree(packet);
00279         return ret;
00280     }
00281
00282     kfree(packet);
00283     EBA_INFO("send_invoke_ack_packet: Sent EBP_MSG_INVOKE_ACK (status=0x%02x) to %pM via %s\n",
00284              status, dest_mac, ifname);
00285     return 0;
00286 }
```

#### 4.17.1.10 send_invoke_req_packet()

```
int send_invoke_req_packet (
            uint32_t iid,
            uint32_t opid,
            const char * args,
            uint64_t args_len,
            const void * payload,
            uint64_t payload_len,
            const unsigned char dest_mac[6],
            const char * ifname )
```

send_invoke_req_packet - Build and send an EBP_MSG_INVOKE request packet. @iid: Invocation ID (caller-defined). @opid: Operation ID (e.g., EBP_OP_WRITE, EBP_OP_READ). @args: Pointer to the arguments buffer. @args_len: Length of the arguments buffer. @payload: Pointer to optional payload data appended after the arguments. @payload_len: Length of the optional payload. @dest_mac: Destination node's MAC address. @ifname: Outgoing interface name.

This is a generic sender for an Invoke Request. Specialized helper functions, such as those implementing remote allocations or writes, may also call build_invoke_req_packet() directly.

Return: 0 on success or a negative error code on failure.

Definition at line 230 of file eba_net.c.

```
00235 {
00236     uint64_t pkt_len = 0;
00237     char *packet = build_invoke_req_packet(iid, opid,
00238                                            args, args_len,
00239                                            payload, payload_len,
00240                                            &pkt_len);
```

```
    int ret;

    if (!packet) {
        EBA_ERR("send_invoke_req_packet: build_invoke_req_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_invoke_req_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_invoke_req_packet: Sent EBP_MSG_INVOKE (IID=%u, OPID=%u) to %pM via %s\n",
             iid, opid, dest_mac, ifname);
    return 0;
}
```

#### 4.17.1.11 send_raw_ethernet_packet()

```
int send_raw_ethernet_packet (
            const unsigned char * payload,
            size_t payload_len,
            const unsigned char * dst_mac,
            int protocol,
            const char * ifname )
```

Definition at line 3 of file eba_net.c.

```
{
    struct net_device *dev = NULL;
    struct sk_buff *skb = NULL;
    struct ethhdr *eth;
    int ret;

    /* Look up the network device by name */
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("send_raw_ethernet_packet: Could not find device %s\n", ifname);
        return -ENODEV;
    }
    /* Calculate total length: Ethernet header + payload */
    size_t packet_len = ETH_HLEN + payload_len;

    /*
     * Allocate an sk_buff.
     */
    skb = alloc_skb(packet_len + dev->needed_tailroom, GFP_KERNEL);
    if (!skb) {
        EBA_ERR("send_raw_ethernet_packet: Failed to allocate sk_buff\n");
        dev_put(dev);
        return -ENOMEM;
    }

    /* Reserve headroom for the Ethernet header */
    skb_reserve(skb, ETH_HLEN);

    /* Set the device for the packet */
    skb->dev = dev;
    skb->protocol = htons(protocol);
    skb->pkt_type = PACKET_OUTGOING;

    /* Build the Ethernet header */
    eth = (struct ethhdr *)skb_push(skb, ETH_HLEN);
    /* Fill in destination MAC address (provided as argument) */
    memcpy(eth->h_dest, dst_mac, ETH_ALEN);
    /* Use the device's own MAC address as source */
    memcpy(eth->h_source, dev->dev_addr, ETH_ALEN);
    /* Set the protocol field */
    eth->h_proto = htons(protocol);

    /* Append payload data to the packet */
    if (payload && payload_len > 0) {
        void *payload_ptr = skb_put(skb, payload_len);
        memcpy(payload_ptr, payload, payload_len);
    }
    /* Queue the packet for transmission */
    ret = dev_queue_xmit(skb);
```

```
    if (ret < 0) {
        EBA_ERR("send_raw_ethernet_packet: Packet transmission failed: %d\n", ret);
        /* In case of error, the kernel should free the skb */
    } else {
        EBA_INFO("send_raw_ethernet_packet: Packet transmitted successfully on %s\n", ifname);
    }
    /* Release the reference to the network device */
    dev_put(dev);
    return ret;
}
```

## 4.18 eba_net.c

Go to the documentation of this file.

```
#include "eba_net.h"
#include "eba.h"
int send_raw_ethernet_packet(const unsigned char *payload, size_t payload_len,const unsigned char
*dst_mac,int protocol,const char *ifname)
{
    struct net_device *dev = NULL;
    struct sk_buff *skb = NULL;
    struct ethhdr *eth;
    int ret;

    /* Look up the network device by name */
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("send_raw_ethernet_packet: Could not find device %s\n", ifname);
        return -ENODEV;
    }
    /* Calculate total length: Ethernet header + payload */
    size_t packet_len = ETH_HLEN + payload_len;

    /*
     * Allocate an sk_buff.
     */
    skb = alloc_skb(packet_len + dev->needed_tailroom, GFP_KERNEL);
    if (!skb) {
        EBA_ERR("send_raw_ethernet_packet: Failed to allocate sk_buff\n");
        dev_put(dev);
        return -ENOMEM;
    }

    /* Reserve headroom for the Ethernet header */
    skb_reserve(skb, ETH_HLEN);

    /* Set the device for the packet */
    skb->dev = dev;
    skb->protocol = htons(protocol);
    skb->pkt_type = PACKET_OUTGOING;

    /* Build the Ethernet header */
    eth = (struct ethhdr *)skb_push(skb, ETH_HLEN);
    /* Fill in destination MAC address (provided as argument) */
    memcpy(eth->h_dest, dst_mac, ETH_ALEN);
    /* Use the device's own MAC address as source */
    memcpy(eth->h_source, dev->dev_addr, ETH_ALEN);
    /* Set the protocol field */
    eth->h_proto = htons(protocol);

    /* Append payload data to the packet */
    if (payload && payload_len > 0) {
        void *payload_ptr = skb_put(skb, payload_len);
        memcpy(payload_ptr, payload, payload_len);
    }
    /* Queue the packet for transmission */
    ret = dev_queue_xmit(skb);
    if (ret < 0) {
        EBA_ERR("send_raw_ethernet_packet: Packet transmission failed: %d\n", ret);
        /* In case of error, the kernel should free the skb */
    } else {
        EBA_INFO("send_raw_ethernet_packet: Packet transmitted successfully on %s\n", ifname);
    }
    /* Release the reference to the network device */
    dev_put(dev);
    return ret;
}


int eba_net_get_max_mtu(const char *ifname, int *max_mtu)
{
```

```
    struct net_device *dev;

    if (!ifname || !max_mtu)
        return -EINVAL;

    /* Look up the network device by name in the initial network namespace */
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("EBA_NET: Could not find device %s\n", ifname);
        return -ENODEV;
    }

    /* Some drivers set max_mtu in the net_device structure */
    *max_mtu = dev->max_mtu;
    EBA_INFO("EBA_NET: Device %s current MTU: %d, Maximum supported MTU: %d\n",
            dev->name, dev->mtu, dev->max_mtu);

    dev_put(dev);
    return 0;
}

int eba_net_set_mtu(const char *ifname, int new_mtu)
{
    struct net_device *dev;
    int ret = 0;

    if (!ifname)
        return -EINVAL;

    /* Look up the network device by name */
    dev = dev_get_by_name(&init_net, ifname);
    if (!dev) {
        EBA_ERR("EBA_NET: Could not find device %s\n", ifname);
        return -ENODEV;
    }

    /* Check if the device's driver provides the ndo_change_mtu callback */
    if (dev->netdev_ops && dev->netdev_ops->ndo_change_mtu) {
        ret = dev->netdev_ops->ndo_change_mtu(dev, new_mtu);
        if (ret) {
            EBA_ERR("EBA_NET: Failed to change MTU on %s to %d, error: %d\n",
                    dev->name, new_mtu, ret);
        } else {
            EBA_INFO("EBA_NET: Successfully changed MTU on %s to %d\n",
                     dev->name, new_mtu);
        }
    } else {
        EBA_ERR("EBA_NET: Device %s does not support MTU change\n", dev->name);
        ret = -EOPNOTSUPP;
    }

    dev_put(dev);
    return ret;
}

char *build_discover_req_packet(uint16_t mtu, uint64_t *out_len)
{
    uint64_t len = sizeof(struct ebp_discover_req);
    struct ebp_discover_req *req = kmalloc(len, GFP_KERNEL);
    if (!req)
        return NULL;
    req->header.msgType = EBP_MSG_DISCOVER;
    req->mtu = htons(mtu);
    *out_len = len;
    return (char *)req;
}

char *build_discover_ack_packet(uint64_t buffer_id, uint64_t *out_len)
{
    uint64_t len = sizeof(struct ebp_discover_ack);
    struct ebp_discover_ack *ack = kmalloc(len, GFP_KERNEL);
    if (!ack)
        return NULL;
    ack->header.msgType = EBP_MSG_DISCOVER_ACK;
    ack->buffer_id = cpu_to_be64(buffer_id);
    *out_len = len;
    return (char *)ack;
}

char *build_invoke_req_packet(uint32_t iid, uint32_t opid,
    const char *args, uint64_t args_len,
    const char *payload, uint64_t payload_len,
    uint64_t *out_len)
{
    uint64_t total_len = sizeof(struct ebp_invoke_req) + args_len + payload_len;
    char *buf = kmalloc(total_len, GFP_KERNEL);
    if (!buf)
```

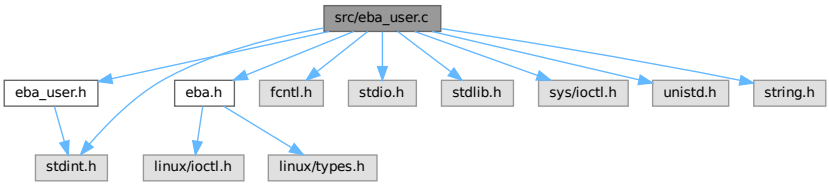
```
        return NULL;

    struct ebp_invoke_req *req = (struct ebp_invoke_req *)buf;
    req->header.msgType = EBP_MSG_INVOKE;
    req->iid = htonl(iid);
    req->opid = htonl(opid);
    /* Set args_len as the sum of the write header length and the payload length */
    req->args_len = cpu_to_be64(args_len + payload_len);
    if (args && args_len > 0)
        memcpy(buf + sizeof(struct ebp_invoke_req), args, args_len);
    if (payload && payload_len > 0)
        memcpy(buf + sizeof(struct ebp_invoke_req) + args_len, payload, payload_len);
    *out_len = total_len;
    return buf;
}

char *build_invoke_ack_packet(uint8_t status, uint64_t *out_len)
{
    uint64_t len = sizeof(struct ebp_invoke_ack);
    struct ebp_invoke_ack *ack = kmalloc(len, GFP_KERNEL);
    if (!ack)
        return NULL;
    ack->header.msgType = EBP_MSG_INVOKE_ACK;
    ack->status = status;
    *out_len = len;
    return (char *)ack;
}

int send_discover_req_packet(uint16_t mtu, const unsigned char dest_mac[6], const char *ifname)
{
    uint64_t pkt_len = 0;
    char *packet = build_discover_req_packet(mtu, &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_discover_req_packet: build_discover_req_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_discover_req_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_discover_req_packet: Sent EBP_MSG_DISCOVER to %pM via %s (MTU = %u)\n",
             dest_mac, ifname, mtu);
    return 0;
}

int send_discover_ack_packet(uint64_t buffer_id, const unsigned char dest_mac[6], const char *ifname)
{
    uint64_t pkt_len = 0;
    char *packet = build_discover_ack_packet(buffer_id, &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_discover_ack_packet: build_discover_ack_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_discover_ack_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_discover_ack_packet: Sent EBP_MSG_DISCOVER_ACK to %pM via %s (buffer_id=0x%llx)\n",
             dest_mac, ifname, (unsigned long long)buffer_id);
    return 0;
}

int send_invoke_req_packet(uint32_t iid, uint32_t opid,
                           const char *args, uint64_t args_len,
                           const void *payload, uint64_t payload_len,
                           const unsigned char dest_mac[6],
                           const char *ifname)
{
    uint64_t pkt_len = 0;
    char *packet = build_invoke_req_packet(iid, opid,
                                           args, args_len,
                                           payload, payload_len,
                                           &pkt_len);
```

```
    int ret;

    if (!packet) {
        EBA_ERR("send_invoke_req_packet: build_invoke_req_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_invoke_req_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_invoke_req_packet: Sent EBP_MSG_INVOKE (IID=%u, OPID=%u) to %pM via %s\n",
             iid, opid, dest_mac, ifname);
    return 0;
}


int send_invoke_ack_packet(uint8_t status,
                           const unsigned char dest_mac[6],
                           const char *ifname)
{
    uint64_t pkt_len = 0;
    char *packet = build_invoke_ack_packet(status, &pkt_len);
    int ret;

    if (!packet) {
        EBA_ERR("send_invoke_ack_packet: build_invoke_ack_packet() failed.\n");
        return -ENOMEM;
    }

    ret = send_raw_ethernet_packet(packet, pkt_len, dest_mac, EBP_ETHERTYPE, ifname);
    if (ret < 0) {
        EBA_ERR("send_invoke_ack_packet: send_raw_ethernet_packet failed, ret = %d\n", ret);
        kfree(packet);
        return ret;
    }

    kfree(packet);
    EBA_INFO("send_invoke_ack_packet: Sent EBP_MSG_INVOKE_ACK (status=0x%02x) to %pM via %s\n",
             status, dest_mac, ifname);
    return 0;
}
```

## 4.19 src/eba_user.c File Reference

```
#include "eba_user.h"
#include "eba.h"
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include <string.h>
```

Include dependency graph for eba_user.c:



**Functions**

- static int open_eba_device (void)
- uint64_t eba_alloc (uint64_t size, uint64_t life_time, uint8_t type)
- int eba_write (const void ∗data, uint64_t buf_id, uint64_t off, uint64_t size)
- int eba_read (void ∗data_out, uint64_t buf_id, uint64_t off, uint64_t size)
- int eba_remote_alloc (uint64_t size, uint64_t life_time, uint64_t local_buff_id, const char mac[6])
- int eba_remote_write (uint64_t buff_id, uint64_t offset, uint64_t size, const char ∗payload, const char mac[6])
- int eba_remote_read (uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset, uint64_t src_offset, uint64_t size, const char mac[6])

## 4.19.1 Function Documentation

### 4.19.1.1 eba_alloc()

```
uint64_t eba_alloc (
            uint64_t size,
            uint64_t life_time,
            uint8_t type )
```

eba_alloc - Request a buffer from the local node. @size: The size (in bytes) for the buffer allocation. @life_time: The duration of the allocation in seconds; 0 indicates "infinite". @type: The type of memory requested (depends on node specifications).

Return: The allocated buffer ID on success, or 0 on failure.

Definition at line 31 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_alloc_data alloc;

    alloc.size = size;
    alloc.life_time = life_time;
    alloc.buff_id = 0;

    fd = open_eba_device();
    if (fd < 0)
        return 0;

    ret = ioctl(fd, EBA_IOCTL_ALLOC, &alloc);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_ALLOC)");
        return 0;
    }

    return alloc.buff_id;
}
```

### 4.19.1.2 eba_read()

```
int eba_read (
            void * data_out,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_read - Read data from a locally allocated buffer. @data_out: Pointer to the destination where data should be copied. @buff_id: Identifier of the target local buffer. @off: The offset (in bytes) within the buffer to begin reading. @size: The number of bytes to read.

Return: 0 on success, or 1 on failure.

Definition at line 79 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_rw_data rw;

    rw.buff_id = buf_id;
    rw.off = off;
    rw.size = size;
    rw.user_addr = (uint64_t)data_out;

    fd = open_eba_device();
    if (fd < 0)
        return 1;

    ret = ioctl(fd, EBA_IOCTL_READ, &rw);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_READ)");
        return 1;
    }
    return 0;
}
```

#### 4.19.1.3 eba_remote_alloc()

```
int eba_remote_alloc (
            uint64_t size,
            uint64_t life_time,
            uint64_t local_buff_id,
            const char mac[6] )
```

eba_remote_alloc - Request a buffer allocation on a remote node via IOCTL. @size: The size (in bytes) of the requested buffer. @life_time: The life time for the allocation in seconds; 0 indicates "infinite". @local_buff_id: The local buffer identifier where the remote buffer ID will be stored. @mac: The MAC address of the remote node.

Return: 0 on success, or a negative error code on failure.

Definition at line 104 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_remote_alloc remote;

    /* Fill in the remote_alloc structure */
    memset(&remote, 0, sizeof(remote));
    remote.size = size;
    remote.life_time = life_time;
    remote.buffer_id = local_buff_id;
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_ALLOC, &remote);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_REMOTE_ALLOC)");
        return 1;
    }
    return 0;
}
```

#### 4.19.1.4 eba_remote_read()

```
int eba_remote_read (
            uint64_t dst_buffer_id,
            uint64_t src_buffer_id,
            uint64_t dst_offset,
            uint64_t src_offset,
            uint64_t size,
            const char mac[6] )
```

eba_remote_read - Read data from a remote allocated buffer via IOCTL. @dst_buffer_id: Local destination buffer identifier. @src_buffer_id: Remote source buffer identifier. @dst_offset: The offset (in bytes) within the destination buffer where data should be written. @src_offset: The offset (in bytes) within the source buffer where reading begins. @size: The number of bytes to read. @mac: The MAC address of the remote node.

Return: 0 on success, or a negative error code on failure.

Definition at line 169 of file eba_user.c.

```
00170 {
00171     int fd, ret;
00172     struct eba_remote_read remote;
00173
00174     /* Fill in the remote_alloc structure */
00175     remote.dst_buffer_id = dst_buffer_id;
00176     remote.src_buffer_id= src_buffer_id;
00177     remote.dst_offset = dst_offset;
00178     remote.src_offset = src_offset;
00179     remote.size = size;
00180     memcpy((void *)remote.mac, mac, 6);
00181
00182     fd = open("/dev/eba", O_RDWR);
00183     if (fd < 0) {
00184         perror("open(/dev/eba)");
00185         return 1;
00186     }
00187
00188     ret = ioctl(fd, EBA_IOCTL_REMOTE_READ, &remote);
00189     close(fd);
00190
00191     if (ret < 0) {
00192         perror("ioctl(EBA_IOCTL_REMOTE_READ)");
00193         return 1;
00194     }
00195     return 0;
00196 }
```

#### 4.19.1.5 eba_remote_write()

```
int eba_remote_write (
            uint64_t buff_id,
            uint64_t offset,
            uint64_t size,
            const char * payload,
            const char mac[6] )
```

eba_remote_write - Write data to a remote allocated buffer via IOCTL. @buff_id: The identifier of the remote buffer. @offset: The offset (in bytes) within the remote buffer where writing should begin. @size: The number of bytes of the payload to write. @payload: Pointer to the data payload. @mac: The MAC address of the remote node.

Return: 0 on success, or a negative error code on failure.

Definition at line 132 of file eba_user.c.

```
00133 {
00134     int fd, ret;
00135     struct eba_remote_write remote;
00136
00137     /* Fill in the remote_alloc structure */
```

```
    remote.buff_id = buff_id;
    remote.offset = offset;
    remote.size = size;
    remote.payload = malloc(remote.size);
    if(remote.payload == 0)
    {
        perror("malloc failed");
        return 1;
    }
    memcpy((void *)remote.payload, payload, remote.size);
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        free(remote.payload);
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_WRITE, &remote);
    close(fd);

    if (ret < 0) {
        free(remote.payload);
        perror("ioctl(EBA_IOCTL_REMOTE_WRITE)");
        return 1;
    }
    free(remote.payload);
    return 0;
}
```

#### 4.19.1.6 eba_write()

```
int eba_write (
            const void * data,
            uint64_t buff_id,
            uint64_t off,
            uint64_t size )
```

eba_write - Write data to a locally allocated buffer. @data: Pointer to the data to be written. @buff_id: Identifier of the target local buffer. @off: The offset (in bytes) within the buffer to begin writing. @size: The number of bytes to write.

Return: 0 on success, or 1 on failure.

Definition at line 55 of file eba_user.c.

```
{
    int fd, ret;
    struct eba_rw_data rw;

    rw.buff_id = buf_id;
    rw.off = off;
    rw.size = size;
    rw.user_addr = (uint64_t)data;

    fd = open_eba_device();
    if (fd < 0)
        return 1;

    ret = ioctl(fd, EBA_IOCTL_WRITE, &rw);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_WRITE)");
        return 1;
    }
    return 0;
}
```

#### 4.19.1.7 open_eba_device()

```
static int open_eba_device (
            void  )  [static]
```

Definition at line 22 of file eba_user.c.

```
00023  {
00024      int fd = open("/dev/eba", O_RDWR);
00025      if (fd < 0) {
00026          perror("open(/dev/eba)");
00027      }
00028      return fd;
00029  }
```

## 4.20 eba_user.c

Go to the documentation of this file.

```
00001 /*
00002  * eba_user.c - User-space API for EBA Kernel Module IOCTL Interface
00003  *
00004  * This file implements simple wrapper functions (eba_alloc, eba_write, eba_read)
00005  * so that user applications do not have to build IOCTL request structures manually.
00006  */
00007
00008  #include "eba_user.h"
00009  #include "eba.h"          /* Public IOCTL definitions and structures for the kernel module */
00010  #include <fcntl.h>        /* open() */
00011  #include <stdio.h>        /* printf(), perror() */
00012  #include <stdlib.h>       /* exit() */
00013  #include <stdint.h>
00014  #include <sys/ioctl.h>    /* ioctl() */
00015  #include <unistd.h>       /* close() */
00016  #include <string.h>       /* memset() */
00017
00018  /*
00019   * Internal helper: open the device and return its file descriptor.
00020   * Returns a valid file descriptor on success or -1 on error.
00021   */
00022  static int open_eba_device(void)
00023  {
00024      int fd = open("/dev/eba", O_RDWR);
00025      if (fd < 0) {
00026          perror("open(/dev/eba)");
00027      }
00028      return fd;
00029  }
00030
00031  uint64_t eba_alloc(uint64_t size, uint64_t life_time, uint8_t type)
00032  {
00033      int fd, ret;
00034      struct eba_alloc_data alloc;
00035
00036      alloc.size = size;
00037      alloc.life_time = life_time;
00038      alloc.buff_id = 0;
00039
00040      fd = open_eba_device();
00041      if (fd < 0)
00042          return 0;
00043
00044      ret = ioctl(fd, EBA_IOCTL_ALLOC, &alloc);
00045      close(fd);
00046
00047      if (ret < 0) {
00048          perror("ioctl(EBA_IOCTL_ALLOC)");
00049          return 0;
00050      }
00051
00052      return alloc.buff_id;
00053  }
00054
00055  int eba_write(const void *data, uint64_t buf_id, uint64_t off, uint64_t size)
00056  {
00057      int fd, ret;
00058      struct eba_rw_data rw;
00059
00060      rw.buff_id = buf_id;
00061      rw.off = off;
```

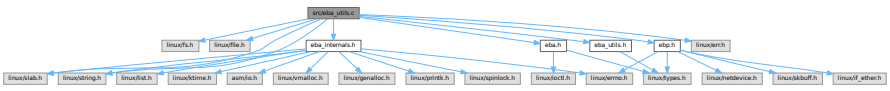
```
    rw.size = size;
    rw.user_addr = (uint64_t)data;

    fd = open_eba_device();
    if (fd < 0)
        return 1;

    ret = ioctl(fd, EBA_IOCTL_WRITE, &rw);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_WRITE)");
        return 1;
    }
    return 0;
}

int eba_read(void *data_out, uint64_t buf_id, uint64_t off, uint64_t size)
{
    int fd, ret;
    struct eba_rw_data rw;

    rw.buff_id = buf_id;
    rw.off = off;
    rw.size = size;
    rw.user_addr = (uint64_t)data_out;

    fd = open_eba_device();
    if (fd < 0)
        return 1;

    ret = ioctl(fd, EBA_IOCTL_READ, &rw);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_READ)");
        return 1;
    }
    return 0;
}


int eba_remote_alloc(uint64_t size, uint64_t life_time, uint64_t local_buff_id,const char mac[6]/* TODO modify it to be come node*/)
{
    int fd, ret;
    struct eba_remote_alloc remote;

    /* Fill in the remote_alloc structure */
    memset(&remote, 0, sizeof(remote));
    remote.size = size;
    remote.life_time = life_time;
    remote.buffer_id = local_buff_id;
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_ALLOC, &remote);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_REMOTE_ALLOC)");
        return 1;
    }
    return 0;
}

int eba_remote_write(uint64_t buff_id, uint64_t offset, uint64_t size,const char* payload ,const char mac[6]/* TODO modify it to be come node*/)
{
    int fd, ret;
    struct eba_remote_write remote;

    /* Fill in the remote_alloc structure */
    remote.buff_id = buff_id;
    remote.offset = offset;
    remote.size = size;
    remote.payload = malloc(remote.size);
    if(remote.payload == 0)
    {
        perror("malloc failed");
        return 1;
    }
```

```
    memcpy((void *)remote.payload, payload, remote.size);
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        free(remote.payload);
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_WRITE, &remote);
    close(fd);

    if (ret < 0) {
        free(remote.payload);
        perror("ioctl(EBA_IOCTL_REMOTE_WRITE)");
        return 1;
    }
    free(remote.payload);
    return 0;
}

int eba_remote_read(uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset,uint64_t
    src_offset ,uint64_t size,const char mac[6]/* TODO modify it to be come node*/)
{
    int fd, ret;
    struct eba_remote_read remote;

    /* Fill in the remote_alloc structure */
    remote.dst_buffer_id = dst_buffer_id;
    remote.src_buffer_id= src_buffer_id;
    remote.dst_offset = dst_offset;
    remote.src_offset = src_offset;
    remote.size = size;
    memcpy((void *)remote.mac, mac, 6);

    fd = open("/dev/eba", O_RDWR);
    if (fd < 0) {
        perror("open(/dev/eba)");
        return 1;
    }

    ret = ioctl(fd, EBA_IOCTL_REMOTE_READ, &remote);
    close(fd);

    if (ret < 0) {
        perror("ioctl(EBA_IOCTL_REMOTE_READ)");
        return 1;
    }
    return 0;
}
```

## 4.21 src/eba_utils.c File Reference

```
#include <linux/fs.h>
#include <linux/file.h>
#include <linux/slab.h>
#include <linux/string.h>
#include <linux/err.h>
#include "eba_utils.h"
#include "eba_internals.h"
#include "eba.h"
#include "ebp.h"
```

Include dependency graph for eba_utils.c:



### Functions

- int eba_utils_file_to_buf (const char *filepath, uint64_t buff_id)
- int eba_utils_buf_to_file (uint64_t buff_id, uint64_t size, const char *filepath)
- int eba_export_node_specs (void)
- int test_eba_utils_file_to_buf (void)
- int test_eba_utils_buf_to_file (void)
- int test_eba_export_node_specs (void)

### Variables

- struct list_head eba_buffer_list
- spinlock_t eba_buffer_list_lock
- struct node_info node_infos [MAX_NODE_COUNT]
- int nodes_count

## 4.21.1 Function Documentation

### 4.21.1.1 eba_export_node_specs()

```
int eba_export_node_specs (
            void  )
```

eba_export_node_specs - Exports each node's node_specs buffer into a file.

Definition at line 154 of file eba_utils.c.

```
{
    int i;
    char filepath[256];

    EBA_INFO("Exporting node_specs buffers to files...\n");

    for (i = 0; i < MAX_NODE_COUNT; i++) {
        if (node_infos[i].id != -1 && node_infos[i].node_specs != 0) {
            struct eba_buffer *buf;
            int found = 0;
            uint64_t buff_id = node_infos[i].node_specs;

            /* Acquire global lock, search the buffer list for buff_id */
            spin_lock(&eba_buffer_list_lock);
            list_for_each_entry(buf, &eba_buffer_list, node) {
                if (buf->address == buff_id) {
                    found = 1;
                    break;
                }
            }
            spin_unlock(&eba_buffer_list_lock);

            if (!found) {
                EBA_ERR("Node %d has node_specs=0x%llx, but no matching buffer\n",
                        i, buff_id);
                continue;
            }

            /* Build the path like "/var/lib/eba/node_2_specs" */
            snprintf(filepath, sizeof(filepath),
                     "/var/lib/eba/node_%d_specs", i);

            /* Dump the entire buffer to that file */
            eba_utils_buf_to_file(buff_id, buf->size, filepath);
        }
    }

    return 0;
}
```

#### 4.21.1.2 eba_utils_buf_to_file()

```
int eba_utils_buf_to_file (
            uint64_t buff_id,
            uint64_t size,
            const char * filepath )
```

eba_utils_buf_to_file - Writes data from an EBA buffer to a file. @buff_id: EBA buffer pointer @size: How many bytes to read from the buffer @filepath: Where to write on disk

Returns 0 on success, negative error code on failure.

Definition at line 92 of file eba_utils.c.

```
00093 {
00094     struct file *filp;
00095     loff_t pos = 0;
00096     char *kbuf = NULL;
00097     ssize_t bytes_written;
00098     int ret;
00099
00100     if (size == 0) {
00101         EBA_ERR("eba_utils_buf_to_file: zero size, nothing to write\n");
00102         return -EINVAL;
00103     }
00104
00105     /* Allocate a kernel buffer of 'size' to read from EBA */
00106     kbuf = kmalloc(size, GFP_KERNEL);
00107     if (!kbuf)
00108         return -ENOMEM;
00109
00110     /* Read from EBA buffer into kbuf */
00111     ret = eba_internals_read(kbuf, buff_id, 0, size);
00112     if (ret < 0) {
00113         EBA_ERR("eba_internals_read() failed on buff_id 0x%llx\n", buff_id);
00114         goto out_free;
00115     }
00116
00117     /* Open or create the file (truncate if exists) */
00118     filp = filp_open(filepath, O_WRONLY | O_CREAT | O_TRUNC, 0600);
00119     if (IS_ERR(filp)) {
00120         EBA_ERR("Failed to open file %s for writing, err=%ld\n",
00121                 filepath, PTR_ERR(filp));
00122         ret = PTR_ERR(filp);
00123         filp = NULL;
00124         goto out_free;
00125     }
00126
00127     /* Write kbuf to file */
00128     bytes_written = kernel_write(filp, kbuf, size, &pos);
00129     if (bytes_written < 0 || bytes_written != size) {
00130         EBA_ERR("Failed to write entire buffer to file %s, bytes_written=%zd\n",
00131                 filepath, bytes_written);
00132         if (bytes_written >= 0)
00133             ret = -EIO;
00134         else
00135             ret = bytes_written;
00136         goto out_close;
00137     }
00138
00139     EBA_INFO("Wrote %llu bytes from EBA buffer 0x%llx to file %s\n",
00140              size, buff_id, filepath);
00141     ret = 0; /* success */
00142
00143 out_close:
00144     if (filp)
00145         filp_close(filp, NULL);
00146 out_free:
00147     kfree(kbuf);
00148     return ret;
00149 }
```

#### 4.21.1.3 eba_utils_file_to_buf()

```
int eba_utils_file_to_buf (
            const char * filepath,
            uint64_t buff_id )
```

eba_utils_file_to_buf - Reads an entire file into an existing EBA buffer. @filepath: Path to the file on disk @buff_id: The EBA buffer ID (already allocated).

Returns 0 on success, 1 on error.

Definition at line 23 of file eba_utils.c.

```
{
    struct file *filp;
    loff_t pos = 0;
    loff_t file_size;
    char *kbuf = NULL;
    ssize_t bytes_read;
    int ret = 1;  /* default to error */

    /* Open the file read-only */
    filp = filp_open(filepath, O_RDONLY, 0);
    if (IS_ERR(filp)) {
        EBA_ERR("Failed to open file %s, err=%ld\n", filepath, PTR_ERR(filp));
        return 1;
    }

    /* Get file size via i_size_read() */
    file_size = i_size_read(file_inode(filp));
    if (file_size <= 0) {
        EBA_ERR("File %s is empty or error reading size\n", filepath);
        goto out_close;
    }

    /* Safety check if extremely large */
    if (file_size > ULONG_MAX) {
        EBA_ERR("File %s is too large (%lld bytes)\n", filepath, file_size);
        goto out_close;
    }

    /* Allocate a kernel buffer to read the file contents into */
    kbuf = kmalloc(file_size, GFP_KERNEL);
    if (!kbuf) {
        EBA_ERR("Failed to allocate temp buffer\n");
        goto out_close;
    }

    /* Read the entire file into kbuf */
    bytes_read = kernel_read(filp, kbuf, file_size, &pos);
    if (bytes_read < 0 || bytes_read != file_size) {
        EBA_ERR("Failed to read entire file %s, bytes_read=%zd\n",
                filepath, bytes_read);
        goto out_free;
    }

    /* Write that data into the EBA buffer buff_id at offset=0 */
    if (eba_internals_write(kbuf, buff_id, 0, file_size) < 0) {
        EBA_ERR("eba_internals_write() failed for file %s\n", filepath);
        goto out_free;
    }

    EBA_INFO("Read %lld bytes from file %s into EBA buffer 0x%llx\n",
             file_size, filepath, buff_id);
    ret = 0; /* success */

out_free:
    kfree(kbuf);
out_close:
    filp_close(filp, NULL);
    return ret;
}
```

#### 4.21.1.4 test_eba_export_node_specs()

```
int test_eba_export_node_specs (
            void  )
```

test_eba_export_node_specs - Self-test for eba_export_node_specs function.

Return: 0 on success, or a non-zero error code on failure.

Definition at line 281 of file eba_utils.c.

```
{
```

```
    int node_index = 0;  /* We'll pick node 0 for the test */
    void *buff_id;
    char test_data[] = "Hello from node_0_specs!\n";
    int ret;

    EBA_INFO("=== test_eba_export_node_specs: START ===\n");

    /* 1. Mark node_infos[node_index] as valid */
    node_infos[node_index].id = node_index; /* or some other ID != -1 */

    /* 2. Allocate a buffer to hold the specs data */
    buff_id = eba_internals_malloc(1024, 0); /* 1KB for demonstration */
    if (!buff_id) {
        EBA_ERR("Failed to allocate node_specs buffer for node %d\n", node_index);
        return 1;
    }

    /* 3. Write some data into that buffer */
    ret = eba_internals_write(test_data, (uint64_t)buff_id, 0, strlen(test_data));
    if (ret < 0) {
        EBA_ERR("Failed to write test data into node_specs buffer\n");
        eba_internals_free(buff_id);
        return 1;
    }

    /* 4. Store this buffer in node_infos[node_index].node_specs */
    node_infos[node_index].node_specs = (uint64_t)buff_id;

    /*
     * 5. Call eba_export_node_specs()
     *    This should write the entire buffer content to /var/lib/eba/node_0_specs.
     *    Make sure /var/lib/eba/ directory exists with writable permissions for root.
     */
    ret = eba_export_node_specs();
    if (ret != 0) {
        EBA_ERR("eba_export_node_specs returned error %d\n", ret);
        /* We won't free the buffer here so you can inspect if needed, but normally you might. */
        return 1;
    }

    EBA_INFO("Check /var/lib/eba/node_0_specs to see if it contains: \"%s\"\n", test_data);

    /*
     * 6. (Optional) If you want to clean up this buffer immediately:
     */
    // eba_internals_free(buff_id);
    // node_infos[node_index].node_specs = 0;
    // node_infos[node_index].id = -1;

    EBA_INFO("=== test_eba_export_node_specs: END ===\n");
    return 0; /* success */
}
```

#### 4.21.1.5 test_eba_utils_buf_to_file()

```
int test_eba_utils_buf_to_file (
            void  )
```

test_eba_utils_buf_to_file - Self-test for eba_utils_buf_to_file function.

Return: 0 on success, or a non-zero error code on failure.

Definition at line 236 of file eba_utils.c.

```
{
    void *buff_id;
    char pattern[512];
    int i, ret;

    EBA_INFO("=== test_eba_utils_buf_to_file: START ===\n");

    /* 1. Allocate a 512-byte buffer */
    buff_id = eba_internals_malloc(512, 0);
    if (!buff_id) {
        EBA_ERR("Failed to allocate EBA buffer in test_eba_utils_buf_to_file\n");
        return 1;
    }

    /* 2. Fill local array 'pattern' with some data to demonstrate writing */
    for (i = 0; i < (int)sizeof(pattern); i++) {
```

```
        pattern[i] = (char)('A' + (i % 26)); /* 'A'..'Z' repeated */
    }

    /* 3. Write this pattern to the EBA buffer */
    ret = eba_internals_write(pattern, (uint64_t)buff_id, 0, sizeof(pattern));
    if (ret < 0) {
        EBA_ERR("Failed to write pattern to EBA buffer\n");
        eba_internals_free(buff_id);
        return 1;
    }

    /* 4. Now write the EBA buffer to a file: /tmp/test_output */
    ret = eba_utils_buf_to_file((uint64_t)buff_id, sizeof(pattern), "/tmp/test_output");
    if (ret < 0) {
        EBA_ERR("eba_utils_buf_to_file failed, ret=%d\n", ret);
        eba_internals_free(buff_id);
        return 1;
    }

    EBA_INFO("Check /tmp/test_output to see if the 512-byte pattern is there.\n");

    /* 5. Free the buffer */
    eba_internals_free(buff_id);

    EBA_INFO("=== test_eba_utils_buf_to_file: END ===\n");
    return 0; /* success */
}
```

#### 4.21.1.6 test_eba_utils_file_to_buf()

```
int test_eba_utils_file_to_buf (
            void  )
```

test_eba_utils_file_to_buf - Self-test for eba_utils_file_to_buf function.

Return: 0 on success, or a non-zero error code on failure.

Definition at line 195 of file eba_utils.c.

```
{
    void *buff_id;
    int ret;
    char tmpbuf[64]; /* small stack buffer to read/print a piece of data */

    EBA_INFO("=== test_eba_utils_file_to_buf: START ===\n");

    /* 1. Allocate a buffer of 512 bytes with no expiration (lifetime=0). */
    buff_id = eba_internals_malloc(512, 0);
    if (!buff_id) {
        EBA_ERR("Failed to allocate EBA buffer in test_eba_utils_file_to_buf\n");
        return 1;
    }

    /* 2. Read from /tmp/test_input into this buffer.
     *    Make sure /tmp/test_input exists and has <=512 bytes, or adjust code.
     */
    ret = eba_utils_file_to_buf("/tmp/test_input", (uint64_t)buff_id);
    if (ret != 0) {
        EBA_ERR("eba_utils_file_to_buf failed (ret=%d)\n", ret);
        eba_internals_free(buff_id);
        return 1;
    }

    /* 3. Optionally read back a small portion of data from the buffer and print it */
    memset(tmpbuf, 0, sizeof(tmpbuf));
    if (eba_internals_read(tmpbuf, (uint64_t)buff_id, 0, sizeof(tmpbuf) - 1) == 0) {
        EBA_INFO("First 63 bytes from EBA buffer: \"%s\"\n", tmpbuf);
    } else {
        EBA_ERR("Failed to read back from EBA buffer\n");
    }

    /* 4. Free the EBA buffer */
    eba_internals_free(buff_id);

    EBA_INFO("=== test_eba_utils_file_to_buf: END ===\n");
    return 0; /* success */
}
```

### 4.21.2 Variable Documentation

#### 4.21.2.1 eba_buffer_list

```
struct list_head eba_buffer_list  [extern]
```

#### 4.21.2.2 eba_buffer_list_lock

```
spinlock_t eba_buffer_list_lock  [extern]
```

Definition at line 6 of file eba_internals.c.

#### 4.21.2.3 node_infos

```
struct node_info node_infos[MAX_NODE_COUNT]  [extern]
```

Definition at line 184 of file ebp.c.

#### 4.21.2.4 nodes_count

```
int nodes_count  [extern]
```

Definition at line 185 of file ebp.c.

## 4.22 eba_utils.c

Go to the documentation of this file.

```
00001 #include <linux/fs.h>
00002 #include <linux/file.h>
00003 #include <linux/slab.h>
00004 #include <linux/string.h>
00005 #include <linux/err.h>
00006
00007 #include "eba_utils.h"       /* Our prototypes */
00008 #include "eba_internals.h"   /* eba_internals_read/write + eba_buffer_list, eba_buffer_list_lock */
00009 #include "eba.h"             /* node_infos[], MAX_NODE_COUNT, EBA_ERR/EBA_INFO macros, etc. */
00010 #include "ebp.h"
00011 extern struct list_head eba_buffer_list;
00012 extern spinlock_t eba_buffer_list_lock;
00013 extern struct node_info node_infos[MAX_NODE_COUNT];
00014 extern int nodes_count;
00015
00023 int eba_utils_file_to_buf(const char *filepath, uint64_t buff_id)
00024 {
00025     struct file *filp;
00026     loff_t pos = 0;
00027     loff_t file_size;
00028     char *kbuf = NULL;
00029     ssize_t bytes_read;
00030     int ret = 1;  /* default to error */
00031
00032     /* Open the file read-only */
00033     filp = filp_open(filepath, O_RDONLY, 0);
00034     if (IS_ERR(filp)) {
00035         EBA_ERR("Failed to open file %s, err=%ld\n", filepath, PTR_ERR(filp));
00036         return 1;
00037     }
00038
00039     /* Get file size via i_size_read() */
00040     file_size = i_size_read(file_inode(filp));
00041     if (file_size <= 0) {
```

```
        EBA_ERR("File %s is empty or error reading size\n", filepath);
        goto out_close;
    }

    /* Safety check if extremely large */
    if (file_size > ULONG_MAX) {
        EBA_ERR("File %s is too large (%lld bytes)\n", filepath, file_size);
        goto out_close;
    }

    /* Allocate a kernel buffer to read the file contents into */
    kbuf = kmalloc(file_size, GFP_KERNEL);
    if (!kbuf) {
        EBA_ERR("Failed to allocate temp buffer\n");
        goto out_close;
    }

    /* Read the entire file into kbuf */
    bytes_read = kernel_read(filp, kbuf, file_size, &pos);
    if (bytes_read < 0 || bytes_read != file_size) {
        EBA_ERR("Failed to read entire file %s, bytes_read=%zd\n",
                filepath, bytes_read);
        goto out_free;
    }

    /* Write that data into the EBA buffer buff_id at offset=0 */
    if (eba_internals_write(kbuf, buff_id, 0, file_size) < 0) {
        EBA_ERR("eba_internals_write() failed for file %s\n", filepath);
        goto out_free;
    }

    EBA_INFO("Read %lld bytes from file %s into EBA buffer 0x%llx\n",
             file_size, filepath, buff_id);
    ret = 0; /* success */

out_free:
    kfree(kbuf);
out_close:
    filp_close(filp, NULL);
    return ret;
}

int eba_utils_buf_to_file(uint64_t buff_id, uint64_t size, const char *filepath)
{
    struct file *filp;
    loff_t pos = 0;
    char *kbuf = NULL;
    ssize_t bytes_written;
    int ret;

    if (size == 0) {
        EBA_ERR("eba_utils_buf_to_file: zero size, nothing to write\n");
        return -EINVAL;
    }

    /* Allocate a kernel buffer of 'size' to read from EBA */
    kbuf = kmalloc(size, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    /* Read from EBA buffer into kbuf */
    ret = eba_internals_read(kbuf, buff_id, 0, size);
    if (ret < 0) {
        EBA_ERR("eba_internals_read() failed on buff_id 0x%llx\n", buff_id);
        goto out_free;
    }

    /* Open or create the file (truncate if exists) */
    filp = filp_open(filepath, O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (IS_ERR(filp)) {
        EBA_ERR("Failed to open file %s for writing, err=%ld\n",
                filepath, PTR_ERR(filp));
        ret = PTR_ERR(filp);
        filp = NULL;
        goto out_free;
    }

    /* Write kbuf to file */
    bytes_written = kernel_write(filp, kbuf, size, &pos);
    if (bytes_written < 0 || bytes_written != size) {
        EBA_ERR("Failed to write entire buffer to file %s, bytes_written=%zd\n",
                filepath, bytes_written);
        if (bytes_written >= 0)
            ret = -EIO;
        else
            ret = bytes_written;
        goto out_close;
```

```
    }

    EBA_INFO("Wrote %llu bytes from EBA buffer 0x%llx to file %s\n",
             size, buff_id, filepath);
    ret = 0; /* success */

out_close:
    if (filp)
        filp_close(filp, NULL);
out_free:
    kfree(kbuf);
    return ret;
}

int eba_export_node_specs(void)
{
    int i;
    char filepath[256];

    EBA_INFO("Exporting node_specs buffers to files...\n");

    for (i = 0; i < MAX_NODE_COUNT; i++) {
        if (node_infos[i].id != -1 && node_infos[i].node_specs != 0) {
            struct eba_buffer *buf;
            int found = 0;
            uint64_t buff_id = node_infos[i].node_specs;

            /* Acquire global lock, search the buffer list for buff_id */
            spin_lock(&eba_buffer_list_lock);
            list_for_each_entry(buf, &eba_buffer_list, node) {
                if (buf->address == buff_id) {
                    found = 1;
                    break;
                }
            }
            spin_unlock(&eba_buffer_list_lock);

            if (!found) {
                EBA_ERR("Node %d has node_specs=0x%llx, but no matching buffer\n",
                        i, buff_id);
                continue;
            }

            /* Build the path like "/var/lib/eba/node_2_specs" */
            snprintf(filepath, sizeof(filepath),
                     "/var/lib/eba/node_%d_specs", i);

            /* Dump the entire buffer to that file */
            eba_utils_buf_to_file(buff_id, buf->size, filepath);
        }
    }

    return 0;
}

int test_eba_utils_file_to_buf(void)
{
    void *buff_id;
    int ret;
    char tmpbuf[64]; /* small stack buffer to read/print a piece of data */

    EBA_INFO("=== test_eba_utils_file_to_buf: START ===\n");

    /* 1. Allocate a buffer of 512 bytes with no expiration (lifetime=0). */
    buff_id = eba_internals_malloc(512, 0);
    if (!buff_id) {
        EBA_ERR("Failed to allocate EBA buffer in test_eba_utils_file_to_buf\n");
        return 1;
    }

    /* 2. Read from /tmp/test_input into this buffer.
     *    Make sure /tmp/test_input exists and has <=512 bytes, or adjust code.
     */
    ret = eba_utils_file_to_buf("/tmp/test_input", (uint64_t)buff_id);
    if (ret != 0) {
        EBA_ERR("eba_utils_file_to_buf failed (ret=%d)\n", ret);
        eba_internals_free(buff_id);
        return 1;
    }

    /* 3. Optionally read back a small portion of data from the buffer and print it */
    memset(tmpbuf, 0, sizeof(tmpbuf));
    if (eba_internals_read(tmpbuf, (uint64_t)buff_id, 0, sizeof(tmpbuf) - 1) == 0) {
        EBA_INFO("First 63 bytes from EBA buffer: \"%s\"\n", tmpbuf);
    } else {
        EBA_ERR("Failed to read back from EBA buffer\n");
    }
```

```c

    /* 4. Free the EBA buffer */
    eba_internals_free(buff_id);

    EBA_INFO("=== test_eba_utils_file_to_buf: END ===\n");
    return 0; /* success */
}


int test_eba_utils_buf_to_file(void)
{
    void *buff_id;
    char pattern[512];
    int i, ret;

    EBA_INFO("=== test_eba_utils_buf_to_file: START ===\n");

    /* 1. Allocate a 512-byte buffer */
    buff_id = eba_internals_malloc(512, 0);
    if (!buff_id) {
        EBA_ERR("Failed to allocate EBA buffer in test_eba_utils_buf_to_file\n");
        return 1;
    }

    /* 2. Fill local array 'pattern' with some data to demonstrate writing */
    for (i = 0; i < (int)sizeof(pattern); i++) {
        pattern[i] = (char)('A' + (i % 26)); /* 'A'..'Z' repeated */
    }

    /* 3. Write this pattern to the EBA buffer */
    ret = eba_internals_write(pattern, (uint64_t)buff_id, 0, sizeof(pattern));
    if (ret < 0) {
        EBA_ERR("Failed to write pattern to EBA buffer\n");
        eba_internals_free(buff_id);
        return 1;
    }

    /* 4. Now write the EBA buffer to a file: /tmp/test_output */
    ret = eba_utils_buf_to_file((uint64_t)buff_id, sizeof(pattern), "/tmp/test_output");
    if (ret < 0) {
        EBA_ERR("eba_utils_buf_to_file failed, ret=%d\n", ret);
        eba_internals_free(buff_id);
        return 1;
    }

    EBA_INFO("Check /tmp/test_output to see if the 512-byte pattern is there.\n");

    /* 5. Free the buffer */
    eba_internals_free(buff_id);

    EBA_INFO("=== test_eba_utils_buf_to_file: END ===\n");
    return 0; /* success */
}

int test_eba_export_node_specs(void)
{
    int node_index = 0;  /* We'll pick node 0 for the test */
    void *buff_id;
    char test_data[] = "Hello from node_0_specs!\n";
    int ret;

    EBA_INFO("=== test_eba_export_node_specs: START ===\n");

    /* 1. Mark node_infos[node_index] as valid */
    node_infos[node_index].id = node_index; /* or some other ID != -1 */

    /* 2. Allocate a buffer to hold the specs data */
    buff_id = eba_internals_malloc(1024, 0); /* 1KB for demonstration */
    if (!buff_id) {
        EBA_ERR("Failed to allocate node_specs buffer for node %d\n", node_index);
        return 1;
    }

    /* 3. Write some data into that buffer */
    ret = eba_internals_write(test_data, (uint64_t)buff_id, 0, strlen(test_data));
    if (ret < 0) {
        EBA_ERR("Failed to write test data into node_specs buffer\n");
        eba_internals_free(buff_id);
        return 1;
    }

    /* 4. Store this buffer in node_infos[node_index].node_specs */
    node_infos[node_index].node_specs = (uint64_t)buff_id;

    /*
     * 5. Call eba_export_node_specs()
     *    This should write the entire buffer content to /var/lib/eba/node_0_specs.
```

```
     *    Make sure /var/lib/eba/ directory exists with writable permissions for root.
     */
    ret = eba_export_node_specs();
    if (ret != 0) {
        EBA_ERR("eba_export_node_specs returned error %d\n", ret);
        /* We won't free the buffer here so you can inspect if needed, but normally you might. */
        return 1;
    }

    EBA_INFO("Check /var/lib/eba/node_0_specs to see if it contains: \"%s\"\n", test_data);

    /*
     * 6. (Optional) If you want to clean up this buffer immediately:
     */
    // eba_internals_free(buff_id);
    // node_infos[node_index].node_specs = 0;
    // node_infos[node_index].id = -1;

    EBA_INFO("=== test_eba_export_node_specs: END ===\n");
    return 0; /* success */
}

```

## 4.23 src/ebp.c File Reference

```
#include "ebp.h"
#include "eba_internals.h"
#include "eba_net.h"
#include <linux/delay.h>
#include "eba.h"
#include "eba_utils.h"
```

Include dependency graph for ebp.c:

### Macros

- #define INVALID_NODE_ID 0xffff

### Functions

- int ebp_handle_packets (struct sk_buff ∗skb, struct net_device ∗dev, struct packet_type ∗pt, struct net_device ∗orig_dev)
- void ebp_init (void)
- void ebp_exit (void)
- void print_node_infos (void)
- void print_op_entries (void)
- int node_info_array_init (void)
- int invoke_tracker_array_init (void)
- int op_entry_array_init (void)
- int ebp_register_op (uint32_t op_id, ebp_op_t fn)
- int ebp_invoke_op (uint32_t op_id, const void ∗args, uint64_t arg_len, const char mac[6])
- int ebp_ops_init (void)
- int ebp_op_write (const void ∗args, uint64_t arg_len, const char mac[6])

- int ebp_op_alloc (const void *args, uint64_t arg_len, const char mac[6])
- int ebp_op_read (const void *args, uint64_t arg_len, const char mac[6])
- int ebp_remote_alloc (uint64_t size, uint64_t life_time, uint64_t local_buff_id, const char mac[6])
- int ebp_remote_write (uint64_t buff_id, uint64_t offset, uint64_t size, const char *payload, const char mac[6])
- int ebp_remote_read (uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset, uint64_t src_offset, uint64_t size, const char mac[6])
- int ebp_register_node (uint16_t mtu, const char mac[6], uint64_t node_specs)

**Variables**

- void * local_specs = NULL
- struct packet_type ebp_packet_type
- struct node_info node_infos [MAX_NODE_COUNT]
- int nodes_count = 1
- struct invoke_tracker invoke_trackers [MAX_INVOKE_COUNT]
- struct op_entry op_entries [MAX_OP_COUNT]

## 4.23.1 Macro Definition Documentation

### 4.23.1.1 INVALID_NODE_ID

```
#define INVALID_NODE_ID 0xffff
```

Definition at line 182 of file ebp.c.

## 4.23.2 Function Documentation

### 4.23.2.1 ebp_exit()

```
void ebp_exit (
            void  )
```

ebp_exit - Unregister the EBA packet handler from the Linux networking stack.

Definition at line 178 of file ebp.c.

```
00179 {
00180     dev_remove_pack(&ebp_packet_type);
00181 }
```

#### 4.23.2.2 ebp_handle_packets()

```
int ebp_handle_packets (
            struct sk_buff * skb,
            struct net_device * dev,
            struct packet_type * pt,
            struct net_device * orig_dev )
```

ebp_handle_packets - Callback for processing received Ethernet frames. @skb: Pointer to the socket buffer containing the received packet. @dev: Network device that received the packet. @pt: Pointer to the packet_type structure associated with the packet. @orig_dev: Original network device.

This function is registered with the networking stack to handle Ethernet frames with the EBP_ETHERTYPE protocol. It processes the received packet and returns an appropriate status code.

**Returns**

NET_RX_SUCCESS on successful processing, or an error code if processing fails.

Definition at line 17 of file ebp.c.

```
00019 {
00020     struct ebp_header *eba_hdr;
00021     unsigned char *payload;
00022     int payload_len;
00023     struct ethhdr *eth;
00024
00025     if (!skb)
00026     {
00027         EBA_ERR("ebp_handle_packets: Packet is NULL\n");
00028         return NET_RX_DROP;
00029     }
00030
00031     /* Verify packet is long enough for an Ethernet header + EBA header */
00032     if (skb->len < ETH_HLEN + sizeof(struct ebp_header))
00033     {
00034         EBA_ERR("ebp_handle_packets: Packet too short on device %s, length: %u\n",dev->name,
      skb->len);
00035         kfree_skb(skb);
00036         return NET_RX_DROP;
00037     }
00038
00039     /* Get the Ethernet header */
00040     eth = eth_hdr(skb);
00041     EBA_INFO("ebp_handle_packets: Source MAC address: %pM\n", eth->h_source);
00042
00043     /* EBA header */
00044     payload = skb->data;
00045     payload_len = skb->len;
00046     eba_hdr = (struct ebp_header *)payload;
00047
00048     EBA_INFO("ebp_handle_packets: Received packet on %s, protocol: 0x%04x, length: %u, EBA msgType:
      0x%02x\n",dev->name, ntohs(skb->protocol), skb->len, eba_hdr->msgType);
00049
00050     switch (eba_hdr->msgType)
00051     {
00052     case EBP_MSG_DISCOVER: /* 0x01 */
00053         if (skb->len < ETH_HLEN + sizeof(struct ebp_discover_req))
00054         {
00055             EBA_ERR("ebp_handle_packets: Discover Request packet too short\n");
00056             kfree_skb(skb);
00057             return NET_RX_DROP;
00058         }
00059         else
00060         {
00061             struct ebp_discover_req *disc = (struct ebp_discover_req *)eba_hdr;
00062             EBA_INFO("ebp_handle_packets: Discover Request: MTU = %u\n", ntohs(disc->mtu));
00063             //allocate node specs buffer
00064             void* node_specs =
      eba_internals_malloc(EBP_NODE_SPECS_MAX_SIZE,EBP_NODE_SPECS_MAX_LIFE_TIME);
00065             //register the new node
00066             //handle errors lol
00067             int ret = ebp_register_node(ntohs(disc->mtu),eth->h_source,(uint64_t)node_specs);
00068             if(ret < 0)
00069             {
00070                 EBA_ERR("ebp_handle_packets: register node failed\n");
00071                 kfree_skb(skb);
```

```
                return NET_RX_DROP;
            }
            //send the discover ack with the allocated buffer to enable remote node to share its data.
            /*int ret = send_discover_ack_packet((uint64_t)node_specs,eth->h_source,"enp0s8");
            if(ret < 0)
            {
                EBA_ERR("ebp_handle_packets: send discover ack packet failed\n");
                kfree_skb(skb);
                return NET_RX_DROP;
            }*/
        }
        break;
    case EBP_MSG_DISCOVER_ACK: /* 0x03 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_discover_ack))
        {
            EBA_ERR("ebp_handle_packets: Discover Ack packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_discover_ack *ack = (struct ebp_discover_ack *)eba_hdr;
            uint64_t buff_id = be64_to_cpu(ack->buffer_id);
            EBA_INFO("ebp_handle_packets: Discover Ack: buffer_id = 0x%llu \n", buff_id);
            //todo handle mtu
            /*int rc = eba_utils_file_to_buf("/var/lib/eba/node_local.eba",(uint64_t)local_specs);
            if (rc < 0) {
                EBA_ERR("ebp_handle_packets: Failed to read node_local.eba, ret=%d\n", rc);
                kfree_skb(skb);
                return NET_RX_DROP;
            } else {
                EBA_INFO("ebp_handle_packets: Read %llu bytes from node_local.eba, sending to remote\n", (uint64_t)4096);
                uint64_t offset = 0;
                for(int i = 0 ; i < 4 ; i++)
                {
                    rc = ebp_remote_write(buff_id, offset, (uint64_t)1000, (char*) (local_specs) + offset, eth->h_source);
                    if (rc < 0) {
                        EBA_ERR("ebp_handle_packets: Failed to ebp_remote_write() node_local.eba content, ret=%d\n", rc);
                    }
                    offset+= 1000;
                }
            }
        */
        }
        break;
    case EBP_MSG_INVOKE: /* 0x02 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_invoke_req))
        {
            EBA_ERR("ebp_handle_packets: Invoke Request packet too short\n");
        }
        else
        {
            struct ebp_invoke_req *inv = (struct ebp_invoke_req *)eba_hdr;
            uint32_t iid = ntohl(inv->iid);
            uint32_t opid = ntohl(inv->opid);
            uint64_t args_len = be64_to_cpu(inv->args_len);

            EBA_INFO("ebp_handle_packets: Invoke Request: IID = %u, OpID = %u, args_len = %llu\n",iid, opid, args_len);
            int ret = ebp_invoke_op(opid, inv->args, args_len, eth->h_source);
            if (ret < 0) {
                EBA_ERR("ebp_handle_packets: ebp_invoke_op(opid=%u) failed ret=%d\n", opid, ret);
                kfree_skb(skb);
                return NET_RX_DROP;
                /* Possibly build & send an error ack or something... TODO */
            } else {
                /* Possibly build an ack for success... TODO */
            }

        }
        break;
    case EBP_MSG_INVOKE_ACK: /* 0x04 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_invoke_ack))
        {
            EBA_ERR("ebp_handle_packets: Invoke Ack packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_invoke_ack *inv_ack = (struct ebp_invoke_ack *)eba_hdr;
            EBA_INFO("ebp_handle_packets: Invoke Ack: status = 0x%02x TODO \n ", inv_ack->status);
        }
        break;
```

```
    default:
        EBA_ERR("ebp_handle_packets: Unknown EBA msgType: 0x%02x\n", eba_hdr->msgType);
        kfree_skb(skb);
        return NET_RX_DROP;
        break;
    }

    kfree_skb(skb);
    return NET_RX_SUCCESS;
}
```

#### 4.23.2.3 ebp_init()

```
void ebp_init (
            void  )
```

ebp_init - Register the EBA packet handler with the Linux networking stack and all other struct arrays related to ebp.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 166 of file ebp.c.

```
{
    dev_add_pack(&ebp_packet_type);
    node_info_array_init();
    invoke_tracker_array_init();
    op_entry_array_init();
    ebp_ops_init();
    local_specs = eba_internals_malloc(4096,0);
    char mac[6] = {0xff,0xff,0xff,0xff,0xff,0xff};
    send_discover_req_packet(69,mac,"enp0s8");
    print_node_infos();
}
```

#### 4.23.2.4 ebp_invoke_op()

```
int ebp_invoke_op (
            uint32_t op_id,
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_invoke_op - Invoke a registered EBA operation. @op_id: Operation identifier to look up. @args: Pointer to the raw argument data. @arg_len: Length of the argument data in bytes. @mac: Destination MAC address of the invoking node.

This function searches for the operation associated with the provided op_id and calls the corresponding function.

**Returns**

The return value of the invoked operation, or a negative error code if not found.

Definition at line 283 of file ebp.c.

```
{
    int i;
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_id == op_id) {
            EBA_INFO("Found op_id %u in slot %d, calling handler...\n", op_id, i);
            return op_entries[i].op_ptr(args, arg_len,mac);
        }
    }
    EBA_ERR("ebp_invoke_op: No matching op_id %u found\n", op_id);
    return -EINVAL;
}
```

#### 4.23.2.5 ebp_op_alloc()

```
int ebp_op_alloc (
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_op_alloc - Handle a remote buffer allocation request. @args: Pointer to the argument provided in the invoke packet. @arg_len: Length of the argument. @mac: Destination MAC address of the invoking node.

This function processes a remote allocation request by parsing the provided arguments and attempting to allocate a buffer accordingly.

**Returns**

0 on success, or 1 on failure.

Definition at line 338 of file ebp.c.

```
00339 {
00340     if (!args || arg_len < sizeof(struct ebp_op_alloc_args)) {
00341         EBA_ERR("ebp_op_alloc: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",
00342                 arg_len, sizeof(struct ebp_op_alloc_args), args);
00343         return -EINVAL;
00344     }
00345
00346     const struct ebp_op_alloc_args *alloc_args = args;
00347     EBA_INFO("ebp_op_alloc: Allocation request received: size = %llu, life_time = %llu, receive
      bufferID = %llu\n",
00348             alloc_args->size, alloc_args->life_time, alloc_args->buffer_id);
00349
00350     void *new_buf = eba_internals_malloc(alloc_args->size, alloc_args->life_time);
00351     if (!new_buf) {
00352         EBA_ERR("ebp_op_alloc: Allocation failed for size %llu\n", alloc_args->size);
00353         return -ENOMEM;
00354     }
00355     uint64_t buf_id = (uint64_t)new_buf;
00356     ebp_remote_write(alloc_args->buffer_id,0,sizeof(buf_id),(char *)&buf_id,mac);
00357     return 0;
00358 }
```

#### 4.23.2.6 ebp_op_read()

```
int ebp_op_read (
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_op_read - Handle a remote read operation request. @args: Pointer to the argument blob from the invoke packet. @arg_len: Length of the argument data. @mac: Destination MAC address of the invoking node.

This function processes a request to read data from a remote buffer.

**Returns**

0 on success, or 1 on failure.

Definition at line 359 of file ebp.c.

```
{
    /* Verify that a valid argument is provided and that it's at least as large as our header. */
    if (!args || arg_len < sizeof(struct ebp_op_read_args)) {
        EBA_ERR("ebp_op_read: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",arg_len, sizeof(struct ebp_op_read_args), args);
        return -EINVAL;
    }
    const struct ebp_op_read_args *rd_args = args;

    EBA_INFO("ebp_op_read: dest_buffer_id = %llu, src_buffer_id = %llu, dst_offset = %llu, src_offset = %llu, size = %llu\n",rd_args->dst_buffer_id,rd_args->src_buffer_id,
         rd_args->dst_offset,rd_args->src_offset, rd_args->size);
    // TODO here inquue to the invoke queue, if it is queued send an ack
    void* read_data = kmalloc(rd_args->size,GFP_KERNEL);
    int ret = eba_internals_read(read_data, rd_args->src_buffer_id, rd_args->src_offset, rd_args->size);
    ebp_remote_write(rd_args->dst_buffer_id,rd_args->dst_offset,rd_args->size,(const char*)read_data,mac);

    kfree(read_data);
    //build packet and send it then free the read_data
    // TODO send the invoke with ( write request to the concerned node with read_data as the payload !).

    if (ret < 0) {
        EBA_ERR("ebp_op_read: eba_internals_read() failed with error %d\n", ret);
        return ret;
    }

    EBA_INFO("ebp_op_read: Successfully read %llu bytes from src_buffer_id %llu into dest_buffer_id %llu\n", rd_args->size, rd_args->src_buffer_id, rd_args->dst_buffer_id);
    return 0;
}
```

#### 4.23.2.7 ebp_op_write()

```
int ebp_op_write (
            const void * args,
            uint64_t arg_len,
            const char mac[6] )
```

ebp_op_write - Handle a remote write operation request. @args: Pointer to the argument from the invoke packet. @arg_len: Length of the argument data. @mac: Destination MAC address of the invoking node.

This function processes a request to write data to a remote buffer.

**Returns**

0 on success, or 1 on failure.

Definition at line 310 of file ebp.c.

```
{
    /* Check that args is not NULL and that its is large enough for our fixed-size header */
    if (!args || arg_len < sizeof(struct ebp_op_write_args))
    {
        EBA_ERR("ebp_op_write: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",arg_len, sizeof(struct ebp_op_write_args), args);
        return -EINVAL;
    }

    const struct ebp_op_write_args *wr = args;
    uint64_t header_size = sizeof(struct ebp_op_write_args);

    if (arg_len < header_size + wr->size)
    {
        EBA_ERR("ebp_op_write: args too short for payload data: arg_len = %llu, header_size = %llu, expected payload size = %llu\n",
                arg_len, header_size, wr->size);
```

```
        return -EINVAL;
    }

    const uint8_t *payload = (const uint8_t *)args + header_size;

    EBA_INFO("ebp_op_write: buff_id = %llx offset = %llu size = %llu\n",wr->buff_id, wr->offset, wr->size);
     // TODO here inqueue to the invoke queue and send an ack
    //print_hex_dump(KERN_INFO, "ebp_op_write payload :", DUMP_PREFIX_OFFSET, 16, 1,payload, wr->size, true);
    send_invoke_ack_packet(INVOKE_QUEUED,mac,"enp0s8");
    return eba_internals_write(payload, wr->buff_id, wr->offset, wr->size);
}
```

#### 4.23.2.8 ebp_ops_init()

```
int ebp_ops_init (
            void  )
```

ebp_ops_init - Initialize and register the default EBA operations.

This function registers the default set of EBA operations (alloc, read, write) into the global op_entries array.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 298 of file ebp.c.

```
{
    int ret = 0;
    if (ebp_register_op(EBP_OP_ALLOC,ebp_op_alloc) < 0)
        ret = -1;
    if (ebp_register_op(EBP_OP_WRITE,ebp_op_write) < 0)
        ret = -1;
    if (ebp_register_op(EBP_OP_READ,ebp_op_read) < 0)
        ret = -1;
    return ret;
}
```

#### 4.23.2.9 ebp_register_node()

```
int ebp_register_node (
            uint16_t mtu,
            const char mac[6],
            uint64_t node_specs )
```

ebp_register_node - Register a new node in the global node_infos array. @mtu: Maximum Transmission Unit (MTU) for the node. @mac: MAC address of the node. @node_specs: Identifier for the buffer containing node specification details.

This function checks if a node with the specified MAC address is already registered. If not, it adds the node to the node_infos array, increments the global node count, and returns 0. If the node is already registered, it returns -EEXIST. If there is no space available, it returns -ENOSPC.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 427 of file ebp.c.

```
{
    int free_slot = -1;
    int i;

    /* Find the first free slot (indicated by id == -1) */
    for (i = 0; i < MAX_NODE_COUNT; i++) {
        if (node_infos[i].id == INVALID_NODE_ID) {
            free_slot = i;
            break;
        }
    }
    if (free_slot == -1) {
        EBA_ERR("ebp_register_node: No space left to register a new node.\n");
        return -ENOSPC;
    }
    /* Register the node in the found free slot */
    node_infos[free_slot].id = nodes_count;
    node_infos[free_slot].mtu = mtu;
    memcpy(node_infos[free_slot].mac, mac, 6);
    node_infos[free_slot].node_specs = node_specs;
    EBA_INFO("ebp_register_node: Registered node with NodeID=%d (slot %d), MAC=%pM, MTU=%u, node_specs=%llu\n",
            node_infos[free_slot].id, free_slot, mac, mtu, node_specs);
    nodes_count++;
    print_node_infos();
    return 0;
}
```

#### 4.23.2.10 ebp_register_op()

```
int ebp_register_op (
            uint32_t op_id,
            ebp_op_t fn )
```

Definition at line 252 of file ebp.c.

```
{
    int i;

    if (!fn) {
        EBA_ERR("eba_register_op: null function pointer\n");
        return -EINVAL;
    }

    /* Check if this op_id is already registered */
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_id == op_id) {
            EBA_ERR("eba_register_op: op_id %u already exists (slot %d)\n", op_id, i);
            return -EEXIST;
        }
    }

    /* Find a free entry */
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_ptr == NULL && op_entries[i].op_id == 0) {
            op_entries[i].op_id  = op_id;
            op_entries[i].op_ptr = fn;
            EBA_INFO("Registered op_id %u in slot %d\n", op_id, i);
            return 0;
        }
    }
    EBA_ERR("eba_register_op: No space for new op_id %u\n", op_id);
    return -ENOSPC;
}
```

#### 4.23.2.11 ebp_remote_alloc()

```
int ebp_remote_alloc (
            uint64_t size,
```

```
            uint64_t life_time,
            uint64_t local_buff_id,
            const char mac[6] )
```

ebp_remote_alloc - Request a buffer allocation from a remote node. @size: Size of the buffer to allocate. @life_↩time: Lifetime of the buffer allocation. @local_buff_id: Local buffer identifier where the allocated buffer's ID will be stored. @mac: MAC address of the remote node (to be modified to a node ID in the future).

This function sends a remote allocation request to a distant node.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 387 of file ebp.c.

```
{
    struct ebp_op_alloc_args alloc_args = {
        .size       = size,
        .life_time  = life_time,
        .buffer_id  = local_buff_id
    };
    send_invoke_req_packet(0x1234,EBP_OP_ALLOC,(char *)&alloc_args,sizeof(alloc_args),NULL,0,mac,"enp0s8");
    EBA_INFO("ebp_remote_alloc: Sent EBP_OP_ALLOC request with local_buf_id = %llu\n",local_buff_id);
    return 0;

}
```

#### 4.23.2.12 ebp_remote_read()

```
int ebp_remote_read (
            uint64_t dst_buffer_id,
            uint64_t src_buffer_id,
            uint64_t dst_offset,
            uint64_t src_offset,
            uint64_t size,
            const char mac[6] )
```

ebp_remote_read - Read data from a remote pre-allocated buffer into a local buffer. @dst_buffer_id: Local buffer identifier where the data will be stored. @src_buffer_id: Remote buffer identifier to read data from. @dst_offset: Byte offset within the local buffer. @src_offset: Byte offset within the remote buffer where reading should start. @size: Number of bytes to read. @mac: MAC address of the remote node (to be modified to a node ID in the future).

This function sends a request to read data from a remote node's buffer into a local buffer.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 413 of file ebp.c.

```
{
    struct ebp_op_read_args read_args = {
        .dst_buffer_id = dst_buffer_id,
        .src_buffer_id = src_buffer_id,
        .dst_offset    = dst_offset,
        .src_offset    = src_offset,
        .size          = size
    };
    send_invoke_req_packet(0x1234,EBP_OP_READ,(char *)&read_args,sizeof(read_args),NULL,0,mac,"enp0s8");
    EBA_INFO("ebp_remote_read: Sent EBP_OP_READ request from  %llu to %llu\n",src_buffer_id,dst_buffer_id);
    return 0;

}
```

#### 4.23.2.13 ebp_remote_write()

```
int ebp_remote_write (
            uint64_t buff_id,
            uint64_t offset,
            uint64_t size,
            const char * payload,
            const char mac[6] )
```

ebp_remote_write - Write data to a remote pre-allocated buffer. @buff_id: Remote buffer identifier. @offset: Byte offset in the remote buffer where writing should begin. @size: Size of the data payload to write. @payload: Pointer to the data payload. @mac: MAC address of the remote node (to be modified to a node ID in the future).

This function sends a request to write data to a remote node's buffer.

**Returns**

0 on success, or a negative error code on failure.

Definition at line 400 of file ebp.c.

```
00401 {
00402     struct ebp_op_write_args write_args = {
00403         .buff_id = buff_id,
00404         .offset = offset,
00405         .size = size
00406     };
00407     send_invoke_req_packet(0x1234,EBP_OP_WRITE,(char*)&write_args,sizeof(write_args),payload,
      write_args.size,mac,"enp0s8");
00408     EBA_INFO("ebp_remote_write: Sent EBP_OP_WRITE request on buffer %llu\n",buff_id);
00409     return 0;
00410
00411 }
```

#### 4.23.2.14 invoke_tracker_array_init()

```
int invoke_tracker_array_init (
            void )
```

invoke_tracker_array_init - Initialize an array of invoke_tracker structures.

This function initializes each element in the invoke_tracker array by setting:

- iid to 0,
- pid to 0,
- done to false,
- status to INVOKE_DEFAULT.

**Returns**

0 on success, or -EINVAL if the provided array pointer is NULL.

Definition at line 229 of file ebp.c.

```
00230 {
00231     uint64_t i;
00232     for (i = 0; i < MAX_INVOKE_COUNT; i++)
00233     {
00234         invoke_trackers[i].iid = 0;
00235         invoke_trackers[i].pid = 0;
00236         invoke_trackers[i].done = false;
00237         invoke_trackers[i].status = INVOKE_DEFAULT;
00238     }
00239     return 0;
00240 }
```

#### 4.23.2.15 node_info_array_init()

```
int node_info_array_init (
            void  )
```

node_info_array_init - Initialize an array of node_info structures.

This function initializes each element in the node_info array by setting:

- id to 0,
- mtu to 0,
- mac to zero,
- node_specs to 0.

**Returns**

0 on success, or -EINVAL if the provided array pointer is NULL.

Definition at line 215 of file ebp.c.

```
00216 {
00217     uint64_t i;
00218     for (i = 0; i < MAX_NODE_COUNT; i++)
00219     {
00220         node_infos[i].id = INVALID_NODE_ID;
00221         node_infos[i].mtu = 0;
00222         /* Zero out the 6-byte MAC address */
00223         memset(node_infos[i].mac, 0, sizeof(node_infos[i].mac));
00224         node_infos[i].node_specs = 0;
00225     }
00226     return 0;
00227 }
```

#### 4.23.2.16 op_entry_array_init()

```
int op_entry_array_init (
            void  )
```

op_entry_array_init - Initialize an array of op_entry structures.

This function initializes each element in the op_entry array by setting:

- op_id to 0,
- op_ptr to NULL.

**Returns**

0 on success, or -EINVAL if the provided array pointer is NULL.

Definition at line 242 of file ebp.c.

```
00243 {
00244     uint64_t i;
00245     for (i = 0; i < MAX_OP_COUNT; i++)
00246     {
00247         op_entries[i].op_id = 0;
00248         op_entries[i].op_ptr = NULL;
00249     }
00250     return 0;
00251 }
```

#### 4.23.2.17 print_node_infos()

```
void print_node_infos (
            void  )
```

Definition at line 189 of file ebp.c.

```
00190 {
00191     int i;
00192     EBA_INFO("=== node_infos Array Dump ===\n");
00193     for (i = 0; i < MAX_NODE_COUNT; i++) {
00194         if (node_infos[i].id != INVALID_NODE_ID) {
00195             EBA_INFO("Slot %d: NodeID=%d, MTU=%u, MAC=%pM, node_specs=%llu\n",
00196                     i,
00197                     node_infos[i].id,
00198                     node_infos[i].mtu,
00199                     node_infos[i].mac,
00200                     node_infos[i].node_specs);
00201         }
00202     }
00203     EBA_INFO("============================\n");
00204 }
```

#### 4.23.2.18 print_op_entries()

```
void print_op_entries (
            void  )
```

print_op_entries - Display the current list of operation entries.

This utility function prints the registered operation entries for debugging and verification.

Definition at line 205 of file ebp.c.

```
00206 {
00207     int i;
00208     EBA_INFO("=== op_entries Array Dump ===\n");
00209     for (i = 0; i < MAX_OP_COUNT; i++) {
00210         EBA_INFO("Slot %d: op_id = %u, op_ptr = %p\n",
00211                  i, op_entries[i].op_id, op_entries[i].op_ptr);
00212     }
00213     EBA_INFO("============================\n");
00214 }
```

### 4.23.3 Variable Documentation

#### 4.23.3.1 ebp_packet_type

```
struct packet_type ebp_packet_type
```

**Initial value:**

```
= {
    .type = htons(EBP_ETHERTYPE),
    .dev = NULL,
    .func = ebp_handle_packets,
}
```

Definition at line 11 of file ebp.c.

```
00011                                   {
00012     .type = htons(EBP_ETHERTYPE), /* Custom protocol type in network byte order */
00013     .dev = NULL,                  /* NULL = match on all devices */
00014     .func = ebp_handle_packets,   /* Packet receive callback function */
00015 };
```

#### 4.23.3.2 invoke_trackers

```
struct invoke_tracker invoke_trackers[MAX_INVOKE_COUNT]
```

Definition at line 186 of file ebp.c.

#### 4.23.3.3 local_specs

```
void* local_specs = NULL
```

Definition at line 7 of file ebp.c.

#### 4.23.3.4 node_infos

```
struct node_info node_infos[MAX_NODE_COUNT]
```

Definition at line 184 of file ebp.c.

#### 4.23.3.5 nodes_count

```
int nodes_count = 1
```

Definition at line 185 of file ebp.c.

#### 4.23.3.6 op_entries

```
struct op_entry op_entries[MAX_OP_COUNT]
```

Definition at line 187 of file ebp.c.

## 4.24 ebp.c

Go to the documentation of this file.

```
00001 #include "ebp.h"
00002 #include "eba_internals.h"
00003 #include "eba_net.h"
00004 #include <linux/delay.h>
00005 #include "eba.h"
00006 #include "eba_utils.h"
00007 void* local_specs = NULL;
00008 /*
00009  * Global packet_type structure for our extended buffer protocol.
00010  */
00011 struct packet_type ebp_packet_type = {
00012     .type = htons(EBP_ETHERTYPE), /* Custom protocol type in network byte order */
00013     .dev = NULL,                  /* NULL = match on all devices */
00014     .func = ebp_handle_packets,   /* Packet receive callback function */
00015 };
00016 //handle errors perfectly
00017 int ebp_handle_packets(struct sk_buff *skb, struct net_device *dev,
00018                        struct packet_type *pt, struct net_device *orig_dev)
00019 {
00020     struct ebp_header *eba_hdr;
00021     unsigned char *payload;
00022     int payload_len;
00023     struct ethhdr *eth;
```

```

    if (!skb)
    {
        EBA_ERR("ebp_handle_packets: Packet is NULL\n");
        return NET_RX_DROP;
    }

    /* Verify packet is long enough for an Ethernet header + EBA header */
    if (skb->len < ETH_HLEN + sizeof(struct ebp_header))
    {
        EBA_ERR("ebp_handle_packets: Packet too short on device %s, length: %u\n",dev->name, skb->len);
        kfree_skb(skb);
        return NET_RX_DROP;
    }

    /* Get the Ethernet header */
    eth = eth_hdr(skb);
    EBA_INFO("ebp_handle_packets: Source MAC address: %pM\n", eth->h_source);

    /* EBA header */
    payload = skb->data;
    payload_len = skb->len;
    eba_hdr = (struct ebp_header *)payload;

    EBA_INFO("ebp_handle_packets: Received packet on %s, protocol: 0x%04x, length: %u, EBA msgType: 0x%02x\n",dev->name, ntohs(skb->protocol), skb->len, eba_hdr->msgType);

    switch (eba_hdr->msgType)
    {
    case EBP_MSG_DISCOVER: /* 0x01 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_discover_req))
        {
            EBA_ERR("ebp_handle_packets: Discover Request packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_discover_req *disc = (struct ebp_discover_req *)eba_hdr;
            EBA_INFO("ebp_handle_packets: Discover Request: MTU = %u\n", ntohs(disc->mtu));
            //allocate node specs buffer
            void* node_specs = eba_internals_malloc(EBP_NODE_SPECS_MAX_SIZE,EBP_NODE_SPECS_MAX_LIFE_TIME);
            //register the new node
            //handle errors lol
            int ret = ebp_register_node(ntohs(disc->mtu),eth->h_source,(uint64_t)node_specs);
            if(ret < 0)
            {
                EBA_ERR("ebp_handle_packets: register node failed\n");
                kfree_skb(skb);
                return NET_RX_DROP;
            }
            //send the discover ack with the allocated buffer to enable remote node to share its data.
            /*int ret = send_discover_ack_packet((uint64_t)node_specs,eth->h_source,"enp0s8");
            if(ret < 0)
            {
                EBA_ERR("ebp_handle_packets: send discover ack packet failed\n");
                kfree_skb(skb);
                return NET_RX_DROP;
            }*/
        }
        break;
    case EBP_MSG_DISCOVER_ACK: /* 0x03 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_discover_ack))
        {
            EBA_ERR("ebp_handle_packets: Discover Ack packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_discover_ack *ack = (struct ebp_discover_ack *)eba_hdr;
            uint64_t buff_id = be64_to_cpu(ack->buffer_id);
            EBA_INFO("ebp_handle_packets: Discover Ack: buffer_id = 0x%llu \n", buff_id);
            //todo handle mtu
            /*int rc = eba_utils_file_to_buf("/var/lib/eba/node_local.eba",(uint64_t)local_specs);
            if (rc < 0) {
                EBA_ERR("ebp_handle_packets: Failed to read node_local.eba, ret=%d\n", rc);
                kfree_skb(skb);
                return NET_RX_DROP;
            } else {
                EBA_INFO("ebp_handle_packets: Read %llu bytes from node_local.eba, sending to remote\n", (uint64_t)4096);
                uint64_t offset = 0;
                for(int i = 0 ; i < 4 ; i++)
                {
```

```
                    rc = ebp_remote_write(buff_id, offset, (uint64_t)1000, (char*) (local_specs) + offset, eth->h_source);
                    if (rc < 0) {
                        EBA_ERR("ebp_handle_packets: Failed to ebp_remote_write() node_local.eba content, ret=%d\n", rc);
                    }
                    offset+= 1000;
                }
            }
        */
        }
        break;
    case EBP_MSG_INVOKE: /* 0x02 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_invoke_req))
        {
            EBA_ERR("ebp_handle_packets: Invoke Request packet too short\n");
        }
        else
        {
            struct ebp_invoke_req *inv = (struct ebp_invoke_req *)eba_hdr;
            uint32_t iid = ntohl(inv->iid);
            uint32_t opid = ntohl(inv->opid);
            uint64_t args_len = be64_to_cpu(inv->args_len);

            EBA_INFO("ebp_handle_packets: Invoke Request: IID = %u, OpID = %u, args_len = %llu\n",iid, opid, args_len);
            int ret = ebp_invoke_op(opid, inv->args, args_len, eth->h_source);
            if (ret < 0) {
                EBA_ERR("ebp_handle_packets: ebp_invoke_op(opid=%u) failed ret=%d\n", opid, ret);
                kfree_skb(skb);
                return NET_RX_DROP;
                /* Possibly build & send an error ack or something... TODO */
            } else {
                /* Possibly build an ack for success... TODO */
            }

        }
        break;
    case EBP_MSG_INVOKE_ACK: /* 0x04 */
        if (skb->len < ETH_HLEN + sizeof(struct ebp_invoke_ack))
        {
            EBA_ERR("ebp_handle_packets: Invoke Ack packet too short\n");
            kfree_skb(skb);
            return NET_RX_DROP;
        }
        else
        {
            struct ebp_invoke_ack *inv_ack = (struct ebp_invoke_ack *)eba_hdr;
            EBA_INFO("ebp_handle_packets: Invoke Ack: status = 0x%02x TODO \n ", inv_ack->status);
        }
        break;
    default:
        EBA_ERR("ebp_handle_packets: Unknown EBA msgType: 0x%02x\n", eba_hdr->msgType);
        kfree_skb(skb);
        return NET_RX_DROP;
        break;
    }

    kfree_skb(skb);
    return NET_RX_SUCCESS;
}

void ebp_init(void)
{
    dev_add_pack(&ebp_packet_type);
    node_info_array_init();
    invoke_tracker_array_init();
    op_entry_array_init();
    ebp_ops_init();
    local_specs = eba_internals_malloc(4096,0);
    char mac[6] = {0xff,0xff,0xff,0xff,0xff,0xff};
    send_discover_req_packet(69,mac,"enp0s8");
    print_node_infos();
}
void ebp_exit(void)
{
    dev_remove_pack(&ebp_packet_type);
}
#define INVALID_NODE_ID 0xffff
/* Global arrays for the EBA protocol data structures */
struct node_info node_infos[MAX_NODE_COUNT];
int nodes_count = 1;
struct invoke_tracker invoke_trackers[MAX_INVOKE_COUNT];
struct op_entry op_entries[MAX_OP_COUNT];
/* Add a helper function to print the global node_infos array */
void print_node_infos(void)
{
```

```
    int i;
    EBA_INFO("=== node_infos Array Dump ===\n");
    for (i = 0; i < MAX_NODE_COUNT; i++) {
        if (node_infos[i].id != INVALID_NODE_ID) {
            EBA_INFO("Slot %d: NodeID=%d, MTU=%u, MAC=%pM, node_specs=%llu\n",
                    i,
                    node_infos[i].id,
                    node_infos[i].mtu,
                    node_infos[i].mac,
                    node_infos[i].node_specs);
        }
    }
    EBA_INFO("============================\n");
}
void print_op_entries(void)
{
    int i;
    EBA_INFO("=== op_entries Array Dump ===\n");
    for (i = 0; i < MAX_OP_COUNT; i++) {
        EBA_INFO("Slot %d: op_id = %u, op_ptr = %p\n",
                 i, op_entries[i].op_id, op_entries[i].op_ptr);
    }
    EBA_INFO("============================\n");
}
int node_info_array_init(void)
{
    uint64_t i;
    for (i = 0; i < MAX_NODE_COUNT; i++)
    {
        node_infos[i].id = INVALID_NODE_ID;
        node_infos[i].mtu = 0;
        /* Zero out the 6-byte MAC address */
        memset(node_infos[i].mac, 0, sizeof(node_infos[i].mac));
        node_infos[i].node_specs = 0;
    }
    return 0;
}

int invoke_tracker_array_init(void)
{
    uint64_t i;
    for (i = 0; i < MAX_INVOKE_COUNT; i++)
    {
        invoke_trackers[i].iid = 0;
        invoke_trackers[i].pid = 0;
        invoke_trackers[i].done = false;
        invoke_trackers[i].status = INVOKE_DEFAULT;
    }
    return 0;
}

int op_entry_array_init(void)
{
    uint64_t i;
    for (i = 0; i < MAX_OP_COUNT; i++)
    {
        op_entries[i].op_id = 0;
        op_entries[i].op_ptr = NULL;
    }
    return 0;
}
int ebp_register_op(uint32_t op_id, ebp_op_t fn)
{
    int i;

    if (!fn) {
        EBA_ERR("eba_register_op: null function pointer\n");
        return -EINVAL;
    }

    /* Check if this op_id is already registered */
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_id == op_id) {
            EBA_ERR("eba_register_op: op_id %u already exists (slot %d)\n", op_id, i);
            return -EEXIST;
        }
    }

    /* Find a free entry */
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_ptr == NULL && op_entries[i].op_id == 0) {
            op_entries[i].op_id  = op_id;
            op_entries[i].op_ptr = fn;
            EBA_INFO("Registered op_id %u in slot %d\n", op_id, i);
            return 0;
        }
    }
```

```c
    EBA_ERR("eba_register_op: No space for new op_id %u\n", op_id);
    return -ENOSPC;
}


int ebp_invoke_op(uint32_t op_id, const void *args, uint64_t arg_len, const char mac[6])
{
    int i;
    for (i = 0; i < MAX_OP_COUNT; i++) {
        if (op_entries[i].op_id == op_id) {
            EBA_INFO("Found op_id %u in slot %d, calling handler...\n", op_id, i);
            return op_entries[i].op_ptr(args, arg_len,mac);
        }
    }
    EBA_ERR("ebp_invoke_op: No matching op_id %u found\n", op_id);
    return -EINVAL;
}



int ebp_ops_init(void)
{
    int ret = 0;
    if (ebp_register_op(EBP_OP_ALLOC,ebp_op_alloc) < 0)
        ret = -1;
    if (ebp_register_op(EBP_OP_WRITE,ebp_op_write) < 0)
        ret = -1;
    if (ebp_register_op(EBP_OP_READ,ebp_op_read) < 0)
        ret = -1;
    return ret;
}

int ebp_op_write(const void *args, uint64_t arg_len, const char mac[6])
{
    /* Check that args is not NULL and that its is large enough for our fixed-size header */
    if (!args || arg_len < sizeof(struct ebp_op_write_args))
    {
        EBA_ERR("ebp_op_write: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",arg_len, sizeof(struct ebp_op_write_args), args);
        return -EINVAL;
    }

    const struct ebp_op_write_args *wr = args;
    uint64_t header_size = sizeof(struct ebp_op_write_args);

    if (arg_len < header_size + wr->size)
    {
        EBA_ERR("ebp_op_write: args too short for payload data: arg_len = %llu, header_size = %llu, expected payload size = %llu\n",
                arg_len, header_size, wr->size);
        return -EINVAL;
    }

    const uint8_t *payload = (const uint8_t *)args + header_size;

    EBA_INFO("ebp_op_write: buff_id = %llx offset = %llu size = %llu\n",wr->buff_id, wr->offset, wr->size);
     // TODO here inqueue to the invoke queue and send an ack
    //print_hex_dump(KERN_INFO, "ebp_op_write payload :", DUMP_PREFIX_OFFSET, 16, 1,payload, wr->size, true);
    send_invoke_ack_packet(INVOKE_QUEUED,mac,"enp0s8");
    return eba_internals_write(payload, wr->buff_id, wr->offset, wr->size);
}

int ebp_op_alloc(const void *args, uint64_t arg_len, const char mac[6])
{
    if (!args || arg_len < sizeof(struct ebp_op_alloc_args)) {
        EBA_ERR("ebp_op_alloc: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",
                arg_len, sizeof(struct ebp_op_alloc_args), args);
        return -EINVAL;
    }

    const struct ebp_op_alloc_args *alloc_args = args;
    EBA_INFO("ebp_op_alloc: Allocation request received: size = %llu, life_time = %llu, receive bufferID = %llu\n",
            alloc_args->size, alloc_args->life_time, alloc_args->buffer_id);

    void *new_buf = eba_internals_malloc(alloc_args->size, alloc_args->life_time);
    if (!new_buf) {
        EBA_ERR("ebp_op_alloc: Allocation failed for size %llu\n", alloc_args->size);
        return -ENOMEM;
    }
    uint64_t buf_id = (uint64_t)new_buf;
    ebp_remote_write(alloc_args->buffer_id,0,sizeof(buf_id),(char *)&buf_id,mac);
    return 0;
}
int ebp_op_read(const void *args, uint64_t arg_len, const char mac[6])
```

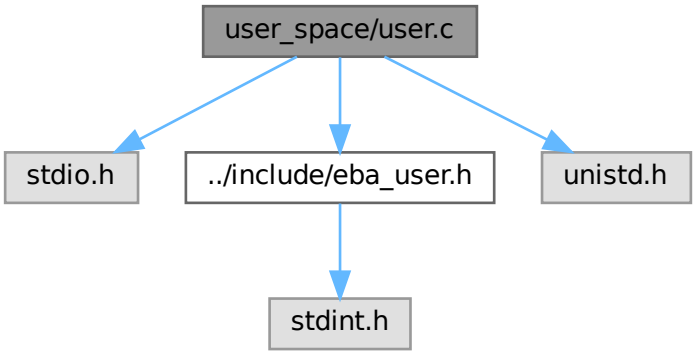
```
{
    /* Verify that a valid argument is provided and that it's at least as large as our header. */
    if (!args || arg_len < sizeof(struct ebp_op_read_args)) {
        EBA_ERR("ebp_op_read: invalid arg_len %llu (must be >= %zu) or null pointer %p\n",arg_len, sizeof(struct ebp_op_read_args), args);
        return -EINVAL;
    }
    const struct ebp_op_read_args *rd_args = args;

    EBA_INFO("ebp_op_read: dest_buffer_id = %llu, src_buffer_id = %llu, dst_offset = %llu, src_offset = %llu, size = %llu\n",rd_args->dst_buffer_id,rd_args->src_buffer_id,
         rd_args->dst_offset,rd_args->src_offset, rd_args->size);
    // TODO here inquue to the invoke queue, if it is queued send an ack
    void* read_data = kmalloc(rd_args->size,GFP_KERNEL);
    int ret = eba_internals_read(read_data, rd_args->src_buffer_id, rd_args->src_offset, rd_args->size);
    ebp_remote_write(rd_args->dst_buffer_id,rd_args->dst_offset,rd_args->size,(const char*)read_data,mac);

    kfree(read_data);
    //build packet and send it then free the read_data
    // TODO send the invoke with ( write request to the concerned node with read_data as the payload !).

    if (ret < 0) {
        EBA_ERR("ebp_op_read: eba_internals_read() failed with error %d\n", ret);
        return ret;
    }

    EBA_INFO("ebp_op_read: Successfully read %llu bytes from src_buffer_id %llu into dest_buffer_id %llu\n", rd_args->size, rd_args->src_buffer_id, rd_args->dst_buffer_id);
    return 0;
}
int ebp_remote_alloc(uint64_t size, uint64_t life_time, uint64_t local_buff_id,const char mac[6]/* TODO modify it to be come node*/)
{
    struct ebp_op_alloc_args alloc_args = {
        .size       = size,
        .life_time  = life_time,
        .buffer_id  = local_buff_id
    };
    send_invoke_req_packet(0x1234,EBP_OP_ALLOC,(char*)&alloc_args,sizeof(alloc_args),NULL,0,mac,"enp0s8");
    EBA_INFO("ebp_remote_alloc: Sent EBP_OP_ALLOC request with local_buf_id = %llu\n",local_buff_id);
    return 0;

}

int ebp_remote_write(uint64_t buff_id, uint64_t offset, uint64_t size,const char* payload ,const char mac[6]/* TODO modify it to be come node*/)
{
    struct ebp_op_write_args write_args = {
        .buff_id = buff_id,
        .offset = offset,
        .size = size
    };
    send_invoke_req_packet(0x1234,EBP_OP_WRITE,(char*)&write_args,sizeof(write_args),payload, write_args.size,mac,"enp0s8");
    EBA_INFO("ebp_remote_write: Sent EBP_OP_WRITE request on buffer %llu\n",buff_id);
    return 0;

}

int ebp_remote_read(uint64_t dst_buffer_id, uint64_t src_buffer_id, uint64_t dst_offset,uint64_t src_offset ,uint64_t size,const char mac[6]/* TODO modify it to be come node*/)
{
    struct ebp_op_read_args read_args = {
        .dst_buffer_id = dst_buffer_id,
        .src_buffer_id = src_buffer_id,
        .dst_offset    = dst_offset,
        .src_offset    = src_offset,
        .size          = size
    };
    send_invoke_req_packet(0x1234,EBP_OP_READ,(char*)&read_args,sizeof(read_args),NULL,0,mac,"enp0s8");
    EBA_INFO("ebp_remote_read: Sent EBP_OP_READ request from  %llu to %llu\n",src_buffer_id,dst_buffer_id);
    return 0;

}
int ebp_register_node(uint16_t mtu, const char mac[6], uint64_t node_specs)
{
    int free_slot = -1;
    int i;

    /* Find the first free slot (indicated by id == -1) */
    for (i = 0; i < MAX_NODE_COUNT; i++) {
```

```
        if (node_infos[i].id == INVALID_NODE_ID) {
            free_slot = i;
            break;
        }
    }
    if (free_slot == -1) {
        EBA_ERR("ebp_register_node: No space left to register a new node.\n");
        return -ENOSPC;
    }
    /* Register the node in the found free slot */
    node_infos[free_slot].id = nodes_count;
    node_infos[free_slot].mtu = mtu;
    memcpy(node_infos[free_slot].mac, mac, 6);
    node_infos[free_slot].node_specs = node_specs;
    EBA_INFO("ebp_register_node: Registered node with NodeID=%d (slot %d), MAC=%pM, MTU=%u,
node_specs=%llu\n",
            node_infos[free_slot].id, free_slot, mac, mtu, node_specs);
    nodes_count++;
    print_node_infos();
    return 0;
}
```

## 4.25 user_space/user.c File Reference

```
#include <stdio.h>
#include "../include/eba_user.h"
#include <unistd.h>
```

Include dependency graph for user.c:




### Functions

- int main (int argc, char *argv[])

### 4.25.1 Function Documentation

#### 4.25.1.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Definition at line 4 of file user.c.

```
{
   uint64_t id1 = eba_alloc(512,10,0);
   const char mac[6]={0x00,0x00,0x00,0x00,0x00,0x02};
   eba_remote_alloc(512,100,id1,mac);
   uint64_t remote_id = 0;
   while(remote_id == 0)
   {
      sleep(1);
      eba_read(&remote_id,id1,0,8);
   }
   eba_remote_write(remote_id,0,sizeof("hello friend"),"hello friend",mac);
   uint64_t id2 = eba_alloc(512,10,0);
   eba_remote_read(id2,remote_id,0,0,sizeof("hello friend"),mac);
   char c[13] = {0};
   while(c[0] == 0)
   {
      sleep(1);
      eba_read(c,id2,0,13);
   }
   printf("read:\n");
   for(int i = 0 ; i < sizeof("hello friend"); i++)
     printf("%c",c[i]);
}
```

## 4.26 user.c

Go to the documentation of this file.

```
#include <stdio.h>
#include "../include/eba_user.h"
#include <unistd.h>
int main(int argc, char *argv[])
 {
   uint64_t id1 = eba_alloc(512,10,0);
   const char mac[6]={0x00,0x00,0x00,0x00,0x00,0x02};
   eba_remote_alloc(512,100,id1,mac);
   uint64_t remote_id = 0;
   while(remote_id == 0)
   {
      sleep(1);
      eba_read(&remote_id,id1,0,8);
   }
   eba_remote_write(remote_id,0,sizeof("hello friend"),"hello friend",mac);
   uint64_t id2 = eba_alloc(512,10,0);
   eba_remote_read(id2,remote_id,0,0,sizeof("hello friend"),mac);
   char c[13] = {0};
   while(c[0] == 0)
   {
      sleep(1);
      eba_read(c,id2,0,13);
   }
   printf("read:\n");
   for(int i = 0 ; i < sizeof("hello friend"); i++)
     printf("%c",c[i]);
}

```

# Index


<Ethernet>, 21
__attribute__
    ebp.h, 65

address
    eba_buffer, 6
args
    ebp.h, 76
    ebp_invoke_req, 17
args_len
    ebp.h, 76
    ebp_invoke_req, 17

buff_id
    eba_alloc_data, 5
    eba_remote_write, 11
    eba_rw_data, 12
    ebp.h, 76
    ebp_op_write_args, 20
buffer_id
    eba_remote_alloc, 8
    ebp.h, 77
    ebp_discover_ack, 13
    ebp_op_alloc_args, 18
build_discover_ack_packet
    eba_net.c, 104
    eba_net.h, 41
build_discover_req_packet
    eba_net.c, 104
    eba_net.h, 42
build_invoke_ack_packet
    eba_net.c, 104
    eba_net.h, 42
build_invoke_req_packet
    eba_net.c, 105
    eba_net.h, 42

DEBUG
    eba.c, 82
done
    invoke_tracker, 22
dst_buffer_id
    eba_remote_read, 9
    ebp.h, 77
    ebp_op_read_args, 19
dst_offset
    eba_remote_read, 9
    ebp.h, 77
    ebp_op_read_args, 19

eba.c
    DEBUG, 82
    eba_cdev, 85
    eba_class, 85
    eba_devno, 85
    eba_fops, 85
    eba_ioctl, 82
    eba_module_exit, 83
    eba_module_init, 83
    eba_open, 84
    eba_release, 84
    eba_timer, 86
    eba_timer_callback, 84
    MODULE_AUTHOR, 84
    MODULE_DESCRIPTION, 84
    module_exit, 85
    module_init, 85
    MODULE_LICENSE, 85
    MODULE_VERSION, 85
eba.h
    EBA_CLEAN_BUFFER_CALLBACK_TIMER, 26
    EBA_ERR, 26
    EBA_INFO, 26
    EBA_IOC_MAGIC, 26
    EBA_IOC_MAXNR, 27
    EBA_IOCTL_ALLOC, 27
    EBA_IOCTL_READ, 27
    EBA_IOCTL_REMOTE_ALLOC, 27
    EBA_IOCTL_REMOTE_READ, 27
    EBA_IOCTL_REMOTE_WRITE, 28
    EBA_IOCTL_WRITE, 28
    EBA_WARN, 28
eba_alloc
    eba_user.c, 114
    eba_user.h, 49
eba_alloc_data, 5
    buff_id, 5
    life_time, 5
    size, 5
eba_buffer, 6
    address, 6
    expires, 6
    lock, 6
    node, 7
    size, 7
    type, 7
eba_buffer_list
    eba_utils.c, 126
eba_buffer_list_lock
    eba_internals.c, 97

eba_utils.c, 126
eba_cdev
eba.c, 85
eba_check_expired_buffers
eba_internals.c, 89
eba_internals.h, 31
eba_class
eba.c, 85
EBA_CLEAN_BUFFER_CALLBACK_TIMER
eba.h, 26
eba_devno
eba.c, 85
EBA_ERR
eba.h, 26
eba_export_node_specs
eba_utils.c, 121
eba_utils.h, 55
eba_fops
eba.c, 85
EBA_INFO
eba.h, 26
eba_internals.c
eba_buffer_list_lock, 97
eba_check_expired_buffers, 89
eba_internals_free, 90
eba_internals_malloc, 91
eba_internals_mem_stress_test, 92
eba_internals_mempool_free, 92
eba_internals_mempool_init, 93
eba_internals_read, 94
eba_internals_rw_stress_test, 95
eba_internals_write, 96
eba_pool, 97
eba_pool_mem, 97
LIST_HEAD, 97
eba_internals.h
eba_check_expired_buffers, 31
eba_internals_free, 31
eba_internals_malloc, 32
eba_internals_mem_stress_test, 33
eba_internals_mempool_free, 34
eba_internals_mempool_init, 35
eba_internals_read, 36
eba_internals_rw_stress_test, 37
eba_internals_write, 38
MEMORY_POOL_SIZE, 31
eba_internals_free
eba_internals.c, 90
eba_internals.h, 31
eba_internals_malloc
eba_internals.c, 91
eba_internals.h, 32
eba_internals_mem_stress_test
eba_internals.c, 92
eba_internals.h, 33
eba_internals_mempool_free
eba_internals.c, 92
eba_internals.h, 34
eba_internals_mempool_init
eba_internals.c, 93
eba_internals.h, 35
eba_internals_read
eba_internals.c, 94
eba_internals.h, 36
eba_internals_rw_stress_test
eba_internals.c, 95
eba_internals.h, 37
eba_internals_write
eba_internals.c, 96
eba_internals.h, 38
EBA_IOC_MAGIC
eba.h, 26
EBA_IOC_MAXNR
eba.h, 27
eba_ioctl
eba.c, 82
EBA_IOCTL_ALLOC
eba.h, 27
EBA_IOCTL_READ
eba.h, 27
EBA_IOCTL_REMOTE_ALLOC
eba.h, 27
EBA_IOCTL_REMOTE_READ
eba.h, 27
EBA_IOCTL_REMOTE_WRITE
eba.h, 28
EBA_IOCTL_WRITE
eba.h, 28
eba_module_exit
eba.c, 83
eba_module_init
eba.c, 83
eba_net.c
build_discover_ack_packet, 104
build_discover_req_packet, 104
build_invoke_ack_packet, 104
build_invoke_req_packet, 105
eba_net_get_max_mtu, 105
eba_net_set_mtu, 106
send_discover_ack_packet, 106
send_discover_req_packet, 107
send_invoke_ack_packet, 107
send_invoke_req_packet, 108
send_raw_ethernet_packet, 109
eba_net.h
build_discover_ack_packet, 41
build_discover_req_packet, 42
build_invoke_ack_packet, 42
build_invoke_req_packet, 42
eba_net_get_max_mtu, 43
eba_net_set_mtu, 44
send_discover_ack_packet, 44
send_discover_req_packet, 45
send_invoke_ack_packet, 45
send_invoke_req_packet, 46
send_raw_ethernet_packet, 47

eba_net_get_max_mtu
eba_net.c, 105
eba_net.h, 43
eba_net_set_mtu
eba_net.c, 106
eba_net.h, 44
eba_open
eba.c, 84
eba_pool
eba_internals.c, 97
eba_pool_mem
eba_internals.c, 97
eba_read
eba_user.c, 114
eba_user.h, 50
eba_release
eba.c, 84
eba_remote_alloc, 7
buffer_id, 8
eba_user.c, 115
eba_user.h, 50
life_time, 8
mac, 8
size, 8
eba_remote_read, 8
dst_buffer_id, 9
dst_offset, 9
eba_user.c, 115
eba_user.h, 51
mac, 9
size, 9
src_buffer_id, 10
src_offset, 10
eba_remote_write, 10
buff_id, 11
eba_user.c, 116
eba_user.h, 51
mac, 11
offset, 11
payload, 11
size, 11
eba_rw_data, 11
buff_id, 12
off, 12
size, 12
user_addr, 12
eba_timer
eba.c, 86
eba_timer_callback
eba.c, 84
eba_user.c
eba_alloc, 114
eba_read, 114
eba_remote_alloc, 115
eba_remote_read, 115
eba_remote_write, 116
eba_write, 117
open_eba_device, 117
eba_user.h
eba_alloc, 49
eba_read, 50
eba_remote_alloc, 50
eba_remote_read, 51
eba_remote_write, 51
eba_write, 52
eba_utils.c
eba_buffer_list, 126
eba_buffer_list_lock, 126
eba_export_node_specs, 121
eba_utils_buf_to_file, 121
eba_utils_file_to_buf, 122
node_infos, 126
nodes_count, 126
test_eba_export_node_specs, 123
test_eba_utils_buf_to_file, 124
test_eba_utils_file_to_buf, 125
eba_utils.h
eba_export_node_specs, 55
eba_utils_buf_to_file, 55
eba_utils_file_to_buf, 56
test_eba_export_node_specs, 57
test_eba_utils_buf_to_file, 58
test_eba_utils_file_to_buf, 59
eba_utils_buf_to_file
eba_utils.c, 121
eba_utils.h, 55
eba_utils_file_to_buf
eba_utils.c, 122
eba_utils.h, 56
EBA_WARN
eba.h, 28
eba_write
eba_user.c, 117
eba_user.h, 52
ebp.c
ebp_exit, 131
ebp_handle_packets, 131
ebp_init, 134
ebp_invoke_op, 134
ebp_op_alloc, 134
ebp_op_read, 135
ebp_op_write, 136
ebp_ops_init, 137
ebp_packet_type, 142
ebp_register_node, 137
ebp_register_op, 138
ebp_remote_alloc, 138
ebp_remote_read, 139
ebp_remote_write, 139
INVALID_NODE_ID, 131
invoke_tracker_array_init, 140
invoke_trackers, 142
local_specs, 143
node_info_array_init, 140
node_infos, 143
nodes_count, 143

op_entries, 143
op_entry_array_init, 141
print_node_infos, 141
print_op_entries, 142
ebp.h
__attribute__, 65
args, 76
args_len, 76
buff_id, 76
buffer_id, 77
dst_buffer_id, 77
dst_offset, 77
EBP_ETHERTYPE, 63
ebp_exit, 65
ebp_handle_packets, 65
ebp_init, 68
ebp_invoke_op, 68
EBP_MSG, 64
EBP_MSG_DISCOVER, 64
EBP_MSG_DISCOVER_ACK, 64
EBP_MSG_INVOKE, 64
EBP_MSG_INVOKE_ACK, 64
EBP_NODE_SPECS_MAX_LIFE_TIME, 63
EBP_NODE_SPECS_MAX_SIZE, 63
EBP_OP_ALLOC, 65
ebp_op_alloc, 68
EBP_OP_IDS, 64
EBP_OP_READ, 65
ebp_op_read, 69
ebp_op_t, 64
EBP_OP_WRITE, 65
ebp_op_write, 70
ebp_ops_init, 71
ebp_register_node, 71
ebp_register_op, 72
ebp_remote_alloc, 72
ebp_remote_read, 73
ebp_remote_write, 73
header, 77
iid, 77
INVOKE_COMPLETED, 65
INVOKE_DEFAULT, 65
INVOKE_FAILED, 65
INVOKE_IN_PROGRESS, 65
INVOKE_QUEUED, 65
INVOKE_STATUS, 65
invoke_tracker_array_init, 74
life_time, 77
MAX_INVOKE_COUNT, 63
MAX_NODE_COUNT, 63
MAX_OP_COUNT, 63
msgType, 77
mtu, 77
node_info_array_init, 74
offset, 78
op_entry_array_init, 75
opid, 78
print_node_infos, 75
print_op_entries, 76
size, 78
src_buffer_id, 78
src_offset, 78
status, 78
ebp_discover_ack, 13
buffer_id, 13
header, 13
ebp_discover_req, 14
header, 14
mtu, 14
EBP_ETHERTYPE
ebp.h, 63
ebp_exit
ebp.c, 131
ebp.h, 65
ebp_handle_packets
ebp.c, 131
ebp.h, 65
ebp_header, 15
msgType, 15
ebp_init
ebp.c, 134
ebp.h, 68
ebp_invoke_ack, 15
header, 16
status, 16
ebp_invoke_op
ebp.c, 134
ebp.h, 68
ebp_invoke_req, 16
args, 17
args_len, 17
header, 17
iid, 17
opid, 17
EBP_MSG
ebp.h, 64
EBP_MSG_DISCOVER
ebp.h, 64
EBP_MSG_DISCOVER_ACK
ebp.h, 64
EBP_MSG_INVOKE
ebp.h, 64
EBP_MSG_INVOKE_ACK
ebp.h, 64
EBP_NODE_SPECS_MAX_LIFE_TIME
ebp.h, 63
EBP_NODE_SPECS_MAX_SIZE
ebp.h, 63
EBP_OP_ALLOC
ebp.h, 65
ebp_op_alloc
ebp.c, 134
ebp.h, 68
ebp_op_alloc_args, 18
buffer_id, 18
life_time, 18

size, 18
EBP_OP_IDS
ebp.h, 64
EBP_OP_READ
ebp.h, 65
ebp_op_read
ebp.c, 135
ebp.h, 69
ebp_op_read_args, 19
dst_buffer_id, 19
dst_offset, 19
size, 19
src_buffer_id, 20
src_offset, 20
ebp_op_t
ebp.h, 64
EBP_OP_WRITE
ebp.h, 65
ebp_op_write
ebp.c, 136
ebp.h, 70
ebp_op_write_args, 20
buff_id, 20
offset, 20
size, 21
ebp_ops_init
ebp.c, 137
ebp.h, 71
ebp_packet_type
ebp.c, 142
ebp_register_node
ebp.c, 137
ebp.h, 71
ebp_register_op
ebp.c, 138
ebp.h, 72
ebp_remote_alloc
ebp.c, 138
ebp.h, 72
ebp_remote_read
ebp.c, 139
ebp.h, 73
ebp_remote_write
ebp.c, 139
ebp.h, 73
expires
eba_buffer, 6

header
ebp.h, 77
ebp_discover_ack, 13
ebp_discover_req, 14
ebp_invoke_ack, 16
ebp_invoke_req, 17

id
node_info, 23
iid
ebp.h, 77
ebp_invoke_req, 17
invoke_tracker, 22
include/eba.h, 25, 29
include/eba_internals.h, 30, 39
include/eba_net.h, 40, 48
include/eba_user.h, 48, 53
include/eba_utils.h, 53, 60
include/ebp.h, 60, 79
INVALID_NODE_ID
ebp.c, 131
INVOKE_COMPLETED
ebp.h, 65
INVOKE_DEFAULT
ebp.h, 65
INVOKE_FAILED
ebp.h, 65
INVOKE_IN_PROGRESS
ebp.h, 65
INVOKE_QUEUED
ebp.h, 65
INVOKE_STATUS
ebp.h, 65
invoke_tracker, 21
done, 22
iid, 22
pid, 22
status, 22
invoke_tracker_array_init
ebp.c, 140
ebp.h, 74
invoke_trackers
ebp.c, 142

life_time
eba_alloc_data, 5
eba_remote_alloc, 8
ebp.h, 77
ebp_op_alloc_args, 18
LIST_HEAD
eba_internals.c, 97
local_specs
ebp.c, 143
lock
eba_buffer, 6

mac
eba_remote_alloc, 8
eba_remote_read, 9
eba_remote_write, 11
node_info, 23
main
user.c, 149
MAX_INVOKE_COUNT
ebp.h, 63
MAX_NODE_COUNT
ebp.h, 63
MAX_OP_COUNT
ebp.h, 63
MEMORY_POOL_SIZE

eba_internals.h, 31
MODULE_AUTHOR
eba.c, 84
MODULE_DESCRIPTION
eba.c, 84
module_exit
eba.c, 85
module_init
eba.c, 85
MODULE_LICENSE
eba.c, 85
MODULE_VERSION
eba.c, 85
msgType
ebp.h, 77
ebp_header, 15
mtu
ebp.h, 77
ebp_discover_req, 14
node_info, 23

node
eba_buffer, 7
node_info, 22
id, 23
mac, 23
mtu, 23
node_specs, 23
node_info_array_init
ebp.c, 140
ebp.h, 74
node_infos
eba_utils.c, 126
ebp.c, 143
node_specs
node_info, 23
nodes_count
eba_utils.c, 126
ebp.c, 143

off
eba_rw_data, 12
offset
eba_remote_write, 11
ebp.h, 78
ebp_op_write_args, 20
op_entries
ebp.c, 143
op_entry, 24
op_id, 24
op_ptr, 24
op_entry_array_init
ebp.c, 141
ebp.h, 75
op_id
op_entry, 24
op_ptr
op_entry, 24
open_eba_device
eba_user.c, 117
opid
ebp.h, 78
ebp_invoke_req, 17

payload
eba_remote_write, 11
pid
invoke_tracker, 22
print_node_infos
ebp.c, 141
ebp.h, 75
print_op_entries
ebp.c, 142
ebp.h, 76

send_discover_ack_packet
eba_net.c, 106
eba_net.h, 44
send_discover_req_packet
eba_net.c, 107
eba_net.h, 45
send_invoke_ack_packet
eba_net.c, 107
eba_net.h, 45
send_invoke_req_packet
eba_net.c, 108
eba_net.h, 46
send_raw_ethernet_packet
eba_net.c, 109
eba_net.h, 47
size
eba_alloc_data, 5
eba_buffer, 7
eba_remote_alloc, 8
eba_remote_read, 9
eba_remote_write, 11
eba_rw_data, 12
ebp.h, 78
ebp_op_alloc_args, 18
ebp_op_read_args, 19
ebp_op_write_args, 21
src/eba.c, 81, 86
src/eba_internals.c, 89, 98
src/eba_net.c, 103, 110
src/eba_user.c, 113, 118
src/eba_utils.c, 120, 126
src/ebp.c, 130, 143
src_buffer_id
eba_remote_read, 10
ebp.h, 78
ebp_op_read_args, 20
src_offset
eba_remote_read, 10
ebp.h, 78
ebp_op_read_args, 20
status
ebp.h, 78
ebp_invoke_ack, 16

invoke_tracker, 22

test_eba_export_node_specs
eba_utils.c, 123
eba_utils.h, 57
test_eba_utils_buf_to_file
eba_utils.c, 124
eba_utils.h, 58
test_eba_utils_file_to_buf
eba_utils.c, 125
eba_utils.h, 59
type
eba_buffer, 7

user.c
main, 149
user_addr
eba_rw_data, 12
user_space/user.c, 149, 150